UiPath Principal Applied Scientist

Onsite interview question bank

01. UiPath Principal Applied Scientist Loop: Format, ML System Design Focus, Research Presentation and the Principal Bar

- Format notes
- Questions
- References

02. Research Presentation Round: Presenting Your ML System, and the Design Probes Principal Scientists Get Afterwards

- Format notes (inferred, not confirmed)
- Questions
- References

03. ML/LLM Science Depth for System Design: Agents, Document Understanding, Screen Understanding, NLP and Evaluation

- Format notes (inferred, not confirmed)
- Questions
- References

04. Applied Science System Design: Agentic Automation, Computer-Use Agents, Document Understanding, Communications Mining, Process Mining and Orchestration

- Format notes for this round
- A. Agentic automation: agents + robots + humans
- B. Computer-use agents (ScreenAgent-style UI automation)
- C. Document Understanding and IXP pipelines
- D. Communications Mining (email, chat and ticket understanding)
- E. Process mining and task mining
- F. Autopilot (natural-language assistance across the platform)
- G. Agent Builder, coded agents and agent evaluation
- H. Maestro-style orchestration of long-running agentic processes
- I. Evaluation, reliability and quality across the platform
- J. Trust, safety, governance and deployment constraints
- K. Cost, latency and model strategy
- L. Principal-level framing and roadmap probes
- References

05. Coding and ML Coding for System Design Components: Building the Pieces of Document AI, Agent and Retrieval Systems

- Format notes (inferred, not confirmed)
- Questions
- References

06. Principal-Level Leadership Through ML System Design: Technical Strategy, Design Reviews, Influence, Research-to-Product and Roadmaps

- Format notes (inferred, not confirmed)
- Questions
- References

07. UiPath AI Context: DocPath and Document Understanding, the LLM Gateway and AI Trust Layer, Agentic Automation Strategy and the Peak Acquisition

- Format notes for this round
- 1. Company AI strategy and "why UiPath"
- 2. DocPath and Document Understanding (DU)
- 3. CommPath, Communications Mining and IXP
- 4. LLM Gateway and AI Trust Layer
- 5. Agentic automation: Agent Builder, Maestro, Context Grounding and computer use
- 6. Peak acquisition and decision intelligence

7. UiPath research and publications
8. Cross-cutting ML system design probes on UiPath's stack
9. Questions you should be ready to ask the interviewer
References

08. End-to-End ML System Design: A PDF Tagging System (Structure, Accessibility and Document Understanding)
Format notes for this round
A. Problem framing, users and success metrics
B. Ingest, routing and native vs scanned detection
C. OCR and the perception layer
D. Layout analysis and reading order
E. Logical structure tagging: headings, lists, tables, figures, key-value pairs, spans
F. Tag taxonomy and schema
G. Tagged-PDF and PDF/UA output
H. Confidence, calibration and human review
I. Annotation and labelling tooling
J. Versioned label store and data management
K. Evaluation
L. Serving at scale
M. Integration with UiPath products and agents
N. Research depth and open problems
O. Principal-level scoping, roadmap and trade-offs
References

09. Cross-Portal ML System Design Bank: Peer-Company Prompts with Block Diagrams and UiPath Recasts
The tag system (read this first)
Format notes for this round
A. Search and ranking
B. Recommendation
C. Anomaly and fraud detection
D. RAG enterprise assistants and agents
E. Content moderation and safety
F. LLM and agent evaluation platforms
G. Serving, cost, data and platform (cross-cutting)
H. Classic NLP and vision prompts that recast directly
I. Principal-level stress probes (apply to any prompt above)
Coverage summary
References

10. ML Platform and MLOps System Design: Data Flywheel, Model Lifecycle, Capacity and Rollout (with Diagrams)
Format notes for this round
A. Training-data pipelines and labelling operations
B. Feature stores and embedding stores
C. Model registry and versioning (AI Center-style)
D. CI/CD for models and prompts
E. Drift and data-quality monitoring for document and agent models
F. Per-tenant fine-tuning and adapter management
G. GPU training and inference capacity planning
H. On-prem, air-gapped and multi-region rollout
I. Rollback and incident response for model regressions
J. Principal-level integrative prompts
References

01. UiPath Principal Applied Scientist Loop: Format, ML System Design Focus, Research Presentation and the Principal Bar

How to read this chapter. You asked for ML system design, so this chapter centres on it. It explains where the ML system design round sits in the UiPath Principal Applied Scientist loop, what the principal-level bar looks like in that round, and the process-level questions you are likely to get there. The research presentation and values rounds are covered only as far as they connect to system design. Diagrams are plain-text block diagrams, so they render the same in Markdown and in a PDF export.

Tagging system. Every question carries one of two tags:

- **[Reported] [n]**: a candidate report, question bank or official page links this question, or this round type, to UiPath. The number points to the References list.
- **[Likely]**: an inference from the role, the level and UiPath's AI products (agentic automation, Document Understanding, Communications Mining, Autopilot, the AI Trust Layer, computer vision for UI automation). No candidate has reported it.
- Indented sub-bullets are follow-up probes an interviewer may use to push a strong first answer toward the principal bar.

Research limits (please read). This run could not do any live research. The shared web-search budget for this workflow was already used up before the chapter started. Direct fetches of glassdoor.com, teamblind.com and uipath.com also failed, because the egress proxy rejected them and DNS lookups for WebFetch failed. So **no question below is tagged [Reported]**: none could be checked against a candidate write-up, and this chapter does not invent reports. Every question is **[Likely]**. The format notes describe the usual principal-scientist loop at enterprise AI companies of UiPath's size and shape; they are not a confirmed UiPath process. Confirm round names, durations and the presentation format with your recruiter. If you need sourced reports, a follow-up run with search budget can upgrade tags from [Likely] to [Reported].

Review pass (2026-10-14). An adversarial review re-checked this chapter. Web search was again unavailable (the shared search budget was used up). Direct fetches of 12 candidate source pages were tried: UiPath careers, hiring process, blog, product and documentation pages, Glassdoor, Blind, Levels.fyi and Interview Query. Every fetch failed with a DNS error or a proxy 403, so no source could be verified and no question was upgraded to [Reported]. The review removed two

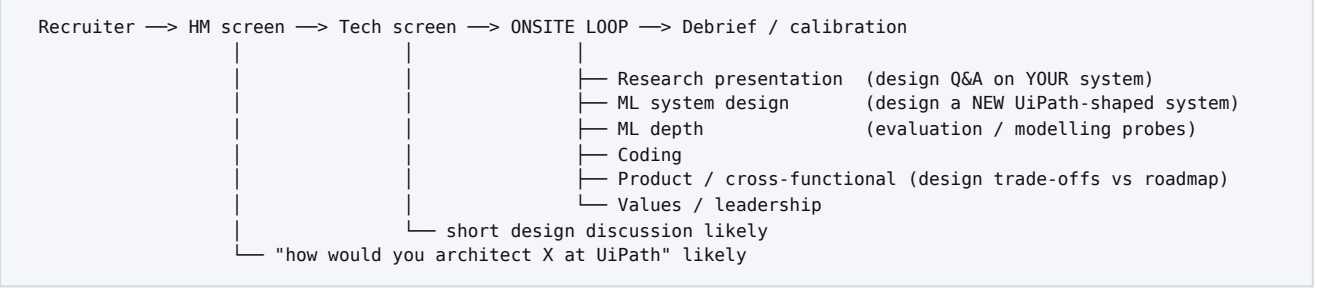
> generic leadership questions that did not test system design (they are covered in chapter 06). It added 21 [Likely] principal-level design prompts on gaps: capacity and cost estimation, idempotent agent actions, process versioning, agent tracing, computer-use evaluation harnesses, multi-agent attribution, data flywheels and tenant isolation, synthetic document data, rule-validated extraction, PII redaction, cross-source retrieval, red-teaming, model retirement, cost attribution, judge drift, delayed-outcome experiments, on-device inference, caching and platform consolidation.

Format notes

Typical loop shape (inferred, not confirmed)

Stage	Typical duration	Interviewer	Status	Principal bar assessed
Recruiter screen	30 min	Recruiter	Likely	Scope match (agents, document AI, comms mining, CV for UI), level calibration, compensation
Hiring manager screen	45-60 min	Director or Head of AI/ML	Likely	Track record of research shipped into product; ability to set direction for a team
Technical screen	45-60 min	Senior or staff scientist	Likely	ML/LLM depth plus a short design discussion or ML coding
Research / technical presentation	60 min (about 30-40 min talk plus Q&A)	Panel of scientists, engineers and a product lead	Likely	Depth, rigour, and whether you can turn research into a product system
ML system design	60 min	Principal scientist or architect	Likely	End-to-end design of an LLM or agentic system with evaluation, safety, cost and rollout
ML depth / applied science	60 min	Staff or principal scientist	Likely	Modelling choices, evaluation methodology, failure analysis
Coding (ML or general)	45-60 min	Engineer or scientist	Likely	Clean, working code; less weight than at junior levels
Cross-functional / product	45 min	PM or engineering leader	Likely	Turning customer automation problems into ML roadmaps
Values / leadership (Humble, Bold, Immersed, Fast)	45 min	Senior leader	Likely	Influence without authority, judgement, pace, ownership

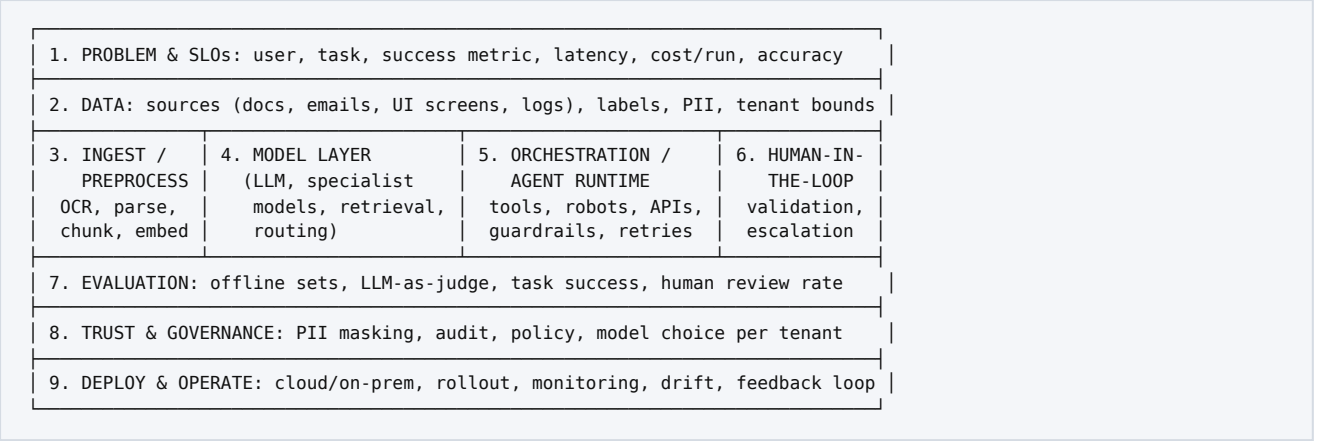
Where ML system design shows up



System design probably comes up in three places, not one. You may be asked to (a) re-architect your own past system during presentation Q&A, (b) design a new system in the dedicated round, and (c) defend design trade-offs against product constraints in the cross-functional round.

Canvas for the design round (a whiteboard layout, not an answer)

Use this layout to structure any prompt below, so the interviewer can see every principal-level concern on the board.



What the principal bar likely means in design rounds

- **Problem framing before architecture.** You turn a vague automation goal into measurable SLOs: straight-through processing rate, human review rate, cost per document or per agent run, latency, and error cost.
- **Mix of deterministic and probabilistic parts.** UiPath's core is deterministic RPA robots. A principal candidate is expected to decide what belongs in a model, what belongs in a rule or a robot, and where people approve or correct the result.
- **Evaluation as a first-class part of the system.** Benchmarks, regression suites, LLM-as-judge calibration, and online metrics that tie back to customer ROI.
- **Enterprise constraints.** Multi-tenant isolation, PII and data residency, on-prem and air-gapped deployments, customer choice of model provider, and auditability.
- **Platform thinking.** You design a reusable capability (for example, one extraction or agent-evaluation framework) that many product teams and customers can use, not a one-off model.
- **Technical leadership.** You can explain the roadmap, the staffing, what you would build first, and how you would know to stop or pivot.

Questions

A. Process and expectations for the design round

1. [Likely] Walk me through how you approach a 60-minute ML system design interview, and what you clarify in the first ten minutes.
    - Which requirements would you refuse to assume, and ask about instead?
    - How do you decide when to go deep on one component versus staying broad?

2. [Likely] At principal level, what do you think separates a good system design answer from a great one?
    - Can you give an example from your own work where the design choice mattered more than the model choice?
3. [Likely] How would you design a system when the interviewer gives you no numbers? What numbers would you estimate yourself?
    - How would you size cost per agent run or per document for an enterprise customer processing 1M documents a month?
4. [Likely] How do you draw the boundary between what an LLM should do and what a deterministic robot, rule or API call should do?
    - Give a case where you moved logic out of a model and into deterministic code. What did it buy you?
5. [Likely] What does "production ready" mean for an ML system you own, and what checklist would you use before launch?
    - Who signs off, and on what evidence?
6. [Likely] How do you present trade-offs to an interviewer who keeps changing the constraints mid-design?
    - What would you re-do on the board if latency suddenly had to be 10x lower?
7. [Likely] Do a back-of-envelope estimate on the board: 50,000 agent runs a day, about 12 LLM steps per run, about 3,000 input tokens per step. What are tokens per day, peak requests per second and monthly cost at two price tiers?
    - Which assumption dominates the answer, and how would you check it with the customer?
8. [Likely] In the last ten minutes the interviewer asks, "What breaks first at 10x scale?" How do you structure that answer for the system on your board?

B. Agentic automation system design (UiPath-shaped prompts)

9. [Likely] Design an agentic automation platform that lets enterprise users build agents that call tools, trigger RPA robots and hand off to people.
    - How do agents, robots and people share state across a long-running process?
    - Where do you put guardrails, and what happens when a tool call fails halfway through?
10. [Likely] Design an orchestration layer that coordinates several agents and robots across a multi-step business process (for example, order-to-cash).

- How do you make the process recoverable and auditable after a crash?
- How do you stop an agent from looping forever or spending too much?

11. [Likely] Design an evaluation system for enterprise agents whose prompts, tools and underlying LLMs change often.

- How do you build test sets when customer data is private and differs per tenant?
- How would you calibrate an LLM-as-judge against human labels, and what agreement level would you accept?

12. [Likely] Design a system that turns a natural-language description of a task into a working automation workflow.

    - How do you check the generated workflow before it runs on production systems?
    - What is your fallback when generation confidence is low?
13. [Likely] Design an agent that operates desktop and web applications from screenshots, without selectors or APIs.
    - How do you combine a vision-language model with UI element detection and existing selectors?
    - How do you measure task success, and how do you make it safe to run against live systems?
14. [Likely] Design a self-healing system for UI automations that break when a target application's interface changes.
    - How do you detect breakage before customers report it?
    - When should the system repair the automation itself, and when should it ask a developer?
15. [Likely] Design a tool-selection and routing layer for an agent that has hundreds of enterprise connectors available.
    - How do you keep the prompt small while keeping recall high on the right tool?
    - How would you evaluate routing errors separately from execution errors?
16. [Likely] Design memory and context management for long-running agents that work on a case over days.
    - What do you store, what do you summarise, and what must never be stored?
17. [Likely] How would you design human-in-the-loop escalation so agents ask for help at the right rate?
    - How do you set confidence thresholds per action type and per customer?
    - How do reviewer corrections flow back into the system?
18. [Likely] Design a cost-and-latency-aware model router that picks between small, large, hosted and self-hosted models per step.
    - What signals decide the route, and how do you stop quality from silently dropping?
19. [Likely] Design idempotency and compensation for agent actions that write to systems of record such as an ERP or CRM.
    - How do you handle a tool call that timed out but may have succeeded?
    - When would you use a saga with compensating actions instead of a human approval step?
20. [Likely] Design a long-running process orchestrator in which modelled process steps mix agent steps, robot steps and human tasks.
    - How do you ship a new process version while thousands of instances of the old one are still running?
21. [Likely] Design tracing and observability for agent runs.
    - What goes into a span, and how do you make a non-deterministic LLM run replayable for debugging?
    - How do you strip PII from traces and still keep them useful?
22. [Likely] Design an evaluation harness for computer-use agents that work in enterprise applications (ERP, CRM, legacy desktop apps).
    - How do you build sandboxed replicas, reset their state between runs and score partial task completion?
23. [Likely] Design a multi-agent system where a planner hands work to specialist agents.

    - When is multi-agent worse than one agent with tools, and how do you attribute a failure to the right agent?
24. [Likely] Design a data flywheel that turns agent traces, reviewer corrections and business outcomes into evaluation and training data.
    - How do you guarantee one tenant's data never improves, or leaks into, another tenant's model?

C. Document AI and communications system design

25. [Likely] Design an intelligent document processing system that extracts fields from invoices, contracts and forms across many layouts and languages.
    - Where do OCR, layout models, specialised extractors and LLMs each fit?
    - How do you produce calibrated field-level confidence for straight-through processing?
26. [Likely] Design a system that lets a customer build a new document extractor with ten labelled examples.
    - How do you choose between prompting, few-shot retrieval and fine-tuning?
    - How do you tell the customer whether the extractor is good enough to go live?
27. [Likely] Design a document classification and splitting pipeline for multi-document PDFs that arrive in one email.
    - How do you handle page-order errors and mixed scans?
28. [Likely] Design a communications mining system that finds intents, entities and sentiment in high-volume enterprise email and tickets, then triggers automations.
    - How do you discover new intents nobody labelled yet?
    - How would you handle taxonomy drift when the business changes?
29. [Likely] Design an active learning loop for document and message models where every label costs expert time.
    - Which sampling strategy would you use, and how would you show it beats random labelling?
30. [Likely] Design a table extraction system for complex financial documents with nested and multi-page tables.
    - How do you evaluate table structure accuracy as well as cell values?
31. [Likely] How would you decide whether a document understanding product should use one general foundation model or many task-specific models?
    - What would the cost, latency and accuracy data need to show to change your mind?
32. [Likely] Design a synthetic data pipeline (layouts, handwriting, poor scans) to bootstrap extractors for a new document type or language.
    - How do you show that synthetic data helps on the real distribution and is not just inflating offline scores?
33. [Likely] Design schema-constrained LLM extraction with business-rule validation (line items sum to the total, dates are consistent, vendor matches master data).
    - How should rule violations change field-level confidence and routing to human review?
34. [Likely] Design support for handwriting, low-resource languages and right-to-left scripts in a document understanding product.
    - How do you decide which languages get dedicated models and which rely on a general model?

35. [Likely] Design a PII detection and redaction model that runs before any document or email reaches an external LLM.
    - How do you measure its recall, and what happens to a request when recall is uncertain?

D. Retrieval, grounding and enterprise knowledge

36. [Likely] Design a retrieval-augmented system that answers questions over a customer's processes, documents and automation logs.
    - How do you enforce per-user access rights during retrieval?
    - How do you measure groundedness and catch answers that sound plausible but are wrong?
37. [Likely] Design an assistant that helps automation developers write and debug workflows inside the studio IDE.
    - What context do you retrieve from the project, and how do you keep it fresh?
    - How do you measure developer productivity gains rather than suggestion acceptance alone?
38. [Likely] Design a process mining plus ML system that recommends which processes a customer should automate next.
    - How do you estimate automation ROI from event logs and task-mining data?
    - How do you avoid recommending processes that are frequent but too variable to automate?
39. [Likely] Design an embedding and indexing service that many product teams share.
    - How do you version embeddings when the model changes, without re-indexing every tenant at once?
40. [Likely] Design question answering over semi-structured enterprise sources (spreadsheets, ticket histories, process definitions) where the answer needs a join across sources.
    - When do you use text-to-SQL or tool calls instead of vector retrieval?

E. Trust, safety, governance and enterprise deployment

41. [Likely] Design an AI trust layer that sits between UiPath products and any LLM provider.
    - Where do PII masking, prompt-injection defence, logging and policy enforcement run?
    - How do you prove to an auditor what the model saw and what it did?
42. [Likely] How would you defend an agent that reads untrusted emails and documents against prompt injection that tries to trigger robot actions?
    - Which actions need human approval no matter what the model's confidence is?
43. [Likely] Design an ML system that must run in the cloud, on-prem and in air-gapped customer environments.
    - What changes in model choice, updates and monitoring for each?
44. [Likely] How would you support customers who bring their own LLM, when your evaluation and prompts were tuned on a different model?
    - How do you certify a new model and roll it out per tenant?
45. [Likely] Design monitoring for model drift and quality regression when you cannot look at customer data.

    - Which privacy-preserving signals would you rely on?
46. [Likely] How would you design data governance for training on customer data that some tenants allow and others forbid?
    - How would you trace which data trained which model version?
47. [Likely] Design an audit and explainability feature for an agent that made a financial decision inside an automated process.
48. [Likely] Design a red-teaming and guardrail evaluation programme for an agent platform.
    - How do you weigh the cost of guardrail false positives (blocked legitimate work) against missed attacks?
49. [Likely] An LLM provider will retire a model that customers' production agents depend on in 90 days. Design the migration process.
    - How do you find every affected agent, re-certify behaviour and handle customers who refuse to move?
50. [Likely] Design per-tenant cost attribution and budget enforcement for LLM usage across several products.

F. Evaluation, experimentation and rollout

51. [Likely] How would you build an offline benchmark for agentic tasks that predicts customer-facing success?
    - How do you stop the benchmark from going stale or leaking into training?
52. [Likely] Design an A/B or staged rollout for a new extraction or agent model when each customer has a small, different workload.
    - What would you do when the aggregate metric improves but a large customer gets worse?
53. [Likely] Which online metrics would you put on the executive dashboard for an agent product, and which would you keep internal?
54. [Likely] How do you set and defend a "good enough to automate" threshold with a customer whose error tolerance differs from yours?
55. [Likely] Design a regression testing pipeline that runs every time a prompt, tool schema or model version changes.
    - How do you handle non-determinism in LLM outputs during testing?
56. [Likely] How would you measure whether human reviewers are a reliable gold standard?
57. [Likely] Your LLM-as-judge model is upgraded by its provider. How would you detect that its scores have shifted, and what would you do about historical comparisons?
58. [Likely] Design an online experiment for an agent whose true outcome (for example, an invoice paid correctly) is known only weeks later.
    - Which proxy metrics would you use, and how would you validate them against the delayed outcome?

G. Scaling, efficiency and serving

59. [Likely] Design LLM serving for millions of short agent steps a day with strict cost targets.
    - Where do caching, batching, distillation and quantisation help, and where do they hurt?

60. [Likely] How would you distil a large model into a small specialised model for a high-volume document or classification task?
    - How do you decide the small model is safe to replace the large one?
61. [Likely] Design a fine-tuning platform that adapts base models per customer without a full model copy per tenant.
    - How would you serve many adapters efficiently?
62. [Likely] How do you plan GPU capacity for a product whose usage is spiky and tied to customers' business hours?
63. [Likely] What would you change in your design if per-request latency must drop below one second for an attended (user-facing) automation?
64. [Likely] Design inference for a screen-understanding model that must run on customer desktops with CPU only, for attended robots.
    - What do you trade among model size, accuracy and startup time, and how do you ship model updates?
65. [Likely] Design caching for agent steps (prompt-prefix caching, semantic caching of LLM answers and tool results).
    - How do you prevent stale answers and cross-tenant cache hits?

H. Research presentation: design Q&A on your own system

66. [Likely] Present a system you designed end to end. What was the hardest design decision, and what did you reject?
    - If you rebuilt it today with current LLMs, what would you delete?
67. [Likely] Draw the architecture of your presented system and mark where it failed in production.
    - How did you find out, and what did you change in the design?
68. [Likely] How did you go from a research result to something a product team could own and operate?
    - What did you hand over: code, eval suite, runbooks, people?
69. [Likely] Which baseline did you compare against, and why should we believe the comparison was fair?
70. [Likely] What is the single number that best describes the impact of your system, and how was it measured?
71. [Likely] How would your presented system map onto a UiPath product such as agents, document understanding or communications mining?
    - What would need to change for multi-tenant enterprise deployment?
72. [Likely] What part of your presented work would you publish, and what would you keep as product advantage?

I. Principal-level scope and technical leadership in design

73. [Likely] Describe a time you set the technical direction for an ML platform used by several teams.
    - How did you get teams that did not report to you to adopt it?
74. [Likely] How would you write a 12-month applied science roadmap for agentic automation at UiPath?
    - What would you build, buy or partner on, given fast-moving foundation models?

75. [Likely] How do you decide when to invest in a custom model versus using a frontier API?
    - What would trigger you to reverse that decision?
76. [Likely] Tell me about a design review where you blocked or significantly changed a launch.
    - How did you handle the disagreement with product or engineering?
77. [Likely] How do you review another scientist's system design, and what do you look for first?
78. [Likely] Describe a system you designed that turned out to be the wrong bet. How early did you notice, and what did you do?
79. [Likely] You inherit three teams, each with its own document extraction stack. Design the consolidation plan and the metrics that tell you it worked.
    - What would you keep separate on purpose?

J. Values lens on design decisions (Humble, Bold, Immersed, Fast)

80. [Likely] **Humble:** Tell me about a design of yours that a junior colleague or a customer improved.
    - What did you change in how you run design reviews afterwards?
81. [Likely] **Humble:** When did your evaluation show that your favourite approach was worse than a simple baseline?
82. [Likely] **Bold:** Describe a time you proposed an architecture that most people thought was too ambitious.
    - How did you reduce the risk while keeping the ambition?
83. [Likely] **Bold:** What design bet would you make for UiPath's agentic platform that others might disagree with?
84. [Likely] **Immersed:** How did you learn the customer's real workflow before designing an ML system for it?
    - Have you sat with operators or reviewers who used your system? What did you learn?
85. [Likely] **Immersed:** Tell me about a design detail you only got right because you read the raw data or logs yourself.
86. [Likely] **Fast:** Describe a time you shipped a smaller version of a system early to learn quickly.
    - What did you deliberately leave out of v1, and how did you protect quality?
87. [Likely] **Fast:** How do you keep moving fast when the foundation models underneath your system change every few months?

K. Questions you should be ready to ask (reverse questions)

88. [Likely] What does the ML system design round expect from a principal candidate compared with a staff candidate?
89. [Likely] Is the research presentation on my own work, a given topic, or a take-home design?
90. [Likely] Which UiPath product area (agents, document AI, communications mining, CV for UI automation, process intelligence) would this role design for first?
91. [Likely] How do applied scientists and platform engineers share ownership of production systems here?

92. [Likely] How does UiPath evaluate agents today, and what is the biggest gap?
93. [Likely] What constraints from on-prem and regulated customers most shape model and architecture choices?

References

No source pages could be retrieved in this run (see "Research limits" above), so no question cites a source. The official pages below are the right starting points for checking the format notes and product context. They are listed for verification and were not read in this run. A fetch of each was tried again on 2026-10-14 and blocked by the network (DNS failure or proxy 403), so whether they are live could not be confirmed. Both are long-standing official UiPath URLs and are kept only as starting points; no question relies on them.

1. UiPath Careers, uipath.com (official careers page; not retrieved in this run). https://www.uipath.com/company/careers
2. UiPath Blog, uipath.com (official product and AI announcements; not retrieved in this run). https://www.uipath.com/blog

02. Research Presentation Round: Presenting Your ML System, and the Design Probes Principal Scientists Get Afterwards

How to read this chapter. You asked for ML system design, so this chapter treats the research or job-talk presentation as a system design round you lead yourself. It covers how to structure the talk around architecture diagrams, and the probing questions a panel is likely to ask about your system's design, evaluation, deployment and transfer to UiPath products. General research-talk questions (publication history, theory) are included only where they lead into design.

Tagging system (the same as Chapter 01):

- **[Reported] [n]**: a candidate report, question bank or official page links the question, or this round type, to UiPath. The number points to the References list.
- **[Likely]**: an inference from the role, the principal level and UiPath's AI products (agentic automation, Document Understanding, Communications Mining, Autopilot, the AI Trust Layer, computer vision for UI automation, process mining). No candidate has reported it.
- Indented sub-bullets are follow-up probes, phrased as questions.

Research limits (please read). Two runs have now tried to source this chapter, and neither could do live research. In the first run the workflow's shared web-search budget (200 searches per turn) was already used up. In the second (review) run the search budget was again exhausted, and direct fetches of glassdoor.com, teamblind.com, uipath.com, docs.uipath.com and arxiv.org all failed with DNS errors (ENOTFOUND). No cited URL could be verified, and no new candidate-reported question could be found. So **no question below is tagged [Reported]**. Every question is **[Likely]**, and the format notes describe how principal-scientist job talks usually run at enterprise AI companies; they are not a confirmed UiPath process. Ask your recruiter for the presentation format, the length and who sits on the panel. A follow-up run with working search could upgrade tags to [Reported].

Review changes. The review run removed generic and duplicate questions (for example, separate PII, mentoring and "what would you do differently" questions that repeated others). It also added principal-level stress tests that probe specific arrows on your architecture diagram (Section L) and moved general agent-design questions that Chapter 04 already covers out of this chapter.

Format notes (inferred, not confirmed)

Item	Typical for principal applied scientist job talks	Status
Duration	60 min: about 30-40 min talk, 20-30 min Q&A; some companies interrupt during the talk	Likely
Topic	Your own past work, usually one or two projects that reached production; sometimes a "vision for X at our company" section	Likely
Audience	Panel of 3-8: scientists (staff/principal), ML or platform engineers, a product lead, often the hiring manager	Likely
Materials	Slides shared in advance or presented live; confidential details abstracted	Likely
Debrief weight	Often a heavy signal, because every panel member sees the same material	Likely

What the principal bar likely assesses in this round

- **Depth and rigour.** Sound baselines, ablations and error analysis; you know where the method breaks.
- **Research-to-product conversion.** A clear path from a result to a system that others operate, with SLOs, costs and monitoring.
- **System design ownership.** You can draw the architecture, defend each box and say what you would change today.
- **Scope and influence.** The work changed a roadmap, a platform or several teams, not only one model.
- **Communication.** Mixed audiences (scientists, engineers, product) all follow the talk.
- **Transfer.** You can map the work onto UiPath problems: agents that call robots and tools, document and message understanding, UI vision, enterprise trust and governance.

Suggested talk structure built around system diagrams

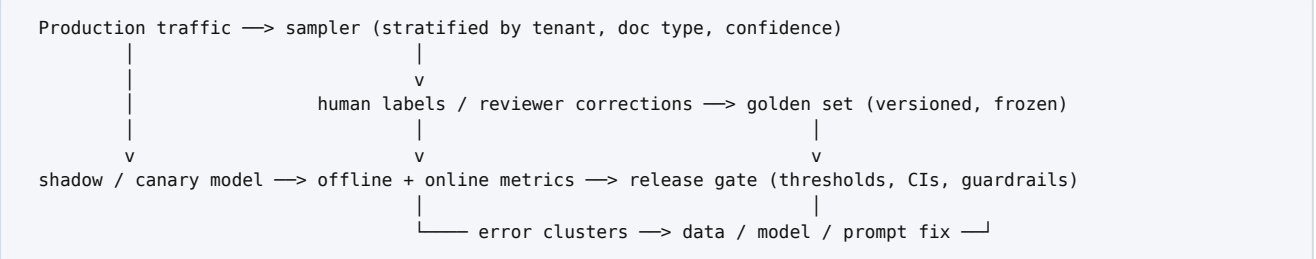
```
┌──────────────────────────────────────────────────────────────────────┐
| 1. Problem & business stakes      (2-3 min)  who hurt, how much, SLOs     |
| 2. Why it was hard                (3 min)    data, scale, constraints      |
| 3. SYSTEM ARCHITECTURE DIAGRAM    (5-7 min)  boxes, data flow, owners      |
| 4. Key technical idea(s)          (8-10 min) the research contribution     |
| 5. Evaluation                     (5 min)    offline -> online -> business |
| 6. Production & failures          (4 min)    what broke, what you changed  |
| 7. Impact & scope                 (2 min)    metrics, teams, roadmap       |
| 8. If I did it at UiPath          (3 min)    mapping + next 12 months      |
└──────────────────────────────────────────────────────────────────────┘
```

One architecture slide the panel will probe (template)

```
Data sources ──> Ingest / labelling ──> Training & eval ──> Model registry
     |                 |                     |                  |
     |            (active learning)     (offline bench)         v
     |                 ^                                  Serving / agent runtime ──> Robots / tools / APIs
     |                 |                                        |
     └─── feedback ────┴─── human review / corrections <─────────┤
                                                              v
                                    Monitoring: quality, drift, cost, latency, audit log
```

Expect probes at every arrow: who owns it, what fails there, how you measure it, and what it costs.

Second slide worth preparing: evaluation and feedback loop



Panels at this level often skip the model and ask about this loop: how the sample is drawn, how reviewer corrections are de-biased, and what stops a release.

Questions

A. Framing and problem selection

1. [Likely] Why did you choose this project for your presentation, and what does it show about you at principal level?
    - What other project did you consider presenting, and why not that one?
2. [Likely] What was the business problem in one sentence, and who measured success?
    - What SLOs (accuracy, latency, cost per request, review rate) did you commit to?
3. [Likely] How did you decide this problem needed research rather than an off-the-shelf model?
    - What did the simplest baseline achieve, and how far was it from the target?
4. [Likely] What constraints (data access, privacy, latency, hardware, regulation) shaped the design most?
    - Which constraint did you push back on, and did you win?

B. Architecture walk-through

5. [Likely] Walk us through the architecture diagram box by box. Which box did you design yourself?
    - Which box took the most engineering time, and was that expected?
6. [Likely] Where is the single point of failure in this design?
    - What happens to users when that component goes down?
7. [Likely] Which parts are learned models and which are deterministic code or rules, and why did you split them that way?
    - Did you ever move logic from a model into code, or the reverse?
8. [Likely] How does data flow from raw input to the final action, and where is it stored along the way?
    - Where does personally identifiable information appear, and how is it protected?
9. [Likely] What alternative architectures did you reject, and what evidence decided it?
10. [Likely] How did the system handle inputs far outside the training distribution?
    - How did it signal low confidence to the downstream system or user?
11. [Likely] What were the interfaces between your system and the teams upstream and downstream?
    - Who could change those interfaces, and how did you version them?

12. [Likely] Draw how the architecture would look if traffic grew 100x. What breaks first?
13. [Likely] How did you handle retries, idempotency and partial failure when the model triggered actions in other systems?

C. Modelling and the core technical contribution

14. [Likely] What is the one technical idea in this work that someone else would not have tried?
    - Is it new in the literature, or new in how you applied it?
15. [Likely] Why this model family? What would a large general LLM have done on the same task?
    - Did you test prompting or fine-tuning a foundation model as a baseline?
16. [Likely] Walk us through the loss function or training objective, and why it fits the business metric.
17. [Likely] Which ablation surprised you most?
    - Which component could we remove with almost no loss?
18. [Likely] How did you get labels, and how noisy were they?
    - How did you measure label quality and annotator agreement?
19. [Likely] How much labelled data did you need, and how would that change for a new customer or domain?
    - Did you use active learning, weak supervision or synthetic data? How did you validate it?
20. [Likely] If you had to make the model ten times cheaper to serve, what would you do?
    - Distillation, quantisation, caching, smaller models per step: which did you try?

D. Evaluation rigour

21. [Likely] What was your offline evaluation set, and how do you know it represented production traffic?
    - How often was it refreshed, and how did you stop leakage into training?
22. [Likely] Which baselines did you compare against, and why should we believe the comparison was fair?
    - Did baselines get the same tuning budget as your method?
23. [Likely] Report the confidence interval or variance on your headline number.
    - Was the improvement statistically and practically significant?
    - How much of the gain survives a change of seed, data ordering or hyperparameters?
24. [Likely] How did offline gains translate to online metrics, and where did they not?
25. [Likely] How did you evaluate the parts of the output that have no single correct answer (for example, generated text or agent plans)?
    - If you used an LLM as a judge, how did you calibrate it against humans?
26. [Likely] Show us the error analysis. What are the top three failure categories?
    - Which failure is most costly to the customer, rather than most frequent?
27. [Likely] How did you evaluate fairness or performance across subgroups, languages or customers?
28. [Likely] What did the A/B test or staged rollout look like, and what guardrail metrics did you watch?
    - What would have made you stop the rollout?

E. Production, operations and failures

29. [Likely] What went wrong after launch that your offline evaluation did not predict?
    - How did you detect it, and how long did it take?
30. [Likely] How did you monitor model quality without ground-truth labels in production?
31. [Likely] How did you detect and handle data drift or concept drift?
    - Did you retrain on a schedule or on a trigger?
32. [Likely] What was the on-call or ownership model after launch, and who got paged?
33. [Likely] What did the system cost to run per month, and how did that compare to the value it created?
34. [Likely] How did you handle a rollback, and how fast could you do it?
35. [Likely] What did you hand over to the product or platform team: code, evaluation suite, runbooks, people?
    - Is the system still running without you?

F. Agentic and LLM-system follow-ups

36. [Likely] If you rebuilt this system today with current LLMs and agents, what would you delete and what would you keep?
37. [Likely] How would you turn this system into an agent that calls tools, triggers robots and asks people for help?
    - Which actions would always need human approval?
38. [Likely] How would you evaluate an agent version of your system whose prompts, tools and underlying model change often?
    - How would you build a regression suite that runs on every change?
39. [Likely] How would you route between small, large, hosted and self-hosted models for different steps?
    - How would you stop quality dropping silently when the router changes?
40. [Likely] What part of your system would you replace with a vision-language model reading screens, and what risk does that add?

G. Enterprise constraints and trust

41. [Likely] How would your system work for a multi-tenant enterprise product where each customer's data must stay isolated?
42. [Likely] How would you deploy this on-prem or in an air-gapped customer environment?
    - How would you update models and monitor quality there?
43. [Likely] How would the design change if customers bring their own LLM provider?
44. [Likely] How would you explain a decision your system made to an auditor or a regulator?
    - What would the audit log contain?
45. [Likely] How would you train or improve the model when some customers allow their data to be used and others forbid it?

H. Mapping your work to UiPath products

46. [Likely] Which UiPath product area would benefit most from your work: agents, document understanding, communications mining, UI automation or process mining?
    - What is the first experiment you would run there in your first 90 days?
47. [Likely] How would your extraction or classification approach handle invoices, contracts and forms across many layouts and languages?
    - How would it produce field-level confidence for straight-through processing?
48. [Likely] How would your approach help customers build a new extractor or classifier with only a handful of examples?
49. [Likely] How would your work apply to a system that finds intents in high-volume email and then triggers automations?
    - How would you handle new intents nobody labelled?
50. [Likely] How would your work apply to UI automations that break when the target application changes?
51. [Likely] How would you decide whether UiPath should build this capability, buy it, or rely on a foundation-model provider?
52. [Likely] What would make your approach a mistake for UiPath's customers?

I. Scope, influence and technical leadership

53. [Likely] What was your exact role, and what did others on the team do?
    - Which decisions would not have happened without you?
54. [Likely] How many teams adopted this work, and how did you get teams that did not report to you to use it?
55. [Likely] Describe a disagreement about this design with product or engineering. How was it resolved?
56. [Likely] What did this project change in your organisation's roadmap or platform?
57. [Likely] How did you balance publication goals with product deadlines on this project?
    - What did you publish, and what did you keep as product advantage?
58. [Likely] If you led this at UiPath, what would your 12-month roadmap look like, and how would you staff it?
59. [Likely] What would make you stop or pivot this line of work?

J. Reflection and judgement

60. [Likely] What is the weakest part of your work, and how would a sceptical reviewer attack it?
61. [Likely] Which result in your talk do you trust least?
62. [Likely] Tell us about a design decision where you were wrong and a junior colleague or customer was right. (Humble)
63. [Likely] What was the boldest bet in this project, and how did you reduce its risk? (Bold)
64. [Likely] What detail did you only get right because you looked at raw data or sat with users? (Immersed)
65. [Likely] What did you leave out of version one to ship faster, and did it hurt later? (Fast)

K. Presentation mechanics and clarification probes

66. [Likely] Can you go back to the architecture slide and explain the arrow between these two components?
67. [Likely] What does this metric mean in customer terms, for example hours saved or documents processed without review?
    - Can you explain the core idea to the product manager on the panel in two minutes, without jargon?
68. [Likely] Which numbers on your slides are from production, and which are from offline experiments?
69. [Likely] What part of this work is confidential, and can you describe it at a level that lets us judge it?

L. Stress tests on your architecture diagram (principal-level)

70. [Likely] Point to the arrow on your diagram where training-serving skew is most likely. How did you detect it, and what contract stops it recurring?
    - Were features or embeddings computed by the same code path offline and online?
71. [Likely] Your confidence threshold decides what goes to straight-through processing and what goes to a human. Derive that threshold from the cost of a false accept, the cost of a review and the reviewer capacity.
    - What happens to the threshold when a new customer has a different cost of error?
72. [Likely] Human corrections feed back into training on your diagram. How do you stop that loop from only learning from the cases the model already flagged as uncertain (selection bias)?
    - How do you sample cases the model was confident about but got wrong?
73. [Likely] If you swap the embedding model behind a retrieval index, what must be rebuilt, how long does it take at your scale, and how do you serve traffic during the migration?
74. [Likely] Which box on your diagram dominates p99 latency, and which dominates cost? Are they the same box?
    - What would you trade to halve p99 without raising cost?
75. [Likely] Draw the GPU or accelerator capacity plan for peak load. How do you handle a customer whose batch job arrives at the same time as interactive traffic?
76. [Likely] Where on your diagram would you add a cascade (cheap model first, expensive model on escalation), and how do you set the escalation rule without hurting recall on rare, costly cases?
77. [Likely] Which component's output is consumed by another model downstream? How do you stop an upstream model update from silently breaking the downstream one?
    - Who signs off when the upstream team ships a new version?
78. [Likely] How is each artefact on your diagram versioned (data snapshot, labels, model, prompt, config, index), and can you reproduce last quarter's production output exactly?
79. [Likely] Your golden set was frozen at launch. How do you know it still represents traffic a year later, and how do you refresh it without making old and new scores incomparable?
80. [Likely] Show where a single customer with unusual data (one tenant, 40% of volume) would distort your aggregate metrics. How do you report quality so that this is

visible?

81. [Likely] Which metric on your dashboard could improve while customers get worse outcomes? How did you guard against that?
82. [Likely] If one upstream data source silently changed its schema or encoding, which box would notice first, and how long would bad outputs flow before an alert?
83. [Likely] What is the blast radius of a bad model release on your diagram, and what mechanisms (canary, per-tenant rollout, kill switch) limit it?
84. [Likely] Which parts of your system would you keep if the core model were replaced tomorrow by a stronger general foundation model? Which parts are durable advantage?
85. [Likely] How would your diagram change if each customer needed its own fine-tuned variant? How do you train, store, serve and evaluate thousands of variants?
    - Adapters per tenant, prompt-only customisation, or shared model with retrieval: which and why?
86. [Likely] Where would an LLM step on your diagram need a deterministic validator (schema, business rule, cross-check against a system of record), and what happens when the validator rejects the output?
87. [Likely] Walk through one real incident on this system end to end: detection, diagnosis, mitigation and the permanent fix. Which box on the diagram changed afterwards?
88. [Likely] If you had to run this system with half the team, which boxes would you merge, outsource or delete?
89. [Likely] How would you prove to a sceptical panel that the research contribution, rather than more data or more compute, caused the headline improvement?
    - What is the compute-matched or data-matched baseline?
90. [Likely] What would a principal engineer on this panel say is over-engineered on your diagram, and how would you answer?
91. [Likely] Which design decision on this slide is hardest to reverse, and how much evidence did you have when you made it?

M. Questions to ask the panel (reverse questions)

92. [Likely] Is the presentation expected to cover my own work, a given topic, or a vision for UiPath, and how is it weighted in the debrief?
93. [Likely] Who will be on the panel, and is Q&A held to the end or are interruptions expected?
94. [Likely] Which product area's problems would the person in this role own first?
95. [Likely] How does UiPath move applied science results into products today, and where does that stall?
96. [Likely] What does success look like for this role after one year?

References

No source page could be retrieved in either run (see "Research limits" above), so no question cites a source. The official pages below are the right places to check the format notes and product context. They were not read in either run; the review run's fetch of uipath.com failed with a DNS error, so the links are given as unverified.

1. UiPath Careers, uipath.com (official careers page; unverified, fetch failed). https://www.uipath.com/company/careers
2. UiPath Blog, uipath.com (official product and AI announcements; unverified, fetch failed). https://www.uipath.com/blog

03. ML/LLM Science Depth for System Design: Agents, Document Understanding, Screen Understanding, NLP and Evaluation

How to read this chapter. You asked for ML system design, so this chapter covers the science-depth questions that sit underneath design decisions. At principal level, the depth round and the design round overlap: interviewers test whether you can justify each box on the diagram with modelling evidence. Each sub-topic opens with a plain-text reference diagram of the system the questions probe. The diagrams give you a shared picture to reason about. They are not model answers, and they render the same in Markdown and in a PDF export.

Tagging system (same as Chapter 01).

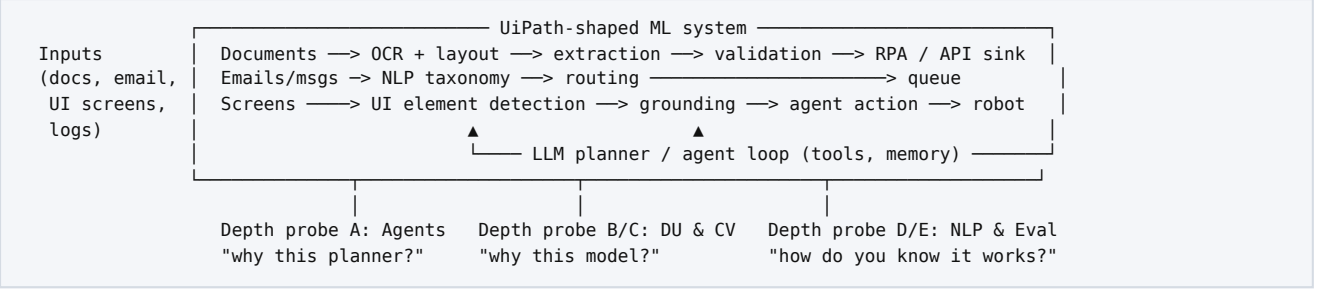
- **[Reported] [n]**: a candidate report, question bank or official page links this question, or this round type, to UiPath. The number points to the References list.
- **[Likely]**: an inference from the role, the level and UiPath's AI products (agentic automation and Agent Builder, Document Understanding, Communications Mining, Autopilot, the AI Trust Layer, and computer vision for UI automation). No candidate has reported it.
- Indented sub-bullets are follow-up probes an interviewer may use to push a strong first answer toward the principal bar.

Research limits (please read). Two runs have now tried to research this chapter, and neither could reach a source. In the first run, the shared web-search budget (200 searches) was used up and fetches of glassdoor.com, uipath.com and arxiv.org failed DNS lookup. In the review run (2026-10-14), the search budget was still used up, and the network proxy rejected direct fetches of glassdoor.com, uipath.com, docs.uipath.com, arxiv.org and reddit.com. So **no question below is tagged [Reported]**. None could be checked against a candidate write-up, and this chapter does not invent reports. Every question is **[Likely]**. The review run removed two duplicates and added 27 new principal-level [Likely] questions on gaps in the first draft: agent protocols and process conformance, credit assignment, line items and field normalisation, VLM image-token budgets, screen tokenisation and timing, taxonomy discovery, simulated-user evaluation, label noise and privacy-preserving training. The format notes are inferred from how principal-scientist loops usually run at enterprise AI companies; they are not a confirmed UiPath process. A later run with network access can upgrade tags to [Reported] where sources support them.

Format notes (inferred, not confirmed)

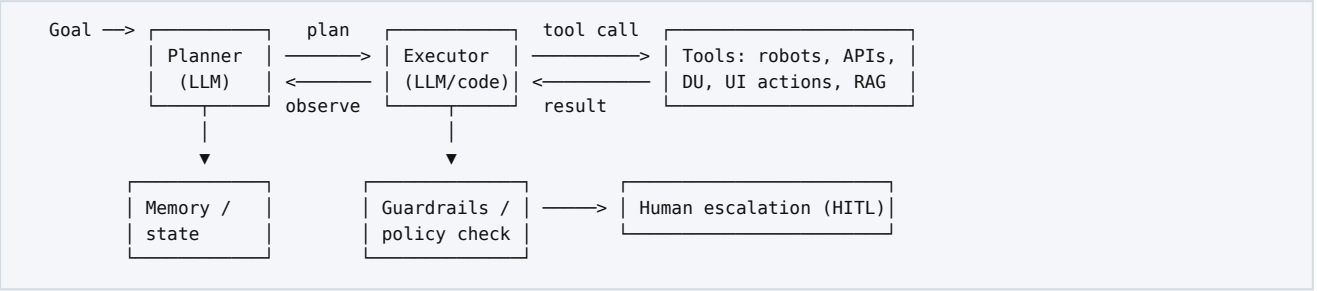
Item	Likely shape	Status
Duration	45-60 min, one or two rounds (ML depth; sometimes merged into research-presentation Q&A)	Likely
Interviewer	Staff or principal applied scientist from the agents, Document Understanding, Communications Mining or CV/UI team	Likely
Style	Starts from your past work or a UiPath-shaped problem, then drills down 3-5 levels until you hit the edge of your knowledge	Likely
Principal bar	(1) You choose a method from first principles and evidence, not fashion. (2) You design evaluation that predicts customer outcomes. (3) You see failure modes before they happen. (4) You know when not to use an LLM. (5) You set direction for a research area across teams.	Likely

How depth questions attach to a design diagram



Questions

A. Agents and LLM reasoning



1. [Likely] Walk me through how you would decide between a single ReAct-style agent, a plan-then-execute agent, and a fixed workflow with LLM steps for an enterprise process such as invoice dispute handling.
    - What evidence would make you move from the agent design back to a deterministic workflow?
    - How does your answer change when each tool call triggers a real side effect in an ERP system?
2. [Likely] Why do LLM agents degrade on long-horizon tasks, and which of those causes can be fixed by better modelling rather than better orchestration?

    - How would you measure error compounding across steps?
    - What does a per-step success rate of 95% imply for a 20-step task, and is that model of failure realistic?

3. [Likely] How would you design the tool interface (schemas, descriptions, granularity) so an LLM selects the correct tool among 200 enterprise tools?

    - When would you use retrieval over tool descriptions instead of putting all tools in context?
    - How would you evaluate tool-selection accuracy separately from argument-filling accuracy?

4. [Likely] Compare chain-of-thought prompting, self-consistency, tree search and trained reasoning models for agent planning. Which would you ship under a fixed cost per run?

    - How would you show that extra test-time compute actually improves task success in your domain?

5. [Likely] How do you make an agent recover from a failed tool call without looping or hallucinating success?

    - How would you detect "fake completion", where the agent says it finished but the system state disagrees?

6. [Likely] What memory architecture would you use for an agent that runs the same business process thousands of times a day?

    - What should be stored as episodic memory versus distilled into prompts, rules or fine-tuning data?
    - How do you stop memory from one tenant leaking into another?

7. [Likely] When would you fine-tune a smaller model for agent actions instead of prompting a frontier model? Describe the data you would need.

    - How would you build training trajectories from existing RPA logs and human corrections?
    - What failure modes do you expect from imitation learning on successful trajectories only?

8. [Likely] How would you apply reinforcement learning (for example, RL from task-level rewards) to improve an enterprise agent, given that real environments are expensive and have side effects?

    - How would you build a safe simulated environment, and how would you check the sim-to-real gap?
    - How would you prevent reward hacking when the reward is "process completed"?

9. [Likely] Describe a multi-agent design you would build for a cross-system process, and argue whether multiple agents are actually better than one.

    - How do you assign accountability for a failure in a multi-agent run?

10. [Likely] How would you defend an agent against prompt injection delivered through documents, emails or web pages it reads?

    - What can be done at the model level, and what must be done at the system and permission level?
    - How would you red-team this before release?

11. [Likely] How would you decide which agent actions need human approval, and how would you set that threshold from data?

    - How would you calibrate the agent's confidence so that the approval policy is trustworthy?

12. [Likely] How do structured outputs, constrained decoding and function calling change error rates in agents? Where do they fail?

13. [Likely] How would you blend deterministic RPA robots with LLM agents so that the robot does the reliable parts and the agent handles exceptions?

    - How would the agent hand control back to a robot mid-process?

14. [Likely] What are the main open research problems in enterprise agents today? If you joined UiPath and could fund one science bet across agents, document understanding and screen understanding, which would it be?

    - How would you staff it, and what would you stop doing to pay for it?
    - How would you know after six months whether that bet was paying off?

15. [Likely] UiPath exposes robots and agents to other vendors' agents through open protocols such as the Model Context Protocol. How would you design and evaluate tool contracts so that an external agent cannot misuse a UiPath tool?

    - How would you version a tool schema without silently breaking agents that were tuned on the old one?

16. [Likely] How would you combine a BPMN-style process model with LLM agents so that each agent run can be checked for conformance against the model?

    - What do you do when the agent finds a better path than the modelled one: flag it, block it or learn from it?

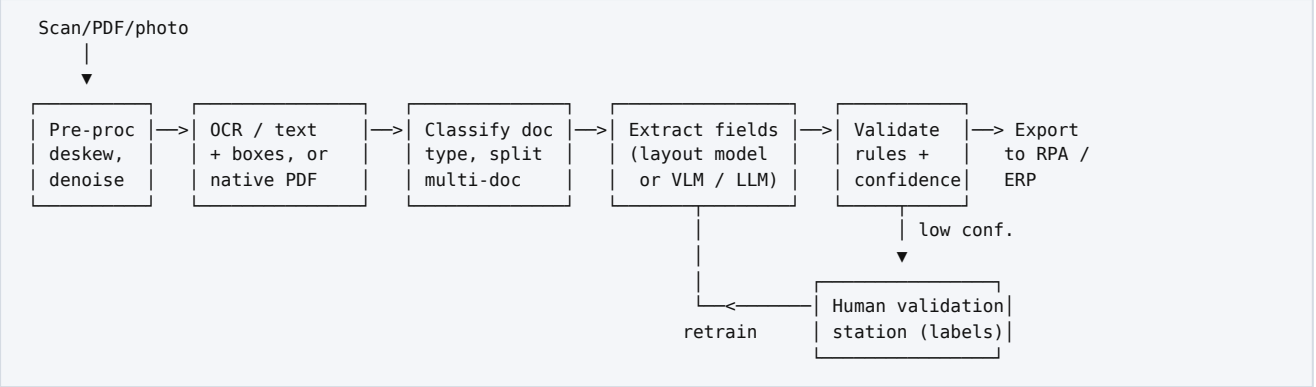
17. [Likely] Given process-mining event logs from a customer, how would you estimate which steps an agent should take over, and the expected value, before anything is built?

    - How would you validate that estimate after deployment?

18. [Likely] How would you estimate uncertainty for an LLM agent's next action (for example, sampling-based agreement, semantic entropy or a trained verifier), and which would you trust for an approval gate?

19. [Likely] How would you assign credit to individual steps when training an agent from sparse, trajectory-level success signals?
    - Compare outcome reward models with process reward models for this setting.
20. [Likely] Agent latency is dominated by sequential LLM calls. Which modelling changes, not only infrastructure changes, would you make to cut end-to-end latency by half?

B. Document understanding: OCR, layout models and extraction



21. [Likely] Compare an OCR-plus-layout-transformer pipeline (LayoutLM-style) with an OCR-free vision-language model (Donut or a general VLM) for invoice extraction. Which would you ship, and for which customers?
    - How does each behave on low-quality scans, handwriting and 30-page documents?
    - What is the cost per page of each at a million pages a day?
22. [Likely] How do layout-aware models encode 2D position, and why does that help over plain text sequences?
    - What breaks when the OCR reading order is wrong?
23. [Likely] How would you handle tables that span pages, have merged cells or have no ruling lines?
    - How would you evaluate table extraction: cell-level, row-level or by downstream task success?
24. [Likely] How would you produce well-calibrated per-field confidence scores from an extraction model, including an LLM that does not expose useful probabilities?
    - How would you set thresholds that trade straight-through processing rate against error rate per customer?
25. [Likely] A customer brings a new document type with 20 labelled examples. How would you get to production accuracy fastest?
    - Compare few-shot LLM prompting, fine-tuning a specialist model and active learning.

    - How would you decide when the specialist model should replace the LLM?

26. [Likely] How would you use human validation corrections as training data without amplifying annotator errors or bias?
    - How would you detect that validators are rubber-stamping low-confidence predictions?
27. [Likely] How do generative extractors hallucinate values that are not in the document, and how would you detect and prevent this?
    - How would you ground every extracted value back to a region on the page?
28. [Likely] How would you design a multilingual and multi-script document model, including right-to-left and CJK scripts?
    - Where would you expect the biggest accuracy drops, and how would you find them before customers do?
29. [Likely] How would you classify and split a 200-page PDF that contains many concatenated documents?
30. [Likely] How would you train a general-purpose extraction foundation model for business documents? Describe data, objectives and evaluation.
    - How would you get pretraining data while respecting customer data rights?
    - How would you show it beats a frontier VLM on your benchmark and not only on public ones such as DocVQA or FUNSD?
31. [Likely] What is the role of synthetic documents in training, and how do you check that synthetic gains transfer to real documents?
32. [Likely] How would you handle handwriting, checkboxes, signatures and stamps in forms?
33. [Likely] How would you detect drift when a vendor changes its invoice template, and what should happen automatically?
34. [Likely] How would you distil a large VLM extractor into a model that can run on-premises on CPU?
    - Which accuracy loss would you accept, and how would you explain it to a customer?
35. [Likely] How would you extract line items and other nested or repeated structures, and link each value to the right key and row?
    - How would you score a line-item prediction where one row is split into two?
36. [Likely] How do you define 'correct' for an extracted field when dates, amounts, names and addresses can be written many ways? Design the normalisation and the metric.

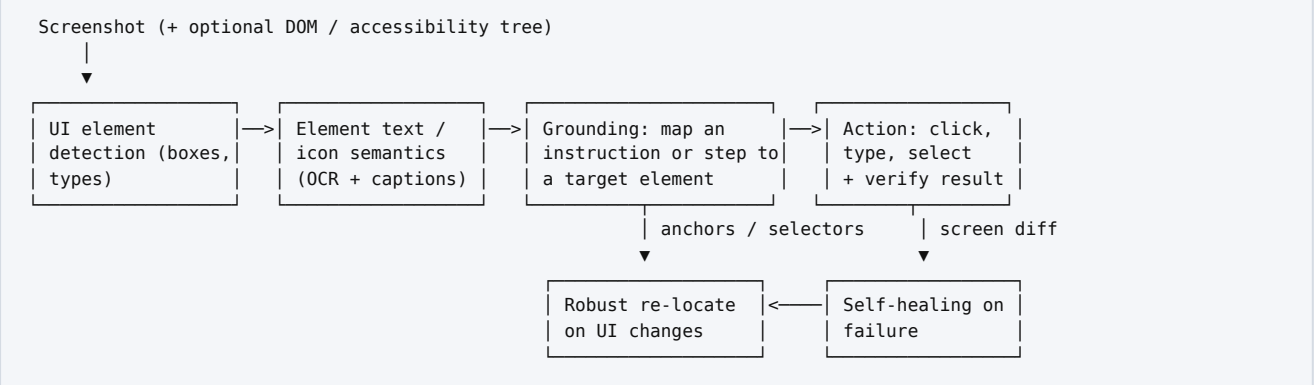
37. [Likely] How would you measure how much OCR error propagates into extraction error, and decide whether to spend the next quarter on OCR or on the extractor?

38. [Likely] A customer uploads documents with no schema. How would you propose a field schema automatically, and how would you evaluate the proposal?

39. [Likely] A VLM must read a 40-page document at enough resolution to see small print. How would you manage the image-token budget (page selection, tiling, cropping, multi-pass reading)?

    - How would you check that the model did not skip the page that held the answer?

C. Computer vision for UI and screen understanding



40. [Likely] How would you build a model that finds UI elements on screens of applications with no accessible DOM, such as Citrix or remote desktop sessions?

    - Detector plus OCR, or an end-to-end VLM? Argue both sides.
    - How do you handle tiny icons, dense tables and high-DPI screens?

41. [Likely] How would you ground a natural-language step ("click the Submit button in the payment dialog") to the correct element when there are several similar candidates?

    - How do you use spatial context, text anchors and visual similarity together?

42. [Likely] Selectors break when an application UI is updated. How would you design a self-healing targeting model, and how would you measure that it heals correctly rather than clicking the wrong element?

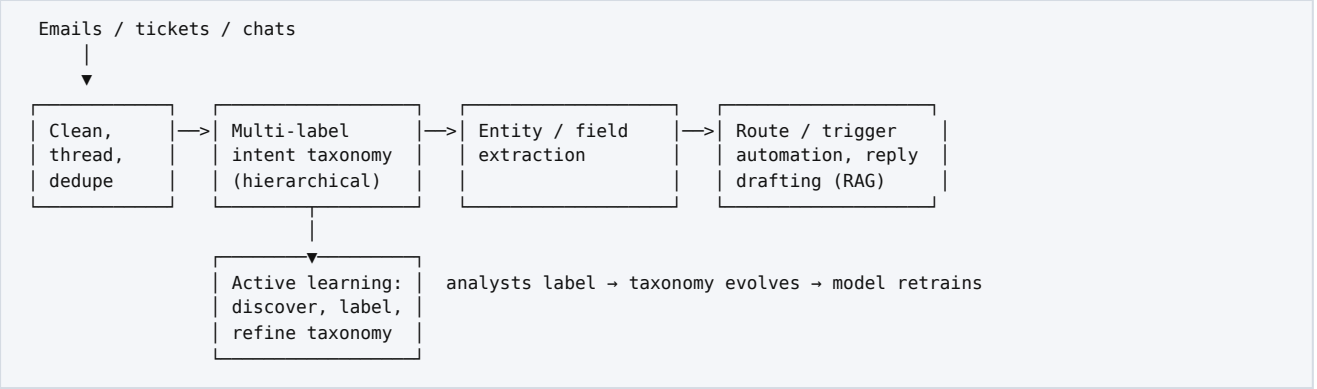
    - What is the cost of a false heal compared with a failed run, and how does that set your threshold?

43. [Likely] How would you build a training dataset for screen understanding across thousands of enterprise applications?

    - How would you use synthetic UI rendering, recorded robot runs and weak labels from accessibility trees?
    - How would you remove sensitive data from screenshots used for training?

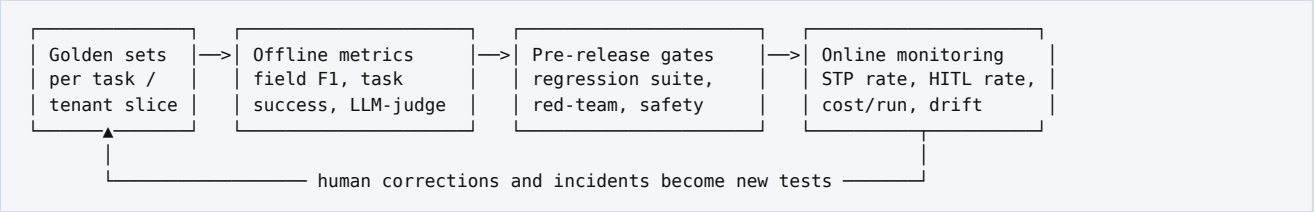
44. [Likely] Compare pixel-only computer-use agents with agents that use the DOM or accessibility tree. When does each win?
    - How would you fuse both signals when they disagree?
45. [Likely] How would you evaluate a computer-use agent? Discuss benchmarks such as OSWorld, WebArena or ScreenSpot and their limits for enterprise apps.
    - What would an internal benchmark look like, and how would you keep it from leaking into training data?
46. [Likely] How would you verify that a UI action had the intended effect, using only the next screenshot?
47. [Likely] How would you make screen-understanding inference fast enough for interactive automation on a user's laptop?
    - Which parts would you cache across frames?
48. [Likely] How would you handle the long tail: custom controls, canvas-rendered apps, legacy terminals and mainframe green screens?
49. [Likely] How would you turn a recorded user session (task mining) into a reusable automation or agent skill?
    - How do you segment a long recording into steps, and how do you evaluate the segmentation?
50. [Likely] Screenshots are high-resolution and UI targets are tiny. How would you tokenise or crop screens for a VLM without losing small elements, and how would you test that choice?
51. [Likely] How would you get a screen model trained mostly on web and Windows apps to generalise to SAP GUI, Java Swing, mobile and terminal UIs?
    - How would you measure the domain gap before collecting new data?
52. [Likely] How would a UI agent know when the screen is ready to act on (loading spinners, delayed pop-ups, animations), and how would you evaluate timing errors separately from targeting errors?
53. [Likely] From a single human demonstration, how would you build an automation that generalises to new inputs and slightly different screens?
    - What should the system ask the user to clarify, and when?

D. NLP: communications mining, classification and retrieval



54. [Likely] How would you build a multi-label, hierarchical intent classifier for customer emails where the taxonomy is defined by business users and changes every month?
    - How do you handle labels being split, merged or renamed without relabelling everything?
55. [Likely] Compare a fine-tuned encoder classifier with zero-shot or few-shot LLM classification on cost, latency, calibration and accuracy at 10 million messages a month.
56. [Likely] How would you design the active-learning loop so business analysts can build a taxonomy from unlabelled data in a few hours?
    - Which sampling strategy would you use for discovery versus refinement?
    - How do you avoid sampling bias in the resulting evaluation set?
57. [Likely] How would you handle email threads, quoted replies, signatures and attachments when modelling intent?
58. [Likely] How would you measure and improve calibration for multi-label predictions so thresholds work per label?
59. [Likely] How would you design retrieval-augmented generation over enterprise knowledge (process documents, SOPs, policies) for an agent?
    - Which chunking strategy suits SOPs with tables and numbered steps?
    - When do hybrid search, reranking or query rewriting pay for themselves?
    - How do you enforce document-level permissions in retrieval?
60. [Likely] How would you detect sentiment, urgency and churn risk in operational messages, and how would you prove the signal is useful to the business?
61. [Likely] How would you support many languages for communications mining with labels mostly in English?
62. [Likely] How would you mask PII in text before it reaches a third-party LLM while keeping enough context for the task?
    - How do you measure the accuracy loss caused by masking?
63. [Likely] How would you use LLMs to discover a taxonomy from unlabelled messages (clustering plus LLM-written label names), and how would you evaluate a taxonomy when there is no ground truth?
64. [Likely] How would you evaluate drafted email replies for factual grounding in the retrieved policy, tone and resolution rate, and which of these can be automated?

E. Evaluation and measurement science



65. [Likely] Design an evaluation framework for enterprise agents that a dozen product teams can share.
    - What is the unit of evaluation: step, trajectory or business outcome?
    - How do you score partial success?
66. [Likely] How would you validate an LLM-as-judge before relying on it for release decisions?
    - How do you measure and correct position bias, verbosity bias and self-preference?
    - How much human agreement data is enough?
67. [Likely] Offline field-level F1 went up, but customers report more errors. List possible causes and how you would check each.
68. [Likely] How would you build test sets that represent customers you cannot see, because their data stays in their tenant?
    - What can privacy-preserving or federated evaluation offer here, and what does it cost?
69. [Likely] How would you compute confidence intervals and test significance for agent task success when runs are non-deterministic?
    - How many runs per task would you need to detect a 2-point improvement?
70. [Likely] How would you tie model metrics to business metrics such as straight-through processing rate, handling time saved and cost per transaction?
71. [Likely] How would you catch benchmark contamination when you use third-party foundation models?
72. [Likely] How would you design an A/B test or staged rollout for a model change in an automation product, where errors cause downstream financial actions?
    - When would you use shadow mode instead of an A/B test?
73. [Likely] How would you evaluate safety and policy compliance (for example, the agent must never approve a payment above a limit) in a way that gives strong guarantees, not only averages?
74. [Likely] How would you evaluate hallucination in extraction and in generated replies, and what metric would you report to a customer?

75. [Likely] How would you decide that a model is good enough to switch a customer from human validation of every document to sampled validation?
76. [Likely] How would you compare foundation-model providers for UiPath products when the AI Trust Layer lets customers choose their own model?
    - How do you keep evaluations stable when providers update models silently?
77. [Likely] How would you evaluate a conversational agent that needs several turns with a user, for example with simulated users as in tau-bench? What makes a simulated user trustworthy?
78. [Likely] Your golden set has label noise and annotators disagree. How would you measure inter-annotator agreement, fix the set and report metrics that account for the noise?
79. [Likely] How would you design an internal benchmark that stays useful as models improve, rather than saturating within a year?
80. [Likely] Which metric would you use to measure human effort saved (for example, time to validate, corrections per document) rather than model accuracy, and how would you collect it without slowing users down?
81. [Likely] How would you compare verbalised confidence, token log-probabilities and a separate verifier as confidence signals for LLM outputs? Which would you ship?

F. Model strategy, efficiency and deployment science

82. [Likely] How would you decide between building UiPath-specific models (for documents, communications or UI) and relying on third-party frontier models?
    - What would change your decision in 12 months?
83. [Likely] How would you route requests between small and large models to minimise cost at fixed quality?
    - How would you train and evaluate the router?
84. [Likely] Which compression methods (quantisation, distillation, pruning, speculative decoding) would you apply for on-premises deployments, and how do you measure quality loss on the long tail?
85. [Likely] How would you fine-tune a model per customer (for example, with LoRA adapters) and serve thousands of adapters efficiently?
    - How do you stop one customer's adapter from learning another customer's data?
86. [Likely] How would you handle continual learning from customer feedback without catastrophic forgetting or silent regressions?
87. [Likely] How do context length and long-context models change document and agent design, and where do they still fail (for example, lost-in-the-middle)?

88. [Likely] How would you design a model that is robust to adversarial or malformed documents and screenshots?

89. [Likely] What is your approach to data governance for training: consent, retention, opt-out and the right to be forgotten in trained models?

90. [Likely] When would you train one generalist model across documents, communications and UI, and when separate specialists? How would you detect negative transfer between tasks?

91. [Likely] How would you use LLM-generated synthetic data to train smaller models without collapsing diversity or copying the teacher's errors?

92. [Likely] When would you apply differential privacy or other formal privacy methods when training on customer documents, and how would you explain the accuracy cost?

G. Principal-level depth on your own work

93. [Likely] Pick the most complex model you shipped. Draw the system, and tell me which design decision you would reverse today and why.
    - What experiment told you it was the wrong decision?

94. [Likely] Tell me about a time an offline evaluation misled you. What did you change in your evaluation practice afterwards?

95. [Likely] Describe a research result you chose not to ship. How did you make that call?

96. [Likely] Which paper from the last two years most changed how you would build agents or document models, and what would you build differently because of it?
    - What is the weakest part of that paper?

97. [Likely] How have you set the technical direction for a group of scientists working across several products?
    - How did you settle a disagreement between research ambitions and product deadlines?

98. [Likely] How do you decide whether a problem needs new research or a well-executed application of known methods?

99. [Likely] I will show you the abstract and main table of a recent agent or document-AI paper. Critique it as if you were deciding whether your team should build on it.
    - Which missing experiment would change your decision?

100. [Likely] What standards for reproducibility, baselines and ablations would you require before a result from your team reaches a product roadmap?

References

No source page could be retrieved in either run (see "Research limits" above), so no question cites a source and every question is tagged [Likely]. The official pages below are the right places to check product context (Agents, Maestro, Document Understanding, Communications Mining and the AI Trust Layer). On 2026-10-14 the network proxy blocked every fetch to them, so they are listed as unverified starting points. No question relies on them.

1. UiPath Careers, uipath.com (official careers page; unverified, fetch blocked). https://www.uipath.com/company/careers
2. UiPath Blog, uipath.com (official product and AI announcements; unverified, fetch blocked). https://www.uipath.com/blog
3. UiPath Documentation portal, docs.uipath.com (product documentation; unverified, fetch blocked). https://docs.uipath.com

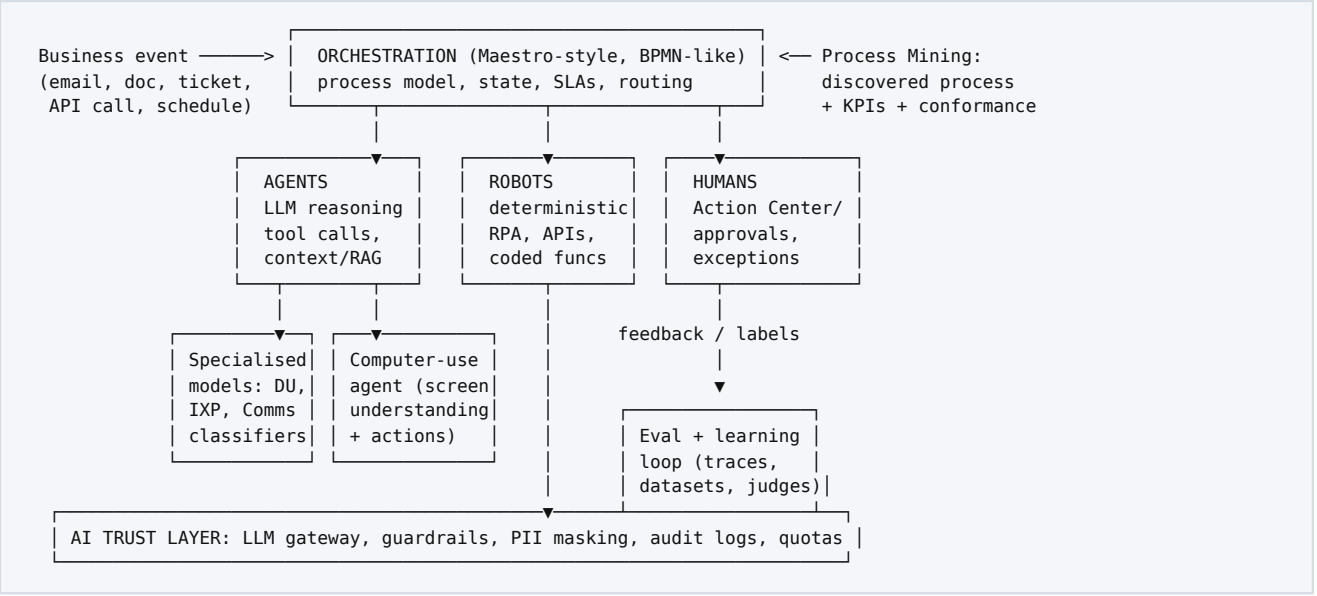
04. Applied Science System Design: Agentic Automation, Computer-Use Agents, Document Understanding, Communications Mining, Process Mining and Orchestration

> **How to read this chapter.** This is the ML system design chapter for UiPath's own product surface. Each section opens with a plain-text block diagram of the system you are likely to be asked to design. The numbered questions follow, with follow-up probes as indented sub-bullets. The chapter has questions only, with no answers. The diagrams are plain text so they render the same in Markdown and in a PDF export.
>
> **Tagging system.** Every question carries one of two tags:
>
> - **[Reported] [n]**: a candidate report or interview question bank links this question to UiPath. The number points to the References list.
> - **[Likely]**: an inference from the role, the level and UiPath's products: the agentic automation platform (agents, robots, humans), computer-use agents, Document Understanding and IXP, Communications Mining, Process Mining, Autopilot, Agent Builder and coded agents, Maestro orchestration and the AI Trust Layer. No candidate has reported it. A **[Likely] [n]** tag means the *premise* of the question (a product feature, an SDK behaviour, a benchmark rule) is confirmed by source [n]. It does not mean a candidate was asked it.
> - Indented sub-bullets are follow-up probes an interviewer may use to push a strong first answer toward the principal bar.
>
> **Research limits (please read).** This review pass (October 2026) could not reach any candidate-report site. The shared web-search budget for the workflow was used up. The egress proxy refused connections to glassdoor.com, uipath.com, docs.uipath.com and arxiv.org (HTTP 403 by policy). GitHub raw content was reachable, so the review read UiPath's own public SDK repositories (documentation, samples and source) and the OSWorld repository. Those sources confirm product behaviour, not interview content. So **no question below is tagged [Reported]**, and this chapter does not invent reports. Product names such as Maestro, Agent Builder, Autopilot and IXP should still be checked against current UiPath documentation before the interview.

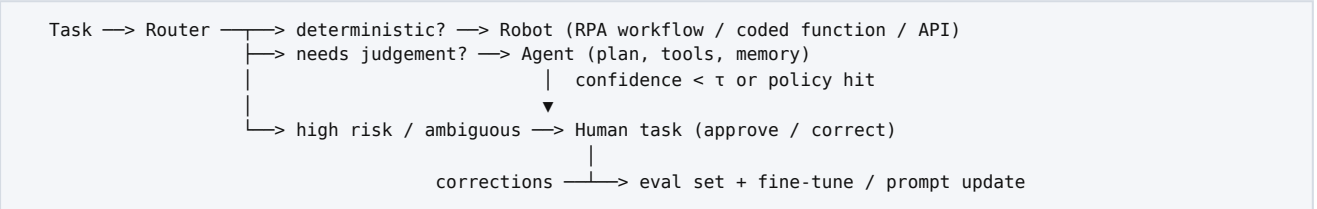
Format notes for this round

Item	Expected shape (inferred, not confirmed)
Round name	ML / applied science system design, sometimes merged with a product or "architecture" round
Duration	60 min: about 5 min framing, 35-40 min design, 10-15 min deep-dive probes, 5 min your questions
Interviewer	Principal or distinguished scientist, or an AI platform architect; sometimes paired with a product lead
Prompt style	Open-ended and UiPath-shaped: "design an agent that processes invoices end to end", "design a computer-use agent for legacy desktop apps", "design the eval system for agents"
Principal-level bar	You set the problem framing and success metrics yourself. You separate deterministic (robot / coded function) work from probabilistic (agent / LLM) work. You design evaluation and the human-in-the-loop path as first-class parts. You reason about cost, latency, on-prem and regulated deployment, and governance. You define a multi-quarter roadmap and say what research is still open.
Common failure modes	Jumping to "use GPT plus RAG" without workload analysis; no offline/online eval plan; ignoring exception handling, retries and human escalation; ignoring the existing RPA asset base (selectors, workflows, queues)

Reference diagram: the agentic automation stack an interviewer has in mind



A. Agentic automation: agents + robots + humans



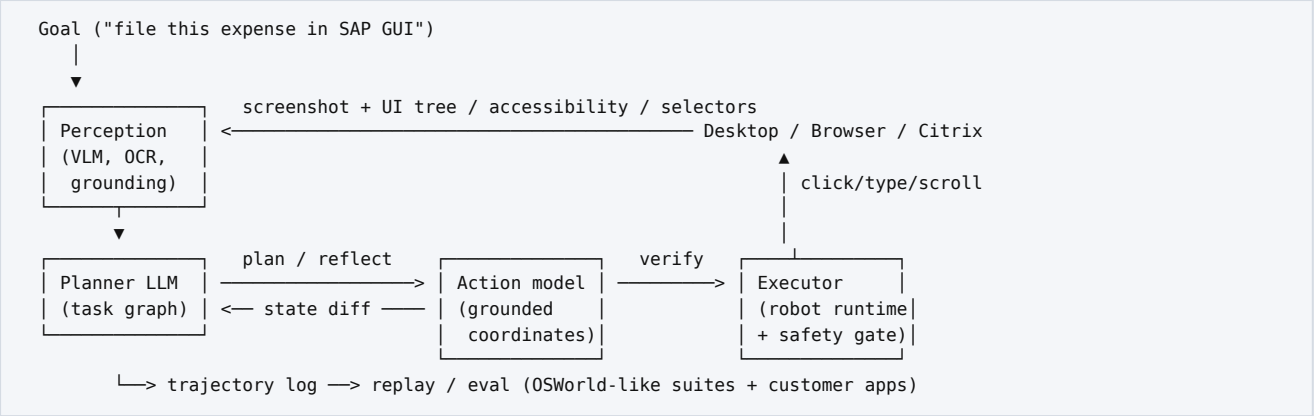
1. [Likely] Design an end-to-end agentic system that resolves inbound customer-service emails for an insurer, using agents, existing RPA robots and human reviewers.
    - How do you decide which steps stay deterministic robots and which become agent steps?

    - What is the unit of work, and how do you define "resolved" as a metric?
    - Where exactly does a human get pulled in, and what context do they see?
2. [Likely] [5] UiPath's SDK separates "coded functions" (deterministic, typed input/output, no LLM) from "coded agents" (an LLM reasoning loop that chooses tools). Give a decision procedure for which one a new automation should be, and design the path for promoting a function to an agent, or demoting an agent back to a function, once you have production traces.
    - What evidence from traces would justify the demotion?
    - How do you keep the typed input/output contract stable for the callers when the implementation changes?
3. [Likely] Given a company with 500 existing RPA workflows, how would you introduce agents without breaking what works?
    - Would agents call robots as tools, or robots call agents? What changes in failure handling?
    - How do you prioritise which workflows to "agentify" first?
4. [Likely] [10] [11] Design the confidence and escalation policy for an agent that takes financial actions (refunds, payments) inside an enterprise process.
    - How do you calibrate an LLM agent's confidence when there are no token-level probabilities you can trust?
    - How do you set thresholds per action type and per customer risk appetite?
    - How do you stop humans from rubber-stamping escalations?
5. [Likely] [11] UiPath publishes an anti-money-laundering triage sample in which the LLM only extracts facts and writes the rationale, while a separate typed classifier makes every decision (red flags, risk level, disposition). When is this "LLM reads and writes, a calibrated model decides" split better than letting the agent decide?
    - How do you calibrate and monitor the decision model, and what happens when the LLM's extraction is wrong upstream?
    - How would you explain one disposition to a regulator?
6. [Likely] [3] [4] How would you design the tool interface layer so that an agent can safely invoke UiPath robots, APIs, Integration Service connectors and MCP servers?
    - How do you describe tools so the model picks the right one among hundreds?
    - How do you handle tool schemas that change under the agent?
    - How do you enforce least privilege per agent and per run?
7. [Likely] [4] UiPath lets customers host their own MCP servers on the platform and expose them to agents. Design the governance for third-party MCP tools: registration, trust review, versioning, sandboxing and per-tenant allow lists.
    - How do you detect that an MCP tool's behaviour changed even though its schema did not?
    - What do you do when a tool's description itself contains an injected instruction?
8. [Likely] Design memory for a long-running enterprise agent that handles a case over days or weeks.
    - What goes into short-term context, case state and long-term memory?
    - How do you prevent stale or poisoned memory from driving actions?
9. [Likely] [8] [2] Design durable suspend and resume for agents that wait days for a human approval, a robot job or an external callback. UiPath's runtime checkpoints agent state at an `interrupt()` and resumes it later.
    - What exactly do you checkpoint, and where, so a resume survives a container restart?

    - The agent's code is redeployed while 3,000 runs are suspended. How do you migrate or pin their checkpoints?
    - How do you guarantee a side effect performed just before the suspend is not repeated on resume?
10. [Likely] [9] Design an event-triggered, long-running agent that sleeps on a connector event (for example, a new email in a watched mailbox), wakes to process it, then suspends again.
    - How do you handle duplicate events, out-of-order events and a burst of 10,000 events after an outage?
    - How do you stop one poison message from blocking the agent forever?
11. [Likely] [13] How would you design a multi-agent system for procure-to-pay, and when is multi-agent (for example, a supervisor routing to specialist agents) worse than a single agent?
    - How do agents hand off context, and who owns the global state?
    - How do you debug a failure that spans three agents and two robots?
12. [Likely] What is your framework for deciding the autonomy level of an agent in a given process?
    - What evidence moves a process from "human approves every action" to "human reviews a sample"?
13. [Likely] [6] How would you make an agentic workflow idempotent and recoverable when a step fails halfway, for example after a payment API call times out?
    - How do you design compensation or rollback for actions an agent already took?
14. [Likely] [7] [12] Design the human-in-the-loop task experience so that corrections become training or evaluation data.
    - What do you log from a human correction, and how do you de-bias it?
    - How do you use corrections without overfitting to one reviewer's preferences?
15. [Likely] [12] Design the human task inbox at scale: 50,000 open approval tasks a day from agents across many processes, with different SLAs and reviewer skills.
    - How do you prioritise and assign tasks, and when do you batch similar approvals?
    - How do you measure reviewer fatigue and the quality of approvals, not just throughput?
16. [Likely] How would you measure ROI for an agentic automation, compared with the RPA baseline and the fully manual baseline?
    - Which metrics would you show a CFO, and which would you show the science team?
17. [Likely] [15] How would you defend an agent against prompt injection that arrives through emails, documents or web pages it processes?
    - How do you separate instructions from data in the agent's context?
    - What runtime guardrails would you add beyond the model itself?
18. [Likely] [3] Design the system that lets an enterprise run the same agent across multiple LLM providers, or on-prem models, with consistent behaviour.
    - How do you detect and manage regression when a provider updates a model?
19. [Likely] [2] [3] [5] UiPath's runtime hosts agents written in LangGraph, LlamaIndex, the OpenAI Agents SDK, Google ADK, PydanticAI, the Claude Agent SDK and its own framework behind one runtime protocol. What must that protocol standardise (inputs/outputs, tracing, errors, human-in-the-loop, resume) and what should it deliberately leave to the framework?

- How do you evaluate and compare two agents built on different frameworks fairly?
- What breaks first when a framework releases a breaking change?

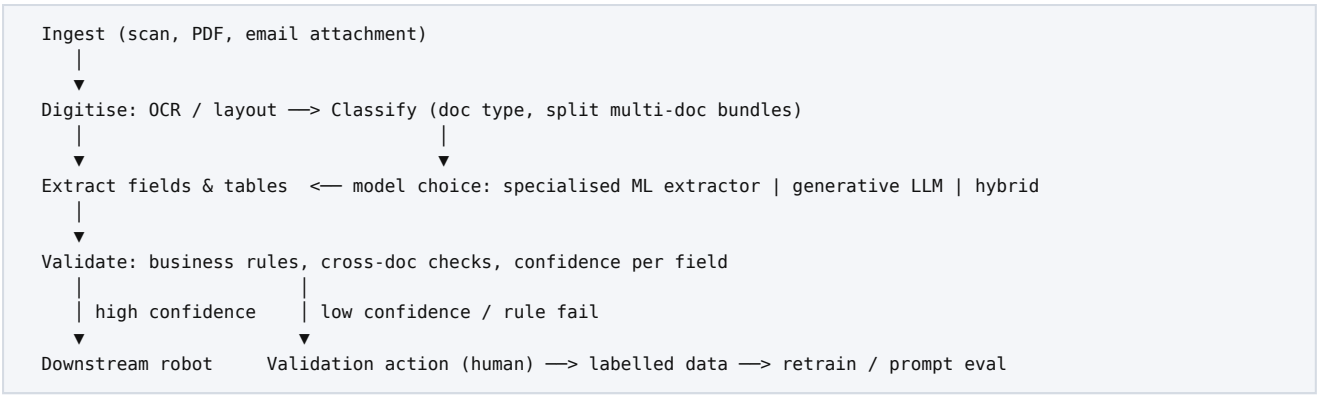
B. Computer-use agents (ScreenAgent-style UI automation)



20. [Likely] Design a computer-use agent that can operate legacy desktop applications (for example SAP GUI or a Citrix-virtualised app) where no API or DOM is available.
    - What inputs do you use: pixels only, accessibility tree, UiPath selectors, OCR? How do you fuse them?
    - How do you ground a natural-language target ("the Submit button") to coordinates reliably?
21. [Likely] How would you combine a generalist computer-use model with UiPath's existing deterministic selectors and recorded workflows?
    - When should the agent "compile" a successful trajectory into a deterministic robot workflow?
    - How would you detect that a compiled workflow has drifted and needs the agent again?
    - If an agent proposes a repaired selector after a UI change, how do you verify the repair before it is committed, so the robot does not silently click the wrong element?
22. [Likely] [21] [22] How would you evaluate a computer-use agent, and how far would you trust public benchmarks such as OSWorld for enterprise use?
    - How do you build an internal benchmark of enterprise apps with verifiable end states?
    - How do you score partial progress versus binary task success?
    - How do you prevent benchmark contamination and overfitting to the leaderboard?
23. [Likely] [21] OSWorld had to be re-released as OSWorld-Verified to fix community-reported task errors, and its maintainers now run verified submissions themselves. Design the governance for an internal computer-use benchmark so you do not repeat that history.
    - How do you detect flaky tasks and environment noise, and separate them from real capability changes?

    - How many reruns per task do you need before you report a score difference, and how do you report variance?
24. [Likely] [22] OSWorld grades each task with an execution-based script that inspects the final machine state. Design state verifiers for enterprise tasks where the true end state lives in a backend you cannot read directly (SAP posting, Salesforce record, mainframe screen).
    - When is a screen-based check acceptable, and when must you query the system of record?
    - How do you keep verifiers correct as the application is upgraded?
25. [Likely] Design the safety layer for a computer-use agent that has a real user's desktop session.
    - Which actions require confirmation, and how does the agent know an action is irreversible?
    - How do you handle unexpected dialogs, pop-ups and login prompts?
26. [Likely] Latency is 5-10 seconds per step and tasks take 40 steps. How do you make this usable and affordable?
    - Where would you use a smaller grounding model versus a frontier planner?
    - Which steps can be cached or replayed without calling a model?
27. [Likely] Build the break-even model: for a given process, at what volume, UI change rate and error cost is a computer-use agent cheaper than a maintained RPA robot, and when is it cheaper than a human?
    - Which inputs to this model would you measure first, and from which telemetry?
28. [Likely] How would you collect training data for UI grounding and action prediction at scale?
    - How would you use existing robot executions and selector data as weak supervision?
    - What are the privacy constraints of using customer screenshots, and how do you design around them?
29. [Likely] How does the agent recover when it gets lost (wrong screen, loop, state it did not expect)?
    - How do you detect loops and lack of progress automatically?
    - When should it give up and hand off to a human, and with what summary?
30. [Likely] How would you handle high-DPI, multi-monitor, localisation (non-English UIs) and theme variation in perception?
31. [Likely] Compare a pixel-only VLM agent with a structured-UI (accessibility tree / DOM) agent for enterprise automation. Where does each break?
32. [Likely] How would you fine-tune or distil a computer-use model for a single customer's application portfolio, and how do you keep that maintainable?

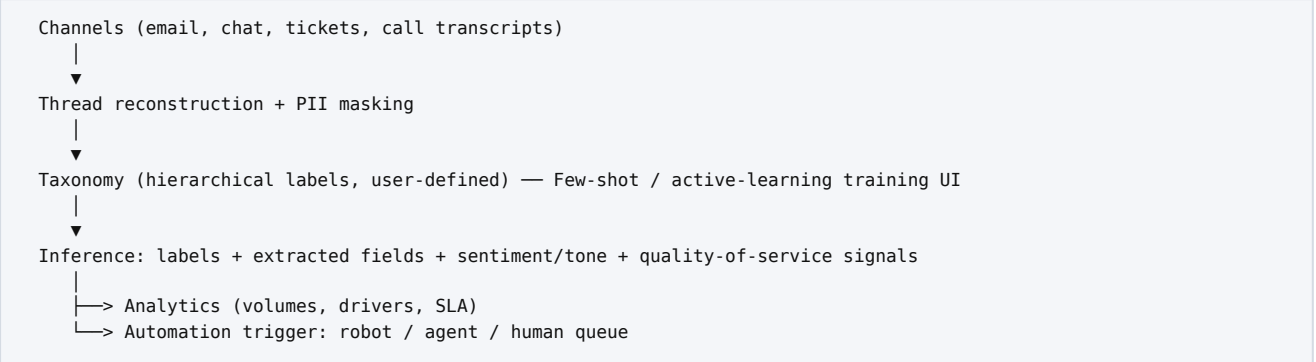
C. Document Understanding and IXP pipelines



33. [Likely] Design an invoice-processing pipeline that handles 1M documents a month across 40 languages and thousands of vendor layouts.
    - Where do you use a trained extractor and where a generative model?
    - How do you hit a straight-through-processing target while keeping field-level accuracy above a contract threshold?
34. [Likely] [20] UiPath's SDK exposes classification, extraction (DU Modern and IXP projects) and human validation actions as separate, non-blocking start/retrieve operations. Design the asynchronous pipeline around them for 1M documents a month.
    - How do you track a document across classify, extract, validate and downstream steps when any of them can fail or time out?
    - What decides that a document goes to a validation action, and at which granularity (document, page, field)?
35. [Likely] Suppose one extraction product must serve both structured documents and unstructured inputs such as emails and long contracts. What shared representation, schema language and confidence model would you design, and what gets worse when you merge the two workloads?
36. [Likely] How would you produce field-level confidence scores that are actually calibrated, for both classic extractors and LLM-based extraction?
    - How do you set per-field thresholds that trade human review cost against error cost?
    - How do you show calibration to a customer?
37. [Likely] Design document classification and splitting for a 300-page mortgage packet containing 25 document types.
    - How do you handle unseen document types at run time?
38. [Likely] Design generative extraction from 200-page commercial contracts where the answer to one field (for example, the termination notice period) depends on definitions, amendments and cross-references spread through the document.
    - How do you chunk, retrieve and resolve conflicting clauses?
    - How do you evaluate this when ground truth needs a lawyer's time?
39. [Likely] How would you extract complex tables (line items spanning pages, merged cells, nested headers) robustly?
    - How do you evaluate table extraction: cell-level, row-level, or downstream reconciliation?
40. [Likely] A customer has 50 labelled examples of a new document type. Design the fastest route to production quality.
    - When is few-shot prompting enough, and when do you fine-tune?

    - How do you use active learning to pick the next documents to label?
41. [Likely] [20] How would you design the human validation loop so that corrections improve the model without introducing label noise?
    - How do you detect inconsistent annotators?
42. [Likely] Handwriting, low-quality scans and stamps degrade OCR. Where in the pipeline do you invest, and how do you measure the gain end to end?
43. [Likely] How would you ground every extracted value back to a location in the document, and why does that matter for audit?
    - How do you stop a generative extractor from producing a plausible value that is not in the document?
44. [Likely] Design the evaluation harness for Document Understanding that runs on every model or prompt change.
    - What is in the golden set, how is it stratified, and how do you keep it from going stale?
45. [Likely] How would you run Document Understanding on-prem or in an air-gapped environment with limited GPU capacity?
    - What do you distil or quantise, and what accuracy loss is acceptable?
46. [Likely] Design cross-document reasoning: match an invoice to a purchase order and a goods receipt and explain mismatches.
47. [Likely] How would you detect drift in incoming document distributions (new vendors, new templates) and trigger action?

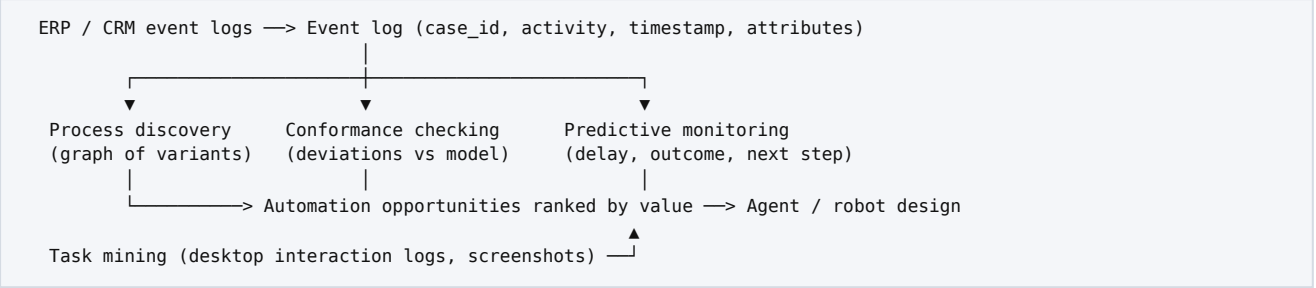
D. Communications Mining (email, chat and ticket understanding)



48. [Likely] Design a system that turns a bank's operations mailbox (200k emails per month) into structured requests that trigger automations.
    - How do you handle one email with several requests in it?
    - How do you handle threads where the intent changes mid-thread?
49. [Likely] Customers define their own label taxonomies with very few examples. How do you design the model and training loop?
    - How do you pick examples for the user to label (uncertainty, diversity, coverage)?
    - How do you tell the user when a label is "good enough" to automate on?
50. [Likely] [10] A UiPath ticket-classification sample has the LLM return a label with a self-reported confidence (for example 0.9) and sends it for human approval. Why is

verbalised LLM confidence a weak basis for automation thresholds, and what would you put in its place per customer taxonomy?
    - How do you measure calibration per label when some labels have ten examples?
51. [Likely] How would you decide between a fine-tuned encoder classifier, an embedding plus kNN approach, and an LLM prompt for intent classification at this scale?
    - What are the cost, latency and accuracy implications per million messages?
52. [Likely] Month-end volume is 8x the daily average and some message types have a 15-minute SLA. Design capacity, prioritisation and graceful degradation for the inference and automation-trigger path.
53. [Likely] Design field extraction from free-text messages (account numbers, amounts, dates) with validation against systems of record.
54. [Likely] How would you discover new, emerging topics in communications that nobody has labelled yet?
    - How do you present clusters so a business user can act on them?
55. [Likely] How do you evaluate a taxonomy-specific model when every customer's labels differ?
    - What platform-level metrics can you still track across customers?
56. [Likely] How would you measure and improve service quality signals (tone, urgency, chaser emails) in a way that operations teams trust?
57. [Likely] How would you handle multilingual mailboxes and code-switching?
58. [Likely] [9] How does Communications Mining feed an agent rather than a fixed robot, and what changes in the design?

E. Process mining and task mining



59. [Likely] Design a system that takes ERP event logs and recommends which process steps to automate with robots, agents or neither.
    - How do you estimate the value and feasibility of each opportunity?
    - How do you avoid recommending automation of a step that only looks slow because of data quality?
60. [Likely] How would you use LLMs to help build event logs from messy enterprise data (choosing case IDs, mapping tables to activities)?
    - How do you validate an LLM-proposed data model?
61. [Likely] Design a predictive process monitoring system that forecasts SLA breaches for in-flight cases and triggers interventions.
    - How do you handle censoring and changing process versions?
    - How do you show the intervention actually caused the improvement?
62. [Likely] How would you design task mining that infers repetitive desktop tasks from user interaction logs while respecting privacy?

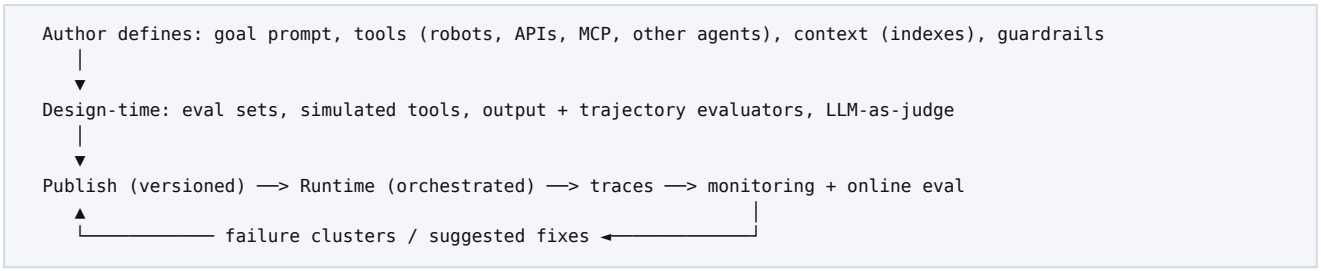
    - How do you segment continuous interaction streams into tasks?
    - What do you blur, drop or keep on-device?
63. [Likely] How would you close the loop so that process mining measures the impact of agents after deployment?
    - How do you represent agent steps (probabilistic, variable paths) in a process model?
    - How would you use conformance checking of agent trajectories against an approved process as an evaluation signal?
64. [Likely] Design root-cause analysis over process variants that a business analyst can trust.

F. Autopilot (natural-language assistance across the platform)

```
User prompt (in Studio / Apps / Assistant / Excel / email)
   │
   ▼
Intent + context collection (open workflow, project, user permissions)
   │
   ▼
Generation: workflow XAML / expressions / code / test cases / answers
   │
   ▼
Validators: compile, static analysis, schema checks, sandbox run
   │                        │ fail
   ▼                        └──> repair loop (bounded) ──> fallback / ask user
Suggestion shown to user ──> accept / edit / reject telemetry ──> eval + improvement
```

65. [Likely] Design a feature that generates a UiPath automation workflow from a natural-language description.
    - What is the target representation the model generates, and why?
    - How do you validate that the generated workflow is correct before the user runs it?
    - How do you measure quality beyond "it compiles"?
66. [Likely] How would you build an assistant that writes and fixes expressions and code inside Studio, given the project's context?
    - What context do you retrieve, and how do you fit it in the window?
67. [Likely] Design automated test-case generation for automations from requirements documents.
    - How do you measure coverage and usefulness of generated tests?
68. [Likely] How would you use accept/edit/reject telemetry to improve Autopilot without leaking customer data across tenants?
69. [Likely] Autopilot answers questions about a customer's processes and data. How do you design retrieval and permission enforcement?
    - How do you stop the assistant from revealing data the user cannot otherwise access?
70. [Likely] How would you prioritise model spend across Autopilot features with very different value per call?

G. Agent Builder, coded agents and agent evaluation

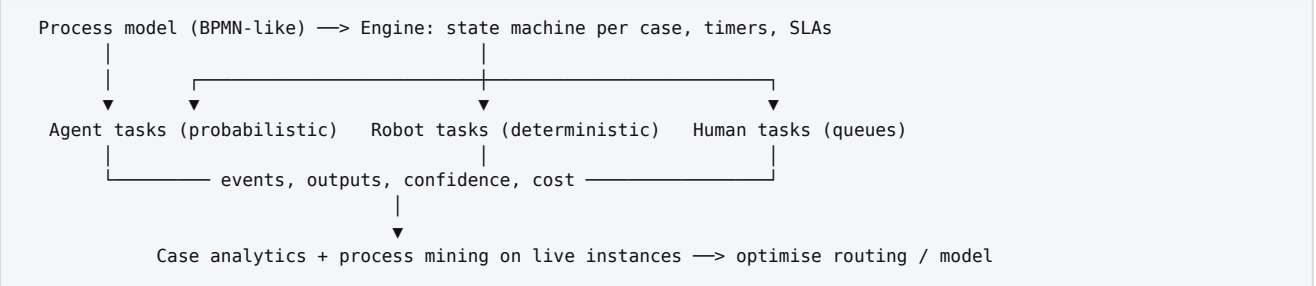


71. [Likely] Design Agent Builder so that a citizen developer can build a reliable agent without being an ML expert.
    - Which decisions do you take away from the author, and which do you expose?
    - What defaults (models, guardrails, escalation) do you ship?
72. [Likely] [17] [18] Design the evaluation system for agents: test sets, simulated tools, judges and release gates.
    - How do you generate realistic test cases when the author has none?
    - How do you validate an LLM-as-judge, audit it against human labels, and detect judge drift?
    - How do you evaluate the trajectory (tool choice, order, arguments), not just the final answer?
73. [Likely] [17] [19] UiPath's SDK offers output evaluators (exact match, contains, JSON similarity, LLM judge) and trajectory evaluators (tool-call order, count, arguments, outputs, LLM-judge trajectory). For a refund agent, choose the evaluators per behaviour and design how their scores combine into a single release gate.
    - The tool-call-order evaluator gives partial credit through the longest common subsequence. When does that partial credit hide a dangerous ordering error (for example, refund before verification)?
    - Which checks must be hard gates rather than averaged scores?
74. [Likely] [18] How would you simulate tools (robots, APIs) during agent testing so evaluation is safe and repeatable? UiPath's trajectory-simulation evaluator mocks tool responses from instructions while the agent does not know it is being simulated.
    - How do you check that the simulator itself behaved as instructed, so a failure is the agent's fault and not the mock's?
    - If the simulator and the agent use the same model family, what correlated errors do you worry about?
75. [Likely] [17] Customers can register custom evaluators with their own domain logic. How do you let them do that safely, version the evaluators, and keep scores comparable across months?
76. [Likely] [8] [17] LLM evaluations are noisy. Design the continuous-integration gate that runs an agent's eval set on every prompt, tool or model change.
    - How many runs per case, and what statistical test, before you call something a regression?
    - How do you keep the eval cost per change under a budget?
77. [Likely] Design versioning and release management for agents: prompts, tools, models and context indexes all change independently.
    - What triggers a re-evaluation, and what blocks a release?
78. [Likely] How would you automatically cluster agent failures from production traces and suggest fixes to the author?
79. [Likely] [1] Design the context grounding (retrieval) component that agents use over enterprise content.

- How do you handle permissions, freshness and document-level citations?
- How do you evaluate retrieval separately from generation?

80. [Likely] How would you optimise an agent's prompt and tool descriptions automatically (prompt optimisation) without overfitting to the test set?
81. [Likely] How do you prevent an author from shipping an agent whose evaluation set is too small or too easy?

H. Maestro-style orchestration of long-running agentic processes



82. [Likely] [8] Design an orchestration engine for long-running processes that mix agents, robots and humans, with SLAs measured in days.
    - How do you persist and resume state across failures and deployments?
    - How do you version the process model while thousands of cases are in flight?
83. [Likely] [6] UiPath's docs warn that Maestro may retry a coded function after a transient failure, and advise checking for an existing result by a stable business key before writing. Extend that to agent tasks: on retry, the LLM may choose different tool arguments. How do you get exactly-once business effects?
    - Where do idempotency keys come from when the agent itself decides what to create?
84. [Likely] [6] The same docs recommend returning business errors as typed output fields so the orchestrator can route on them. Design the error and uncertainty taxonomy for agent tasks so that Maestro can route "not sure", "policy blocked", "tool down" and "bad input" differently.
85. [Likely] How would you design dynamic routing, where the orchestrator chooses agent, robot or human for each case at run time?
    - What signals feed the router, and how do you learn the routing policy safely (bandits, offline RL, rules)?
    - How do you evaluate a routing change before it goes live?
86. [Likely] Where should "agentic" decisions live: inside the agent, inside the process model, or in the orchestrator? Defend your boundaries.
87. [Likely] [5] [6] Design observability for an orchestrated agentic process: traces, cost per case, latency per step, quality per step. UiPath surfaces `@traced` spans in job traces and Maestro dashboards.
    - What is the trace schema that links process instance, agent run, LLM calls, robot jobs and human tasks?
    - How do you sample traces and strip PII without losing the ability to debug?
    - What would an on-call engineer look at first when case completion drops 10%?

88. [Likely] How would you handle a mid-process LLM outage or a degraded model in an orchestrated process?
    - What is the fallback per task type?
89. [Likely] How would you let an agent propose changes to the process model itself, and how would you govern that?
90. [Likely] How would you design case-level explainability so an auditor can reconstruct why a case was approved?

I. Evaluation, reliability and quality across the platform

91. [Likely] Design a single evaluation framework that covers DU extraction, Communications Mining classification, agent trajectories and computer-use tasks.
    - What is shared (datasets, runners, judges, dashboards) and what must stay specific?
92. [Likely] How do you build and maintain golden datasets when customer data cannot leave the tenant?
    - How would you use synthetic data, and how would you check it is representative?
93. [Likely] How would you run online evaluation and A/B tests for agents in enterprise processes where traffic is low and errors are expensive?
    - When do you use shadow mode, interleaving or staged rollout instead of A/B?
94. [Likely] What is your reliability target for an agent step, and how do you compose step reliability into process-level reliability?
    - How does a 95%-reliable step compound across a 20-step process, and what do you do about it?
95. [Likely] How would you detect silent quality regressions in production when there is no ground truth for most cases?
96. [Likely] How would you measure and reduce hallucination in agents that write to systems of record?

J. Trust, safety, governance and deployment constraints

97. [Likely] [15] Design an AI Trust Layer: a model gateway that enforces PII masking, policy, audit logging and usage quotas for every model call on the platform.
    - Where do you mask PII, and how do you unmask for downstream steps that need it?
    - What latency budget does the gateway get?
98. [Likely] [15] [16] UiPath's guardrails apply at three scopes (tool, LLM call, whole agent), before and/or after execution, with actions such as log or block, and include deterministic rule checks alongside model-based detectors for PII, harmful content and prompt attacks. Design the guardrail architecture for a regulated agent.
    - Which risks belong at which scope, and why?
    - How do you evaluate the guardrails themselves (false-block rate, missed attacks), and who owns that eval?
99. [Likely] [14] Customers can plug in their own guardrail vendor through a connector. How do you keep policy semantics consistent across vendors, and do you fail open or

fail closed when the vendor is down?

100. [Likely] [3] How would you support customers who require on-prem, sovereign cloud or "bring your own model" deployments?
    - How do you keep evaluation results valid across different underlying models?
101. [Likely] How would you design permissions so an agent can never do more than the human or robot account it acts for?
102. [Likely] How would you meet EU AI Act or sector rules (financial services, healthcare) for an agentic process?
    - What documentation, logging and human oversight would you build in by default?
103. [Likely] Design the audit log for agent actions so it is complete, tamper-evident and still useful for debugging.
104. [Likely] How would you handle data residency when an agent's model provider is in a different region than the customer data?

K. Cost, latency and model strategy

105. [Likely] How would you decide between frontier API models, open-weight models you host, and small specialised models for each part of the platform?
    - What would make you train a UiPath-specific model rather than use a third-party one?
106. [Likely] Design model routing (cascades) to cut cost by 60% while keeping quality on agentic tasks.
    - How do you decide when to escalate from a small model to a large one?
107. [Likely] Design per-run budgets for agents (tokens, tool calls, wall-clock time) with hard caps. What should an agent do when it hits a cap mid-task: stop, summarise and escalate, or degrade to a cheaper model?
    - How do you set the caps per agent without starving the hard cases?
108. [Likely] How would you build capacity planning and cost attribution per tenant, per agent and per process?
109. [Likely] How would you cache LLM results safely in agentic workflows where inputs look similar but differ in important ways?
110. [Likely] What is your approach to distilling frontier-model behaviour into a smaller model for DU or Communications Mining, and how do you check you lost nothing important?

L. Principal-level framing and roadmap probes

111. [Likely] If you owned applied science for agentic automation, what would your 18-month research and product roadmap be?
    - Which bets are high-risk research, and which are engineering execution?
    - How would you staff and sequence them?
112. [Likely] What are the three biggest open research problems for enterprise agents, and how would UiPath's data position help solve them?
113. [Likely] Pick one UiPath AI product. What would you change in its architecture and why?

114. [Likely] How do you decide when a capability should be a platform service versus a product-specific model?
115. [Likely] [3] How do you keep the platform competitive as frontier models get better at computer use and tool use out of the box, and as customers bring agents built in any framework?
    - What is UiPath's durable advantage at each layer (orchestration, data, governance, models)?
116. [Likely] How would you design a feedback flywheel from millions of robot and agent executions while respecting tenant data boundaries?

References

Sources 1-22 were fetched and read in this review pass (October 2026) as raw GitHub content. They confirm product and benchmark facts used in question premises. None of them is a candidate interview report.

1. UiPath. "UiPath Python SDK" README (services including processes, queues, Action Center tasks and context grounding). GitHub. https://raw.githubusercontent.com/UiPath/uipath-python/main/README.md
2. UiPath. "UiPath Runtime" README (runtime contracts: tracing, structured errors, durable execution, human-in-the-loop). GitHub. https://raw.githubusercontent.com/UiPath/uipath-runtime-python/main/README.md
3. UiPath. "UiPath Agents Runtime Integrations" README (Google ADK, LangChain, LlamaIndex, Microsoft Agent Framework, OpenAI Agents integrations). GitHub. https://raw.githubusercontent.com/UiPath/uipath-integrations-python/main/README.md
4. UiPath. "UiPath MCP Python SDK" README (hosting MCP servers on the UiPath platform). GitHub. https://raw.githubusercontent.com/UiPath/uipath-mcp-python/main/README.md
5. UiPath. "Python Coded Agents" (coded agents versus coded functions, supported frameworks, tracing in Maestro dashboards). GitHub. https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath/docs/core/agents.md
6. UiPath. "Coded functions" documentation (Maestro Service Task, typed error outputs, idempotency under Maestro retries). GitHub. https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath/docs/core/functions.md
7. UiPath. "UiPath LangChain Python SDK" README (LangGraph agents, human-in-the-loop through Action Center). GitHub. https://raw.githubusercontent.com/UiPath/uipath-langchain-python/main/README.md
8. UiPath. "Tool-Calling Suspend/Resume Sample" README (interrupt, checkpoint, resume, `uipath eval`). GitHub. https://raw.githubusercontent.com/UiPath/uipath-langchain-python/main/samples/tool-calling-suspend-resume/README.md
9. UiPath. "Email Triage Agent" sample README (long-running agent suspended on an Integration Service event). GitHub. https://raw.githubusercontent.com/UiPath/uipath-langchain-python/main/samples/email-triage-agent/README.md
10. UiPath. "Support Ticket Classification System" sample README (label plus confidence, Action Center approval). GitHub. https://raw.githubusercontent.com/UiPath/uipath-langchain-python/main/samples/ticket-classification/README.md

11. UiPath. "AML Alert Triage" coded agent sample README (LLM extracts and explains, a typed classifier decides). GitHub. https://raw.githubusercontent.com/UiPath/uipath-langchain-python/main/samples/aml-alert-triage-agent/README.md
12. UiPath. "Human-in-the-Loop Inbox Server" sample README. GitHub. https://raw.githubusercontent.com/UiPath/uipath-langchain-python/main/samples/hitl-inbox-server/README.md
13. UiPath. "Multi-Agent Supervisor System" sample README. GitHub. https://raw.githubusercontent.com/UiPath/uipath-langchain-python/main/samples/multi-agent-supervisor-researcher-coder/README.md
14. UiPath. "Joke Agent: Bring Your Own Guardrail" sample README (customer-managed guardrail vendor through Integration Service; agent and LLM scopes). GitHub. https://raw.githubusercontent.com/UiPath/uipath-langchain-python/main/samples/joke-agent-bring-your-own-guardrail/README.md
15. UiPath. "Guardrails" documentation (tool, LLM and agent scopes; pre/post stages; PII, harmful content, prompt attacks). GitHub. https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath/docs/core/guardrails.md
16. UiPath. Deterministic guardrails service, `uipath-core` source. GitHub. https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath-core/src/uipath/core/guardrails/_deterministic_guardrails_service.py
17. UiPath. "Agent Evaluations" documentation (output-based, trajectory-based and custom evaluators). GitHub. https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath/docs/eval/index.md
18. UiPath. "LLM Judge Trajectory Evaluators" documentation (trajectory and tool-simulation judges). GitHub. https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath/docs/eval/llm_judge_trajectory.md
19. UiPath. "Tool Call Order Evaluator" documentation (strict versus LCS-based scoring). GitHub. https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath/docs/eval/tool_call_order.md
20. UiPath. Documents service source, `uipath-platform` (classify, DU Modern / IXP extraction, validation actions). GitHub. https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath-platform/src/uipath/platform/documents/_documents_service.py
21. XLANG Lab. "OSWorld" README (OSWorld-Verified release of 2025-07-28; maintainer-run verified leaderboard). GitHub. https://raw.githubusercontent.com/xlang-ai/OSWorld/main/README.md
22. XLANG Lab. "OSWorld evaluation_examples" README (task schema with execution-based evaluators). GitHub. https://raw.githubusercontent.com/xlang-ai/OSWorld/main/evaluation_examples/README.md
23. UiPath Documentation portal (official product documentation for Document Understanding, IXP, Communications Mining, Process Mining, Agents and Maestro). Not reachable in this pass (egress policy); listed as the place to check product names. https://docs.uipath.com

05. Coding and ML Coding for System Design Components: Building the Pieces of Document AI, Agent and Retrieval Systems

How to read this chapter. You asked for ML system design, so this chapter does not cover general LeetCode-style coding. It covers coding and ML coding questions where you implement one part of an ML system that you could be asked to design elsewhere in the loop: document and PDF processing, extraction and tagging, retrieval, agent tool loops, evaluation, and serving. Each section opens with a plain-text block diagram that shows which box of the system you are coding. Diagrams are plain text, so they render the same in Markdown and in a PDF export.

Tagging system. Every question carries one of two tags:

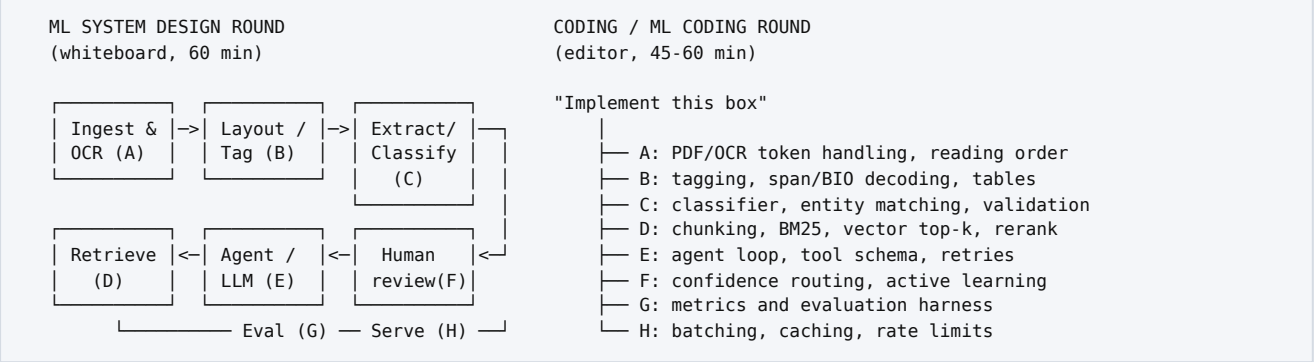
- **[Reported] [n]**: a community-maintained list of LeetCode problems tagged to UiPath links this problem to UiPath interviews. These lists [3][4][5][6] are compiled from LeetCode's company tags. They do not name the role or level, and only one of them [4] has a date window (the last six months before its snapshot). Treat them as evidence that UiPath interviews use the problem, not that a Principal Applied Scientist was asked it. The sub-bullets under each [Reported] problem are [Likely] follow-up probes that connect the algorithm to a box in the system design diagram.
- **[Likely]**: an inference from the role, the level and UiPath's AI products (Document Understanding, Communications Mining, agentic automation, Autopilot, the AI Trust Layer, computer vision for UI automation), or from UiPath's public open-source agent SDKs [7]-[13]. No candidate has reported it.
- Indented sub-bullets are follow-up probes an interviewer may use to push a strong first answer toward the principal bar.

Research limits (please read). The shared web-search budget for this workflow was used up before this review, and DNS lookups failed for glassdoor.com, leetcode.com, uipath.com, docs.uipath.com, reddit.com, teamblind.com, ambitionbox.com, geeksforgeeks.org, interviewquery.com and arxiv.org. The only sources that could be fetched were raw files on GitHub and PyPI. So the [Reported] tags rest only on the UiPath-tagged LeetCode lists. No candidate write-up of a UiPath ML coding round could be read, and no ML-specific question below is tagged [Reported]. The format notes are inferred, not confirmed. Ask your recruiter whether the coding round is general algorithms, ML coding or both, and which language and environment to expect.

Format notes (inferred, not confirmed)

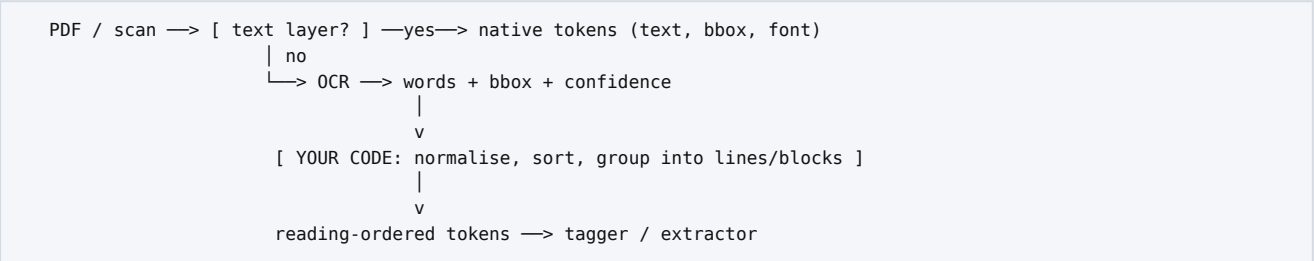
- **Duration:** likely 45-60 minutes, in a shared editor or notebook, usually Python. There may be no execution environment, so you should be able to dry-run code by hand.
- **Interviewer:** likely a senior or staff scientist or ML engineer, not a principal.
- **What is assessed at principal level:** code that is correct and readable, without needing to be the fastest solution. The emphasis likely shifts toward whether you can turn a design box into a clean interface, pick sensible data structures, write tests and edge cases without being prompted, and explain how the component behaves at scale and how it fails. Expect the interviewer to connect the code to a system ("now this runs on 10 million pages a day").
- **Two common variants:** (a) implement an ML primitive from scratch (attention, a metric, a decoding loop); (b) implement a pipeline component with a small amount of ML logic (chunker, matcher, tagger, evaluator).
- **A third variant with some evidence:** a standard algorithm problem. The UiPath-tagged LeetCode lists [3][4][5][6] lean towards graph search (BFS, union-find), trees and dynamic programming at medium and hard difficulty. At principal level, expect the interviewer to ask how the algorithm maps to a real component (Section L).

How the coding round connects to the system design diagram



Questions

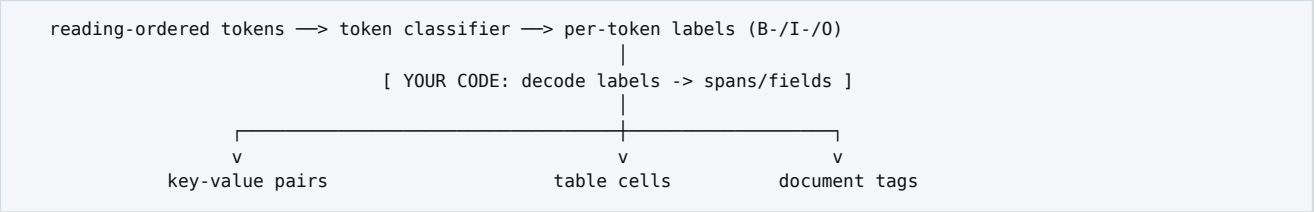
A. PDF and document ingestion: tokens, boxes and reading order



1. [Likely] Given a list of OCR words with bounding boxes `(x0, y0, x1, y1)` and text, write a function that groups them into lines and returns the lines in reading order.
    - How do you handle slight skew, where words on the same line have different y values?
    - What changes for a two-column page?
2. [Likely] Write a function that detects whether a page has two or more text columns from word bounding boxes alone.

    - How would you test it without labelled data?
3. [Likely] Implement intersection over union for two bounding boxes, then a function that merges overlapping boxes from two OCR engines into one set of words.
    - How do you decide which engine's text wins when the boxes overlap but the text differs?
4. [Likely] Given native PDF text tokens and OCR tokens for the same page, align them so each OCR word maps to its native token or to nothing.
    - What is the complexity of your alignment, and how would you make it faster for dense pages?
5. [Likely] Write a function that normalises bounding boxes from different page sizes and rotations (0, 90, 180, 270 degrees) into one coordinate system.
6. [Likely] Implement a de-hyphenation and line-joining step that rebuilds paragraphs from line-broken OCR text.
    - Which cases would wrongly join two words, and how would you guard against them?
7. [Likely] Write a streaming page iterator for a multi-thousand-page PDF batch that never holds more than N pages in memory.
    - How do you resume after a crash halfway through a file?
8. [Likely] Given per-word OCR confidences, compute a page-level quality score and decide whether to re-OCR the page with a slower engine.
    - What threshold policy would you choose, and how would you tune it?

B. Tagging and structure: spans, key-value pairs and tables



9. [Likely] Given tokens and BIO labels, write a function that decodes them into typed entity spans with start and end offsets.
    - What do you do with an I- label that follows O or a different entity type?
10. [Likely] Implement Viterbi decoding for a linear-chain CRF given emission and transition score matrices.
    - How would you add a constraint that forbids illegal BIO transitions?
11. [Likely] Write span-level precision, recall and F1 for entity extraction, with exact match and with partial (overlap) match.
    - Which one would you report to a product team, and why?
12. [Likely] Given word boxes for a page, write a function that links each detected key ("Invoice No.", "Date") to its most likely value using spatial rules (right of, below).
    - How would you replace the rules with a learned scorer while keeping the same interface?
13. [Likely] Given detected horizontal and vertical ruling lines and word boxes, reconstruct a table as a 2D grid of cell strings.
    - How do you handle merged cells and tables with no ruling lines?
14. [Likely] Write a function that matches table header cells to a target schema (for example "Qty", "Quantity", "Units") using fuzzy matching.

    - Where would an embedding model beat string similarity here, and where would it be worse?
15. [Likely] Implement a multi-label document tagger head on top of a pooled encoder output in PyTorch, including the loss.
    - How would you pick a decision threshold per tag?
16. [Likely] Given a hierarchy of document tags (for example Finance > Invoice > Utility invoice), write a function that makes predictions consistent with the hierarchy.
17. [Likely] Write a function that splits a multi-document PDF (several invoices scanned together) into separate documents from per-page classifier scores.
    - How would you formulate this as a sequence problem instead of page-by-page thresholds?

C. Extraction, normalisation and validation

18. [Likely] Write parsers that normalise extracted dates, currency amounts and numbers across locales ("1.234,56", "1,234.56", "12/03/2025").
    - How do you resolve an ambiguous date like 03/04/2025 using other fields on the same document?
19. [Likely] Implement a validator that checks invoice line items sum to the subtotal, and subtotal plus tax equals the total, within a tolerance, and reports which field is most likely wrong.
20. [Likely] Given extracted vendor names and a master vendor list of 1 million entries, write a matcher that returns the top 5 candidates quickly.
    - Which index would you build: n-gram, phonetic, or embedding? Why?
21. [Likely] Write a function that turns LLM output into a strict JSON object for a given schema, with repair for common errors (trailing commas, missing quotes, extra text).
    - When would you prefer constrained decoding over repairing output after generation?
22. [Likely] Implement a field-level confidence aggregator that combines OCR confidence, model probability and validation-rule results into one score.
    - How would you check that the combined score is calibrated?
23. [Likely] Write a regex-plus-model hybrid extractor for IBANs or tax IDs, including checksum validation.
24. [Likely] Given extraction results from three models for the same document, write an ensembling function that votes per field.
    - How do you break ties, and how do you log disagreement for later review?

D. Retrieval and RAG components

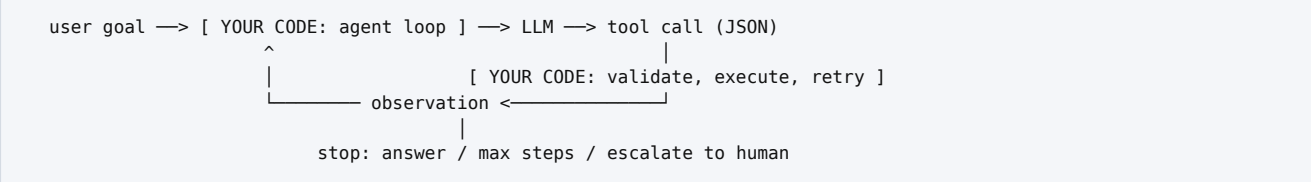
```
documents ──> [ YOUR CODE: chunk ] ──> embed ──> vector index
                                                      |
query ──> embed ──> [ YOUR CODE: top-k + BM25 + fusion ] ──> [ rerank ] ──> LLM context
```

25. [Likely] Implement a chunker that splits a document into overlapping chunks by token count while not breaking sentences, and that keeps page numbers and section titles as metadata.
    - How would you chunk a long table?
26. [Likely] Implement BM25 scoring from scratch over a small corpus.

- How do you update document frequencies when documents are added and deleted?

27. [Likely] Implement exact top-k cosine similarity search with NumPy over a matrix of embeddings, and then explain how you would scale it.
    - Write the version that processes queries in batches without running out of memory.
28. [Likely] Implement reciprocal rank fusion to merge BM25 and dense retrieval results.
29. [Likely] Implement maximal marginal relevance (MMR) to pick diverse chunks for an LLM context window under a token budget.
30. [Likely] Write recall@k, MRR and nDCG@k for a retrieval evaluation set.
    - How would you build the relevance labels for enterprise documents you cannot read?
31. [Likely] Implement a simple inverted file (IVF) index: k-means over embeddings, assign vectors to lists, search the nearest n lists.
    - How does recall change with the number of lists probed?
32. [Likely] Write a permission-aware retrieval filter so a user only gets chunks from documents they can access, applied before ranking.
    - Why is filtering after ranking a problem?
33. [Likely] Implement a deduplication step for near-duplicate chunks using MinHash or SimHash.

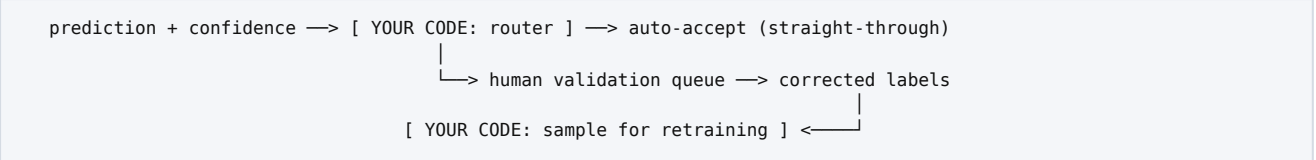
E. LLM and agent components



34. [Likely] Write a minimal agent loop: call a model, parse a tool call, run the tool, append the observation, and stop on a final answer or a step limit.
    - Where do you enforce budgets for tokens, time and money?
35. [Likely] Implement a tool registry that builds a JSON schema from Python function signatures and validates model arguments against it.
36. [Likely] Write retry logic for tool calls with exponential backoff and jitter, and separate retryable from non-retryable errors.
    - How do you avoid running a non-idempotent action, such as sending a payment, twice?
37. [Likely] Implement greedy, temperature, top-k and top-p (nucleus) sampling from a logits vector.
    - What breaks numerically with very low temperature, and how do you fix it?
38. [Likely] Implement beam search for a toy sequence model.
39. [Likely] Write a token-budget manager that fits a system prompt, retrieved chunks and conversation history into a context limit, dropping or summarising the least useful parts.
40. [Likely] Implement a simple planner that turns a task into a DAG of steps and runs independent steps in parallel.
    - How do you handle one failing branch?

41. [Likely] Write a PII redaction step (emails, phone numbers, card numbers, names) that runs before text is sent to an external LLM, and that can restore the values in the response.
    - How do you measure its recall?
42. [Likely] Implement a prompt-injection heuristic filter for content read from documents and emails before it reaches an agent.
    - Why is a filter alone not enough, and what structural defence would you code instead?
43. [Likely] Write a semantic cache for LLM calls keyed by embedding similarity, with a threshold and time-to-live.
    - What is the risk of serving a cached answer for a slightly different query?

F. Human-in-the-loop, confidence routing and active learning



44. [Likely] Write a router that sends each document to straight-through processing or human review based on field confidences and a target precision.
    - How would you choose the thresholds from a labelled validation set?
45. [Likely] Implement temperature scaling for calibration and compute expected calibration error (ECE).
46. [Likely] Implement uncertainty sampling (least confidence, margin, entropy) to choose which documents a human should label next.
    - How do you avoid choosing many near-identical documents?
47. [Likely] Write a function that merges human corrections into a training set while tracking label provenance and annotator disagreement.
48. [Likely] Compute Cohen's kappa for two annotators on a tagging task.
49. [Likely] Given a stream of human corrections, write a drift detector that raises an alert when the correction rate for a field moves beyond normal variation.

G. Evaluation harnesses and metrics

50. [Likely] Write a bootstrap confidence interval for the F1 difference between two extraction models on the same test set.
    - Why should you resample documents rather than fields?
51. [Likely] Implement a field-level evaluation for document extraction that handles missing fields, extra fields and multi-valued fields such as line items.
52. [Likely] Implement character error rate and word error rate for OCR output using edit distance.
53. [Likely] Write an LLM-as-judge harness: run a judge prompt over model outputs, parse a score, and measure agreement with a small human-labelled set.
    - How would you detect position bias or length bias in the judge?
54. [Likely] Implement a task-success evaluator for agent trajectories that checks final state rather than the exact sequence of actions.
55. [Likely] Write a stratified sampler for an evaluation set across document types, languages and vendors with a fixed labelling budget.

56. [Likely] Implement a paired sign test or McNemar's test to compare two classifiers on the same examples.

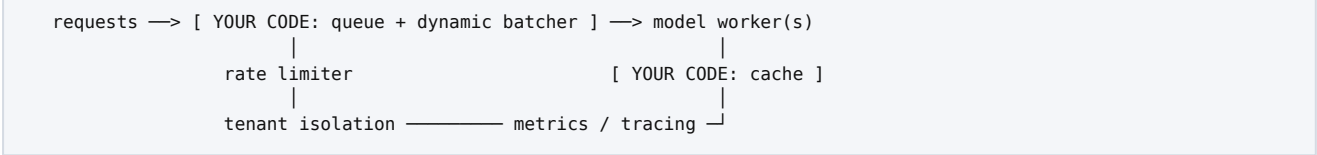
H. ML primitives from scratch

57. [Likely] Implement scaled dot-product attention and multi-head attention in NumPy or PyTorch, including a causal mask.
    - What is the memory cost in sequence length, and how does it affect long documents?
    - Make the softmax numerically stable and fuse it with a cross-entropy loss; what overflows first in half precision?
58. [Likely] Implement 2D sinusoidal or learned position embeddings for tokens with (x, y) layout coordinates, as in layout-aware document models.
    - How would you add bounding box information to a text transformer's input?
59. [Likely] Implement a LoRA adapter layer that wraps a linear layer, with merge and unmerge methods.
60. [Likely] Implement byte-pair encoding training on a small corpus and encoding of new text.
61. [Likely] Implement a contrastive (InfoNCE) loss for training a retrieval embedding model with in-batch negatives.
    - What goes wrong with in-batch negatives when the batch has duplicates?
62. [Likely] Implement non-maximum suppression for UI element or layout region detection.
63. [Likely] Write a training loop in PyTorch with gradient accumulation, mixed precision and checkpointing.

I. Screen and UI understanding components

64. [Likely] Given a list of detected UI elements (type, box, text), write a function that finds the element matching a natural-language target such as "the Submit button under Billing".
    - How would you make it robust to small layout changes between application versions?
65. [Likely] Write a function that builds a containment tree from flat UI element boxes.
66. [Likely] Implement a selector-repair function: when a stored selector no longer matches, find the most similar element on the new screen using attributes and position.
67. [Likely] Given two screenshots' element lists, compute a diff of added, removed and moved elements.

J. Serving, scaling and production code



68. [Likely] Implement a dynamic batcher that groups incoming inference requests up to a maximum batch size or maximum wait time.

    - How do you choose the wait time against a latency SLO?
69. [Likely] Implement a token-bucket rate limiter per tenant for calls to an external LLM provider.
70. [Likely] Implement an LRU cache with a size limit in bytes, then make it safe for concurrent use.
71. [Likely] Write an async client that sends many document pages to a model endpoint with bounded concurrency and collects results in order.
72. [Likely] Write a fallback router that sends a request to a smaller model first and escalates to a larger model when confidence is low.
    - How would you measure whether the router saves money without hurting quality?
73. [Likely] Write a function that shards a large batch of documents across workers so that each worker gets a similar total page count.
74. [Likely] Write structured logging for a model call that records inputs, outputs, latency and model version without logging customer PII.

K. Code review and principal-level judgement in the coding round

75. [Likely] Here is a 60-line extraction pipeline written by a junior scientist. Review it: what is wrong, what is risky, and what would you change first?
    - Which of your comments block the merge and which are suggestions?
76. [Likely] Turn the notebook prototype you just wrote into a module others can use: what interface, tests and configuration would you add?
77. [Likely] Write unit tests for your reading-order or BIO-decoding function, including edge cases.
78. [Likely] Your code works on the sample. The interviewer says it now runs on 10 million pages a day across 500 tenants. What do you change in the code, not only in the architecture?

L. UiPath-tagged algorithm problems, recast as system design components

```
UiPath-tagged problem (graph / tree / DP)        Box in the ML system it maps to
─────────────────────────────────────────        ──────────────────────────────
BFS / multi-source BFS  ──────────────────────> layout grids, UI state exploration (A, I)
Union-find / connected components ────────────> region merging, entity resolution (A, C)
Sliding window / two pointers ────────────────> OCR token streams, request batching (A, J)
Tree DP / tree pruning ───────────────────────> UI element trees, agent plan trees (E, I)
Interval DP / LIS ────────────────────────────> document splitting, box nesting (B, I)
```

79. [Reported] [3][5][6] Escape a Large Maze (LeetCode 1036): decide whether a target cell can be reached on a 10^6 x 10^6 grid with up to 200 blocked cells.

- Why does a BFS bounded by the number of blocked cells give the right answer, and what is the bound?
- An agent explores a large application state graph to reach a target screen. How would you bound its exploration in the same way, and when does the bound stop being valid?

80. [Reported] [3][5] Sorting Three Groups (LeetCode 2826): find the minimum number of removals that make an array of values in {1, 2, 3} non-decreasing.

- Recast it: per-page labels must follow the order cover page, body, appendix. Find the minimum number of relabels, and compare it with constrained Viterbi decoding.

81. [Reported] [3][5][6] As Far from Land as Possible (LeetCode 1162): return the largest distance from any water cell to the nearest land cell.

- Use a multi-source BFS distance transform on a rasterised page to find column gutters and white-space separators. How do you handle scan noise?

82. [Reported] [3][5] Integer to Roman (LeetCode 12), then the reverse parser.

- Write a robust parser for clause numbering in contracts ("IV.2(b)", "4.2.b", "Section iv") that returns a hierarchical section key. How do you test it on OCR errors such as "IV" read as "1V"?

83. [Reported] [3][5] Making A Large Island (LeetCode 827): find the largest island possible after changing at most one 0 to 1.

- Map it to connected-component labelling of text regions where one small gap splits a paragraph. How do you find the merge that helps most without merging two columns by mistake?

84. [Reported] [3][5] Boats to Save People (LeetCode 881): use the fewest boats, each with a weight limit and at most two people.

- Generalise it to packing documents of different token lengths into GPU batches under a memory limit. Why does the greedy two-pointer answer stop being optimal when a batch can hold more than two items?

85. [Reported] [3][5] Satisfiability of Equality Equations (LeetCode 990): decide whether a set of "==" and "!=" constraints can all hold.

- Turn it into vendor entity resolution with must-link and cannot-link constraints from human reviewers. What do you return when the constraints conflict, and to whom?

86. [Reported] [3][5] Longest Repeating Character Replacement (LeetCode 424): find the longest substring that becomes one repeated character after at most k replacements.

- On an OCR token stream with per-token confidence, find the longest span with at most k low-confidence tokens, as a candidate for one human review task.

87. [Reported] [3][5] Collect Coins in a Tree (LeetCode 2603): find the minimum number of edge traversals that collect all coins within distance 2 and return to the start.

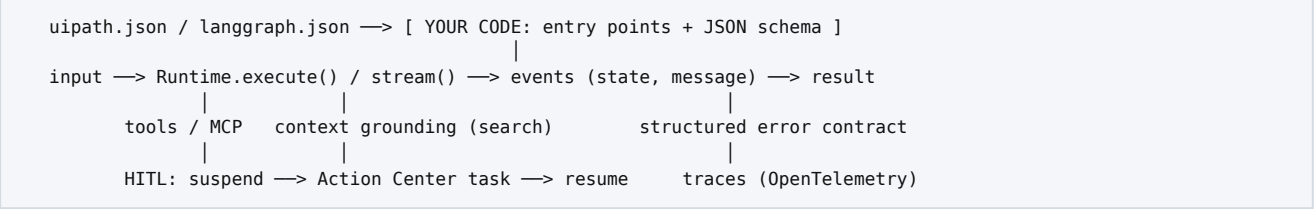
- Prune an accessibility or DOM tree of thousands of UI elements to the smallest subtree that keeps every element an agent may need, before it goes into an LLM prompt.

88. [Reported] [3][5] Burst Balloons (LeetCode 312): find the maximum coins from bursting balloons in the best order, using interval DP.
    - Use interval DP to split a scanned batch into documents so that the total segment score is highest. What is the complexity, and how would you reduce it

for a 2,000-page batch?

89. [Reported] [3][4][5] Min Stack (LeetCode 155): design a stack that returns the minimum element in O(1) time. It is one of only two problems in the six-month window of [4], and the only problem in the six-month list in the same repository as [5].
    - Extend it into an agent checkpoint stack that supports undo and returns the lowest-cost recoverable state in O(1) time. What must be persisted for a long-running agent that resumes after a human approval?
90. [Reported] [3][5] Subsets II (LeetCode 90): return all subsets of a list that contains duplicates, without duplicate subsets.
    - Use it to enumerate tool-set or prompt-component ablations where some components are interchangeable. How do you cut the search when 2^n runs are too expensive to evaluate?
91. [Reported] [3][5] Reorder List (LeetCode 143): reorder a linked list in place to L0, Ln, L1, Ln-1, and so on.
    - Write an in-place interleaving of results from two ranked retrievers that removes duplicates and keeps the reordering stable. When does round-robin interleaving beat score fusion?
92. [Reported] [3][5] Russian Doll Envelopes (LeetCode 354): find the longest chain of envelopes that nest inside each other, in O(n log n) time.
    - Apply it to bounding boxes: find the deepest chain of nested layout regions (page, section, table, cell). Why does the trick of sorting by width ascending and height descending matter?
93. [Reported] [3][4] Binary Tree Cameras (LeetCode 968): find the minimum number of cameras that monitor every node of a binary tree. This problem is in the six-month window of [4].
    - Recast it: choose the fewest human-approval checkpoints in an agent's plan tree so that every irreversible action is covered by an approval at that step or an adjacent one.

M. Agent runtime components on UiPath's public SDK contracts



94. [Likely] [9] Implement a class that satisfies a runtime protocol with `get_schema`, `execute`, `stream` and `dispose`, for an agent that extracts fields from an invoice PDF. `stream` should yield state events before the final result.
    - What must stay true between `execute` and the last event of `stream`, and how would you test it?
95. [Likely] [9] Write a runtime factory that finds entry points from a config file, creates runtimes lazily, and reuses expensive resources (model clients, indexes) across executions without leaking state between tenants.
96. [Likely] [9] Implement a structured error model with a code, a title, a detail and a category (user or system), and map tool, model and validation exceptions onto it.
    - Which errors should an orchestrator retry, and which should go to a human?

97. [Likely] [8][11] Implement durable suspend and resume for a human-in-the-loop step: the agent creates a review task, saves its state, exits, and continues from the same point when the reviewer answers.
    - How do you make the resume idempotent if the callback arrives twice, or after the agent code has changed?
98. [Likely] [7] Write a wrapper around a context-grounding search call (`index name`, `query`, `number of results`) that adds query rewriting, deduplication and a citation check before chunks reach the LLM.
    - How would you test the wrapper offline without access to the real index?
99. [Likely] [10] Expose two document-processing functions (classify and extract) as tools on a small MCP server, with input and output schemas, timeouts and size limits for uploaded PDFs.
    - How does the agent side tell a tool failure apart from a tool result that says "no fields found"?
100. [Likely] [9][12] Wrap any runtime so that each execution's spans and logs are isolated when several runtimes run in the same process. Record model latency, token counts and tool calls as OpenTelemetry-style spans. - What breaks if you use a global logger, and how do you show the bug in a test?
101. [Likely] [7] Write a transaction-queue worker that takes documents from a queue, runs extraction, and marks each item as successful, failed with a retryable error, or failed with a business exception. - How do you avoid processing an item twice when a worker dies after the model call but before the status update?
102. [Likely] [7][9] Write a function that reads Python entry-point functions with type hints or Pydantic models and creates the input and output JSON schemas that a packaging step would need. - Which Python types have no clean JSON Schema equivalent, and what do you do with them?
103. [Likely] [11] Write an adapter layer so that one evaluation harness can run agents built with two different frameworks (for example LangGraph and LlamaIndex) behind one runtime interface. - What behaviour leaks through the adapter (streaming, interrupts, tool-call formats), and how do you keep the evaluation fair?

References

Sources marked "fetched" were read in this review (October 2026). Sources marked "not retrieved" are official pages that could not be reached because DNS lookups failed. They are listed only for product context and support no question.

1. UiPath Careers, uipath.com (official careers page; not retrieved). https://www.uipath.com/company/careers
2. UiPath Blog, uipath.com (official product and AI announcements; not retrieved). https://www.uipath.com/blog
3. liquidslr, "leetcode-company-wise-problems", UiPath "All" list (15 problems with LeetCode frequency scores; fetched). https://raw.githubusercontent.com/liquidslr/leetcode-company-wise-problems/main/UiPath/5.%20All.csv
4. liquidslr, "leetcode-company-wise-problems", UiPath "Six Months" list (Min Stack and Binary Tree Cameras; the 30-day and 3-month lists were empty; fetched). https://raw.

githubusercontent.com/liquidslr/leetcode-company-wise-problems/main/UiPath/3.%20Six%20Months.csv
5. snehasishroy, "leetcode-companywise-interview-questions", UiPath all-time list (14 problems; fetched). https://raw.githubusercontent.com/snehasishroy/leetcode-companywise-interview-questions/master/uipath/all.csv
6. krishnadey30, "LeetCode-Questions-CompanyWise", UiPath all-time list (older snapshot: Escape a Large Maze, As Far from Land as Possible; fetched). https://raw.githubusercontent.com/krishnadey30/LeetCode-Questions-CompanyWise/master/uipath_alltime.csv
7. UiPath, "uipath-python" README: Python SDK and CLI, with services for processes, assets, buckets, context grounding, jobs, queues and Action Center tasks (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-python/main/README.md
8. UiPath, "uipath-langchain-python" README: LangGraph agents on the UiPath platform, with human-in-the-loop through Action Center (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-langchain-python/main/README.md
9. UiPath, "uipath-runtime-python" README: runtime protocol (get_schema, execute, stream, dispose), factory, structured errors, execution context, tracing and durable execution (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-runtime-python/main/README.md
10. UiPath, "uipath-mcp-python" README: hosting MCP servers on the UiPath platform (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-mcp-python/main/README.md
11. UiPath, "uipath-integrations-python" README: agent runtime integrations for LangChain, LlamaIndex, Google ADK, OpenAI Agents and more, with human-in-the-loop through Action Center (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-integrations-python/main/README.md
12. UiPath, "uipath-dev-python" README: developer console for running and debugging runtimes, with OpenTelemetry traces (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-dev-python/main/README.md
13. PyPI, "uipath" package metadata (version 2.14.37 at the time of this review; fetched). https://pypi.org/pypi/uipath/json

06. Principal-Level Leadership Through ML System Design: Technical Strategy, Design Reviews, Influence, Research-to-Product and Roadmaps

How to read this chapter. You asked for ML system design only, so this chapter does not cover general behavioral questions. It covers the leadership part of the Principal Applied Scientist bar *as it shows up in ML system design*: setting the technical strategy for an ML platform, running and defending design reviews, getting other teams to adopt a shared architecture, moving a research prototype into a production system, and planning a multi-quarter roadmap for systems such as Document Understanding (PDF and document tagging and extraction), agents and Communications Mining. Diagrams are plain-text block diagrams, so they render the same in Markdown and in a PDF export.

Tagging system. Every question carries one of two tags:

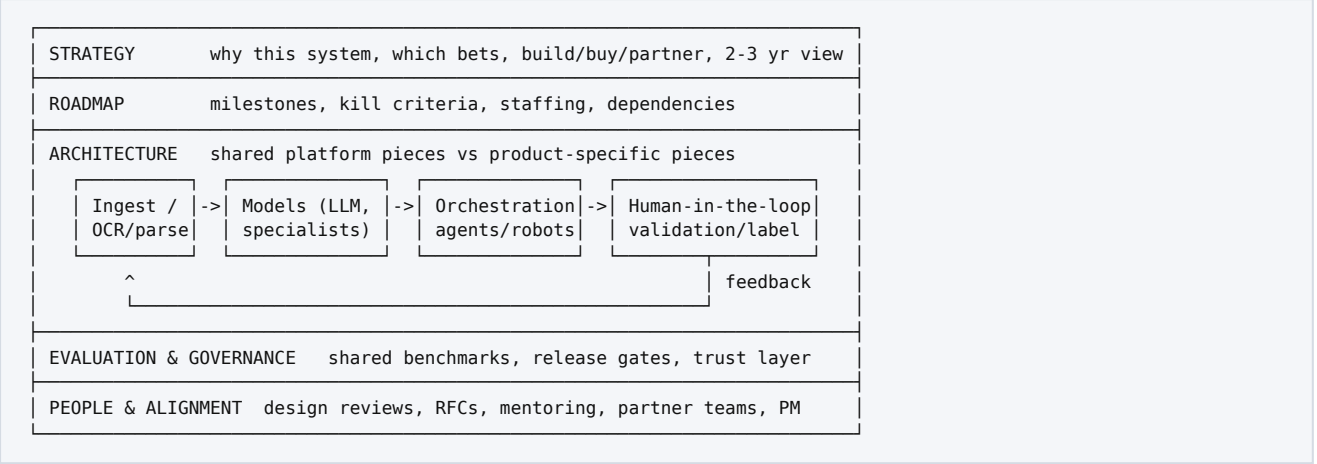
- **[Reported] [n]**: a candidate report, question bank or official page links this question, or this round type, to UiPath. The number points to the References list.
- **[Likely]**: an inference from the role, the level and UiPath's AI products (agentic automation, Document Understanding, Communications Mining, Autopilot, the AI Trust Layer, computer vision for UI automation). No candidate has reported it.
- Indented sub-bullets are follow-up probes an interviewer may use to push a strong first answer toward the principal bar.

Research limits (please read). In the latest review run the shared web-search budget was used up, and direct fetches of uipath.com, docs.uipath.com, Glassdoor, Indeed, Levels.fyi, LeetCode Discuss and Blind were all blocked (DNS failure or proxy denial). Only GitHub raw files could be read. That means no candidate write-up of a UiPath leadership or design round could be checked, so **no question below is tagged [Reported]**, and this chapter does not invent reports. Every question is **[Likely]**. Where a [Likely] question rests on a fact from a page that *was* read (for example UiPath's public agent SDKs, Action Center human-in-the-loop, context grounding, MCP hosting or runtime tracing), the question ends with *(context: [n])*. That marker points to the source of the product fact. It does not mean a candidate reported the question. The format notes describe the usual principal-scientist loop at enterprise AI companies of UiPath's size; they are not a confirmed UiPath process. Confirm round names and durations with your recruiter.

Format notes (inferred, not confirmed)

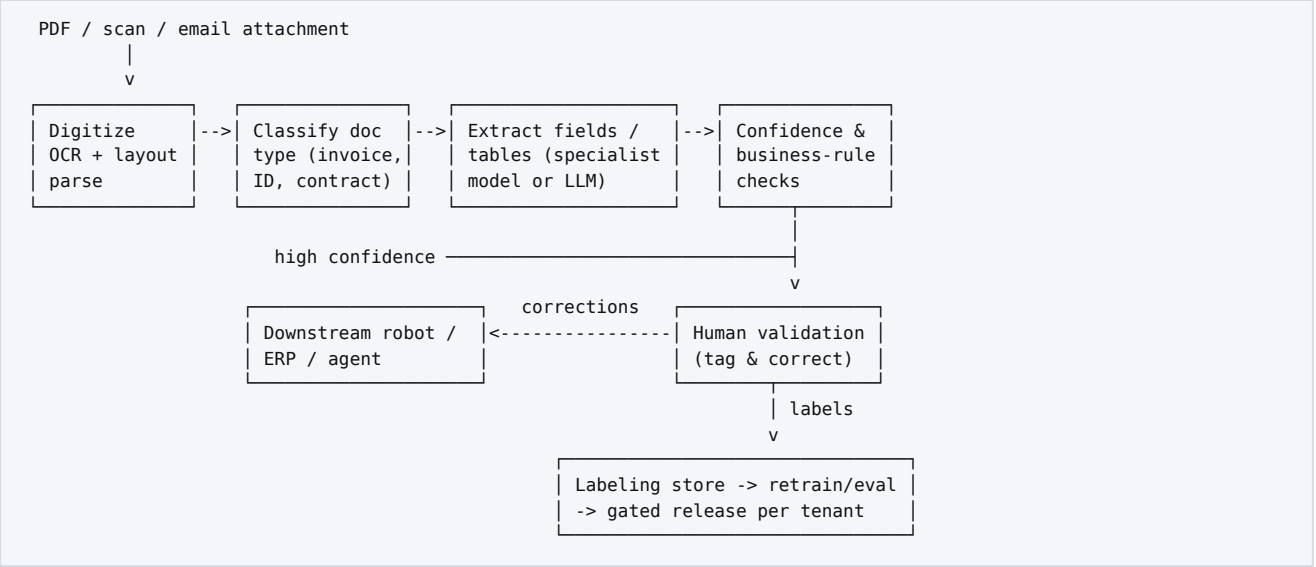
- **Where it happens.** Leadership is unlikely to be tested only in a values round. At principal level it is probably tested *inside* the ML system design round ("now you have a team of six, what do you build first?"), in the research-presentation Q&A ("how did you get this adopted?"), and in a hiring-manager or cross-functional round with a director, head of AI or product leader. Each is typically 45-60 minutes. (Inferred; see chapter 01.)
- **Interviewers.** Likely a principal scientist or architect for the design round, and a director or VP-level AI leader plus a product manager for the strategy and cross-functional rounds.
- **Bar assessed at principal level.** (1) You set direction for a *platform*, not a single model. (2) You make irreversible architecture decisions with evidence and can say when you would reverse them. (3) Other teams adopt your design without reporting to you. (4) You turn research into shipped, measured product value. (5) You grow other scientists' design judgement. UiPath's published values are commonly cited as Humble, Bold, Immersed and Fast; the careers page [1] could not be reached in this run, so check the current wording yourself. Public interview-prep guides also note that GenAI and agent system design rounds in 2025-2026 tend to weigh evaluation, cost, latency, guardrails and human fallback above the architecture diagram itself [11].

Reference diagram: the leadership layers around an ML system design



Reference diagram: a document tagging and extraction (PDF) system, as a leadership prompt

Interviewers can use a familiar system like this to ask leadership questions ("who owns each box, what do you build first?"). This is the shape of the system, not an answer.



Questions

A. Technical strategy for ML systems

1. [Likely] If you joined as the principal scientist for Document Understanding, what would your one-page technical strategy for the next two years say?
    - Which current architecture choices would you keep, and which would you challenge first?
    - What evidence would you gather in your first 90 days before committing to the strategy?
2. [Likely] Specialist extraction models versus a general LLM or vision-language model for PDF and document extraction: how would you set the long-term direction, and how would you explain it to leadership?
    - Under what cost, accuracy or latency numbers would you reverse that decision?
    - How do you avoid stranding customers who have trained custom models on the old stack?
3. [Likely] How would you decide which parts of UiPath's AI stack should be a shared platform and which should stay inside each product team?
    - Give an example of a component you would centralise (for example, evaluation or labeling) and one you would deliberately not centralise.
4. [Likely] Build, buy or partner: how would you decide whether UiPath should train its own models, fine-tune open models or call frontier model APIs for a given capability?
    - How does customer choice of model provider change the architecture?
    - How do on-prem and air-gapped customers change the answer?
5. [Likely] Describe a technical bet you made that most people disagreed with. How did you structure the system so the bet could be tested cheaply?
    - What was the kill criterion, and did you ever use it?
    - Give one bold architecture change that paid off and one that did not. What separated them?
6. [Likely] How would you set the technical strategy for moving from deterministic RPA toward agentic automation without breaking customer trust in reliability?
    - Where must the system stay deterministic, and who decides?

7. [Likely] How do you decide when an ML system needs a rewrite rather than incremental improvement?
    - How did you make that case to people who had built the old system?
8. [Likely] How would you design a model strategy that survives the frontier-model landscape changing every few months?
    - What abstraction layer would you insist on, and what does it cost you?
9. [Likely] What are the two or three ML system metrics you would put on an executive dashboard for an automation platform, and why those?
    - How do you connect straight-through processing rate or agent task success to customer ROI?
10. [Likely] How would you identify the highest-leverage ML investment across Document Understanding, Communications Mining, agents and screen understanding when you can only staff two?
    - What data would you need, and what would you do if product leaders disagreed with your ranking?
11. [Likely] UiPath's agent SDKs let teams and customers bring agents built in LangGraph, LlamaIndex, OpenAI Agents, Google ADK, Microsoft Agent Framework or PydanticAI onto one platform. Would you keep the platform framework-agnostic, or standardise on one framework for agents built in-house? (context: [5][6])
    - What must the shared runtime contract guarantee so that evaluation, tracing and human review work the same way for every framework?
12. [Likely] How would you set the strategy for retrieval ("context grounding") across products: one shared, permission-aware index service, or retrieval owned by each product team? (context: [4])
    - Who owns index freshness, access control and retrieval quality metrics?
13. [Likely] Would you expose document extraction and classification as MCP tools that third-party agents can call? Make the strategic case for and against. (context: [7])
    - How does that change your trust, metering and abuse-prevention design?
14. [Likely] How would you set a cost-per-document or cost-per-agent-run target for the platform, and drive it down across teams you do not manage?
    - Which architecture levers (model routing, caching, distillation, batching) would you push first?
15. [Likely] How would you set a strategy for evaluating agents that customers build in frameworks you do not control?

B. Leading design reviews and architecture decisions

16. [Likely] Walk me through how you run a design review for a new ML system at principal level. What do you require in the design document?
    - What sections do people usually skip, and how do you make them write them?
17. [Likely] A senior engineer proposes a document extraction architecture you think will not scale to multi-tenant use. How do you handle the review?
    - How do you separate "I would do it differently" from "this will fail"?
18. [Likely] Tell me about an architecture decision record you wrote that was later proven wrong. What did you change in your process?
    - How did you tell the people who had followed your recommendation?
19. [Likely] How do you decide which design decisions need broad review and which a team can make alone?

    - How do you classify reversible versus irreversible decisions in an ML system?
20. [Likely] In a design review for an agent platform, which failure modes would you always ask about?
    - How do you make sure evaluation, cost and safety are reviewed, not only the model?
21. [Likely] How do you review an ML system design when you are not the expert in its sub-domain (for example, OCR or speech)?
22. [Likely] Describe a time you blocked a launch on design grounds. What was the evidence and how did you communicate it?
    - What would you have needed to see to approve it?
23. [Likely] How do you keep a large ML architecture coherent when ten teams are each shipping changes to it?
    - What guardrails (interfaces, contracts, shared evaluation) would you put in place?
24. [Likely] Two teams have built overlapping document classification services. How would you decide whether to merge them, and how would you lead that change?
    - How do you handle the team whose system is being retired?
25. [Likely] How do you document an ML system so a new scientist can make safe changes in their first month?
26. [Likely] You are reviewing the design of a long-running agent that pauses for human approval in Action Center and resumes hours later. What do you require on state, idempotency and resume semantics before you approve it? (context: [5][8])
    - What happens if the human edits the agent's proposed action instead of approving it?
27. [Likely] A team proposes adding a new agent-framework integration to the shared runtime protocol. What conformance and evaluation checks must it pass before it ships? (context: [6][8])
28. [Likely] You want every agent run to emit traces in one shared format (LLM calls, tool calls, human tasks). How would you get teams to agree on the span schema, and what would you do with teams that have already shipped their own? (context: [8][9])

C. Influencing without authority across teams

29. [Likely] You want every product team to adopt one shared evaluation framework for agents and extraction models. None of them report to you. How do you get adoption?
    - What would you build first to make adoption cheaper than not adopting?
    - What do you do about a team that refuses?
30. [Likely] How would you convince the platform engineering team to change their serving infrastructure to support a new model architecture your research needs?
    - How do you quantify the benefit in their terms rather than yours?
31. [Likely] Tell me about a time you changed a product leader's mind about an ML system design using data.
    - What data did not persuade them, and why?
32. [Likely] How do you align security, legal and product on PII handling in a document or communications ML pipeline?
    - Where would you put masking in the architecture, and how do you get sign-off?
33. [Likely] A partner team owns the OCR component your extraction quality depends on, and their priorities differ from yours. How do you get the improvements you need?

34. [Likely] How do you build credibility with engineering teams when you join as a principal scientist from outside?
35. [Likely] How do you resolve a disagreement between research and engineering about whether a model is ready for production?
    - What shared release gate would you propose so the argument does not repeat?
36. [Likely] Describe how you would get field and customer-success teams to feed real customer failures back into the ML system.
    - What tooling or process in the system design makes that feedback loop work?
37. [Likely] How do you influence the direction of a system owned by a peer principal who disagrees with you?
38. [Likely] How would you convince the team that owns the human-task (Action Center) experience to change its schema so validator corrections become usable training and evaluation labels? (context: [5])
    - What do you offer them in return?
39. [Likely] A model provider deprecates the model version your extraction pipeline depends on, at short notice. How do you lead the response across your team, the provider and affected customers?
    - What would you change in the architecture so the next deprecation is routine?
40. [Likely] A product team ships an agent feature that bypasses the shared guardrail and audit layer to meet a launch date. How do you respond, and how do you stop it happening again?
41. [Likely] How would you use a customer advisory group to shape the design of a governance feature (model updates, data retention, audit) without letting the loudest customer set the architecture?

D. Research-to-product for ML systems

42. [Likely] Walk me through a research result you took into production. What changed in the system design between the paper and the product?
    - Which offline metric gains disappeared in production, and why?
43. [Likely] How would you take a new layout-aware or vision-language model for documents from a research prototype to a generally available Document Understanding feature?
    - What gates would you set between prototype, preview and GA?
    - How would you handle customers whose documents look nothing like the training data?
44. [Likely] How do you decide which research directions are worth a production investment and which should stay exploratory?
45. [Likely] How do you design the research environment so that experiments can be promoted to production without being rewritten?
    - What shared components (data loaders, eval harness, model registry) would you standardise?
46. [Likely] How would you run a time-boxed research spike on computer-use agents, and how would you decide whether to productise it?
47. [Likely] Tell me about a research project you stopped. How did you decide, and what did you salvage into the product?
48. [Likely] How do you balance publishing research with protecting UiPath's product advantage?

49. [Likely] How would you move a Communications Mining-style model from per-customer training toward a foundation model, and what organisational changes would that require?
50. [Likely] How do you translate an academic benchmark gain into a business case a product leader will fund?
51. [Likely] How would you take a research technique for agent reliability (for example self-verification or plan-then-execute) to production when it doubles token cost and latency?
    - What evidence of reliability gain would justify the cost, and for which customer processes?
52. [Likely] How would you set up a loop where production failures from agents and extraction models directly set the research agenda?
    - Who triages failures, and how do they reach a researcher within a week?
53. [Likely] How would you lead the move to synthetic documents for training and evaluating extraction models, and how would you get legal and customers comfortable with it?
54. [Likely] How would you decide between per-tenant fine-tuning and a shared model with in-context examples for customer-specific extraction, and what does each mean for tenant isolation?

E. Roadmap and execution for ML platforms

55. [Likely] Build a four-quarter roadmap for an enterprise agent evaluation platform. What ships in Q1?
    - Which dependencies on other teams are on the critical path?
    - What would you cut if your team were halved?
56. [Likely] Build a roadmap for a document tagging and labeling system that reduces human validation effort by half.
    - How would you sequence active learning, confidence calibration and better base models?
    - Which milestone would tell you the plan is not working?
57. [Likely] How do you plan ML work when model quality gains are uncertain and hard to estimate?
    - How do you communicate uncertainty in a roadmap to executives?
58. [Likely] How do you balance roadmap items for new capabilities against reliability, technical debt and evaluation work?
59. [Likely] A large customer asks for a custom model feature that conflicts with your platform roadmap. How do you decide?
60. [Likely] How would you staff a new ML system: which roles, in which order, and which you would hire versus borrow?
61. [Likely] How do you define success and kill criteria for each milestone of an ML platform roadmap?
62. [Likely] Tell me about a roadmap you owned that slipped badly. What did you change in the system or the plan?
63. [Likely] How do you plan the migration of customers from an older extraction model generation to a new one?
    - How do you prove to customers that the new model is not worse on their documents?

64. [Likely] Build a roadmap that moves customers from robot-only workflows to processes that combine agents, robots and people. What has to be true in the platform before each step?
65. [Likely] A new frontier model release makes two quarters of your team's specialist-model work look obsolete. How do you re-plan, and how do you explain it to your team and to leadership?
    - What part of that work would you keep, and why?
66. [Likely] How would you set and track a reliability target (for example agent task success or straight-through processing rate) as a roadmap commitment rather than a slogan?

F. Cross-org alignment on platform architecture

67. [Likely] How would you align Document Understanding, agents and Communications Mining teams on a shared model-serving and routing layer?
    - What would the interface contract look like, and who owns it?
68. [Likely] How would you get agreement across product, research and the AI Trust Layer team on what every LLM call must log for audit?
69. [Likely] How do you align cloud and on-prem product lines on one ML architecture when their constraints differ?
70. [Likely] Describe how you would set up a cross-org architecture forum for ML systems. Who attends, and what decisions does it own?
    - How do you stop it becoming a bottleneck?
71. [Likely] How do you handle a situation where two VPs want incompatible ML platform directions?
72. [Likely] How would you align on a single definition of "agent task success" across teams that each measure it differently?
73. [Likely] How do you manage shared labeling and human-validation infrastructure when several products compete for annotator capacity?
74. [Likely] How would you align products and partners on one field schema and label taxonomy for document extraction?
    - Who owns changes to the schema, and how are breaking changes handled?
75. [Likely] How would you get product and research to agree on one offline benchmark suite that gates every model release, when the most useful golden data is customer data under privacy limits?
76. [Likely] A model-provider outage takes down agents, Document Understanding and Communications Mining at once. Who should own failover policy across products, and how would you get it agreed before the next outage?

G. Mentoring and raising design judgement

77. [Likely] How do you teach a strong senior scientist to think about system design, not only model design?
    - What exercises or review habits do you use?
78. [Likely] Tell me about someone you grew into a technical lead for an ML system. What did you delegate, and when?
79. [Likely] How do you give feedback on a design document that is technically correct but misses the product problem?

80. [Likely] How do you build an evaluation-first culture in a team that is used to shipping on demo quality?
81. [Likely] How do you scale your design influence beyond the meetings you personally attend?
    - What templates, checklists or reference architectures have you created?
82. [Likely] How do you mentor engineers who are new to ML on the failure modes of probabilistic systems in production?
83. [Likely] What separates senior from principal design work in ML systems, and how have you coached someone across that gap?
84. [Likely] How would you coach a strong researcher who resists production limits on latency, cost and on-prem deployment for a document model?
85. [Likely] How would you set up design-review shadowing so junior scientists learn to review agent systems, not only build them?

H. Judgement, ownership and values in system decisions

86. [Likely] Tell me about a time you shipped an ML system faster than the team thought possible. What did you deliberately leave out of the design?
    - What risks did that create, and how did you contain them?
87. [Likely] Describe a time you took ownership of a production incident in an ML system you did not build.
    - What design changes did you drive afterwards?
88. [Likely] When would you refuse to deploy an LLM-based automation for a customer process, even if the customer wants it?
    - How do you design the human-in-the-loop step so the customer stays accountable?
89. [Likely] How do you handle a customer escalation where your model's extraction error caused a downstream financial mistake?
    - What changes in confidence thresholds, validation and audit would you lead?
90. [Likely] How did you stay close to real customer usage ("immersed") when leading a platform used by thousands of customers?
91. [Likely] How do you make a high-stakes design call when the data is incomplete and the deadline is fixed?
92. [Likely] How do you weigh model cost against quality when the customer, finance and product each want different trade-offs?
93. [Likely] What would you want your first design review at UiPath to accomplish, and how would you prepare for it?
94. [Likely] An agent takes a wrong, irreversible action in a customer's ERP. Walk through how you lead the response and which design changes you drive afterwards.
    - How do you decide, as a company-wide policy, which agent actions need human approval?
95. [Likely] When would you tell leadership that a much-hyped capability, such as fully autonomous computer-use agents, is not ready for enterprise customers, and what evidence would you bring?
96. [Likely] How do you lead on fairness and quality gaps when your document or communications models are much weaker in some languages or regions?

I. Leadership probes attached to a live design prompt

These are likely to come mid-way through a design question, after you have drawn the architecture.

97. [Likely] You have just designed this document extraction system. You have six people. Who works on which box first, and why?
98. [Likely] Which box in your diagram is the riskiest, and how would you de-risk it in the first month?
99. [Likely] Which parts of this design would you hand to a partner team, and how would you specify the interface?
100. [Likely] If the product manager says the launch date moves up by two months, what do you cut from this design? - What do you refuse to cut?
101. [Likely] How would you present this architecture to the CTO in five minutes?
102. [Likely] What would make you scrap this design six months from now?
103. [Likely] How would you measure whether this system is succeeding one year after launch, and who would you report it to?
104. [Likely] Which decisions in this design would you want reviewed by security and legal, and when in the timeline?
105. [Likely] Which component in your diagram would you build now as a shared platform piece, and which would you leave product-specific until a second team needs it?
106. [Likely] If your LLM inference budget were cut by half, what changes in this diagram?
107. [Likely] Which interface in your diagram needs an RFC first, and who must sign it?
108. [Likely] How would you split this design between a small research team and a platform engineering team, and where are the hand-offs?

References

Sources 1-3 are official UiPath pages. They exist but could not be reached from this environment (blocked by DNS or the proxy), so check them yourself. Sources 4-11 were fetched and read in this review. None of them reports a UiPath interview question; they back the product facts behind the *(context: [n])* markers and the format notes.

1. UiPath Careers, uipath.com (official careers page; blocked in this run, not read). https://www.uipath.com/company/careers
2. UiPath Blog, uipath.com (official product and AI announcements; blocked in this run, not read). https://www.uipath.com/blog
3. UiPath Documentation portal, docs.uipath.com (Document Understanding, Communications Mining, Agents and Maestro documentation; blocked in this run, not read). https://docs.uipath.com
4. UiPath, "uipath-python" README: Python SDK and CLI for UiPath Cloud Platform, with services for processes, assets, buckets, connections, context grounding (semantic search over knowledge indexes), jobs, queues and Action Center tasks (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-python/main/README.md
5. UiPath, "uipath-langchain-python" README: build and deploy LangGraph agents on UiPath, with human-in-the-loop through Action Center (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-langchain-python/main/README.md
6. UiPath, "uipath-integrations-python" README: agent runtimes for Google ADK, LangChain, LlamaIndex, Microsoft Agent Framework, OpenAI Agents and PydanticAI,

all implementing the UiPath Runtime Protocol (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-integrations-python/main/README.md
7. UiPath, "uipath-mcp-python" README: hosting MCP servers on the UiPath platform (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-mcp-python/main/README.md
8. UiPath, "uipath-runtime-python" README: the base contracts for agent runtimes, covering execution context, event streaming, tracing, structured errors, durable execution and human-in-the-loop (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-runtime-python/main/README.md
9. UiPath, "uipath-dev-python" README: developer console for running and debugging runtimes, with structured logs and OpenTelemetry traces (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-dev-python/main/README.md
10. alirezadir, "Machine-Learning-Interviews" (AIMLInterviews), Behavioral and Leadership chapter: common behavioral and leadership prompts and the STAR method; general guide, not specific to UiPath (fetched). https://raw.githubusercontent.com/alirezadir/Machine-Learning-Interviews/main/src/behavioral/behavior.md
11. alirezadir, "Machine-Learning-Interviews" (AIMLInterviews), ML System Design chapter: notes that in 2026 GenAI and agent design rounds weigh evaluation, cost, latency, guardrails and human fallback; general guide, not specific to UiPath (fetched). https://raw.githubusercontent.com/alirezadir/Machine-Learning-Interviews/main/src/MLSD/ml-system-design.md

07. UiPath AI Context: DocPath and Document Understanding, the LLM Gateway and AI Trust Layer, Agentic Automation Strategy and the Peak Acquisition

How to read this chapter. This chapter asks how well you understand UiPath's own AI stack and how you would extend it. You asked for ML system design material, so each section opens with a plain-text block diagram of the UiPath system it covers, followed by numbered questions. Indented sub-bullets are follow-up probes. The chapter has questions only, with no answers. The diagrams are plain text so they look the same in Markdown and in a PDF export.

Tagging system. Every question carries one tag:

- **[Reported] [n]**: a candidate report, question bank or official page links this question to UiPath. The number points to the References list.
- **[Likely]**: an inference from the role (Principal Applied Scientist), the onsite loop format and UiPath's public product line. No candidate has reported it.
- *(context: [n])* at the end of a [Likely] question means the product fact in the question comes from a source that was read in the review pass. It does **not** mean a candidate reported the question.

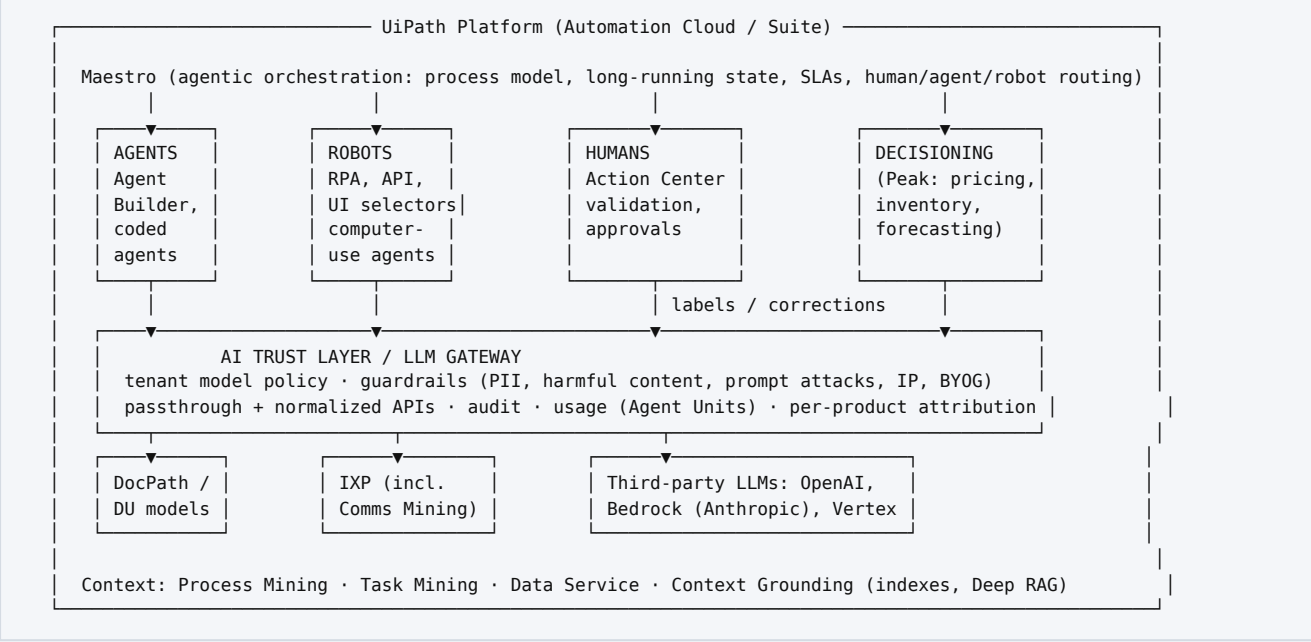
Research limits (please read). In the review pass (October 2026) the shared web-search budget was already used up, and direct fetches of uipath.com, docs.uipath.com, Glassdoor, arXiv, Hugging Face and the OSWorld leaderboard site were blocked (DNS failure or proxy denial). Only raw GitHub files could be read. The review read 15 of them, mostly UiPath's own open-source SDK code and documentation (References 1-15). Those files confirm product facts: the LLM Gateway, AI Trust Layer model policy, guardrails, Document Understanding project types, Context Grounding, agent evaluation and human-in-the-loop. None of them is a candidate interview report. So **no question below is tagged [Reported]**, and this chapter does not invent reports. Every question is **[Likely]**.

Facts still unverified. The names DocPath and CommPath, the ScreenAgent name and its benchmark standing, and the details of the Peak acquisition (announced in 2025, by general knowledge) could not be checked against any page in this pass. Check them on uipath.com before the interview. One verified update: UiPath's Communications Mining CLI (the former Re:infer `re` tool) now describes itself as the CLI for **UiPath IXP**, and the SDK runs both document and IXP extraction through one Documents service (context: [5][12]).

Format notes for this round

Item	Expected shape (inferred, not confirmed)
Where it appears	It is rarely a separate round. It runs through the hiring-manager conversation, the system design round and the "why UiPath" part of the leadership or bar-raiser round.
Duration	10-25 minutes of a 45-60 minute round
Interviewer	Hiring manager (an AI or science director), a principal scientist on DU, IXP, Agents or the platform, or a product leader
What is assessed at principal level	Do you know what UiPath ships and why? Can you name the science gaps in it? Can you reason about the moat (specialised models, enterprise data, governance, the installed robot base) against frontier-model vendors? Can you propose a multi-quarter research agenda that fits the business?
Failure modes	Generic "I'd fine-tune an LLM" answers; not knowing the difference between RPA robots and agents; ignoring on-prem, air-gapped and regulated customers; treating governance as an afterthought

Reference diagram: where the AI pieces sit in UiPath's platform



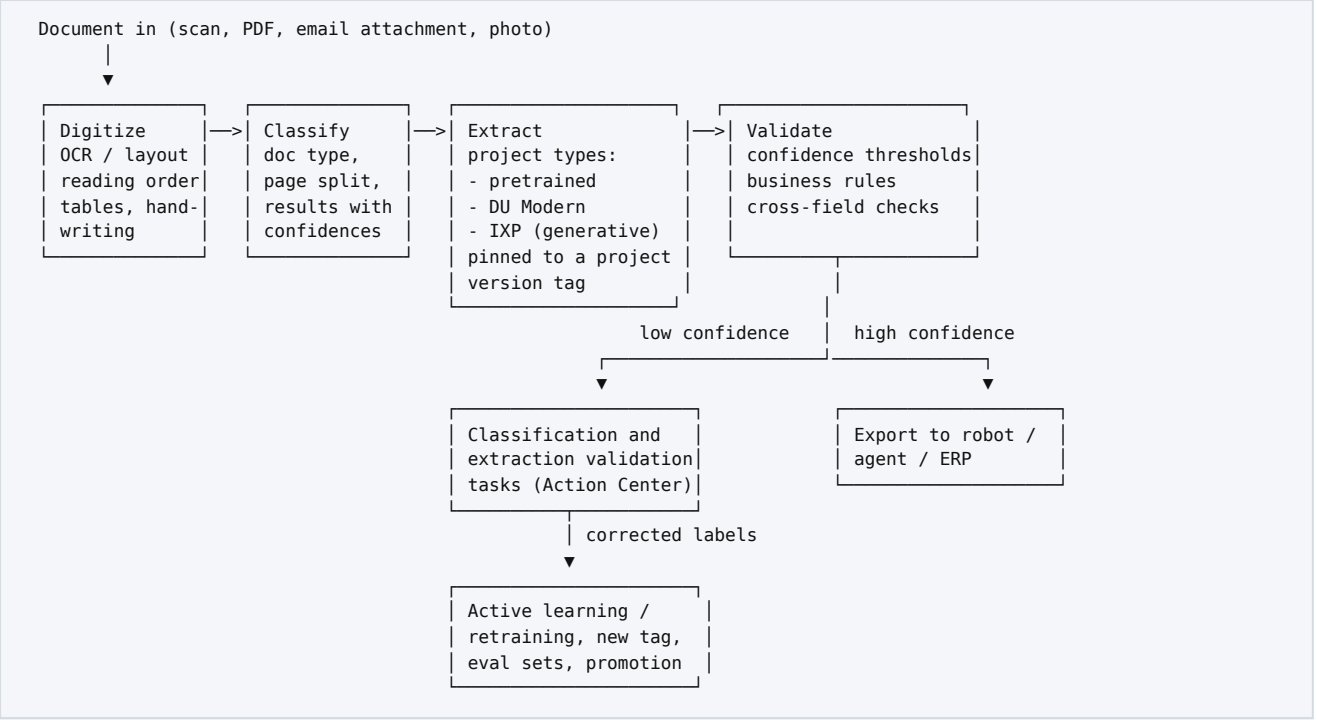
1. Company AI strategy and "why UiPath"

1. [Likely] UiPath has moved from RPA to "agentic automation". In your own words, what does that shift mean technically, and what new science problems does it create?
    - What can an agent do that a robot cannot, and where is a robot still the better choice?
    - How would you explain the shift to a customer who has 500 production robots?
2. [Likely] What is UiPath's defensible advantage against frontier-model labs and hyperscalers that also ship agents?
    - Is the moat the models, the data, the orchestration layer, the installed base or governance? Rank them and defend the order.
    - What would make that moat disappear within two years?

3. [Likely] What do you think is the single biggest open science problem in UiPath's product line today?
    - How would you staff and sequence a team to work on it?
4. [Likely] UiPath sells to regulated enterprises (banking, insurance, healthcare, public sector). How does that change your research priorities compared with a consumer AI company?
    - Which methods would you rule out because of it?
5. [Likely] Where should UiPath build its own models, and where should it rely on third-party LLMs?
    - What evidence would change your mind for a given capability?
    - What is your view on small specialised models against large general models for enterprise automation over the next three years?
6. [Likely] How would you decide whether a research result is ready to go into a product that runs on customers' production processes?
7. [Likely] What should a Principal Applied Scientist own at UiPath that a senior applied scientist does not?
    - Give an example of a cross-team technical decision you would expect to drive in your first year.
8. [Likely] Model calls made through UiPath's agent SDKs consume the customer's "Agent Units" rather than the customer's own provider tokens. How does that pricing model change what the science team should optimise? (context: [3])
    - Who carries the cost of a model that uses three times more tokens for a two-point quality gain: UiPath or the customer?
    - How would you price a call to an in-house specialised model against a call to a frontier model?
9. [Likely] UiPath now runs agents written in LangGraph, LlamaIndex, OpenAI Agents and other frameworks on one runtime, and also hosts customers' MCP servers. Is "be the neutral governed platform for everyone's agents" a stronger strategy than "build the best agents"? What does each strategy demand from applied science? (context: [13][14])

2. DocPath and Document Understanding (DU)

System diagram: the Document Understanding pipeline



10. [Likely] DocPath is positioned as a specialised generative model for documents. Why would a company build a document-specific LLM instead of prompting a general frontier model?

    - On which document types would you expect a general model to win, and on which DocPath?
    - What benchmark would convince a sceptical product leader?

11. [Likely] Design the training data strategy for a document foundation model when most customer documents are confidential and cannot leave the tenant.

    - What can you do with synthetic documents, and where does synthetic data fail?
    - How would you use human validation corrections without breaking data-use commitments?

12. [Likely] How do you produce calibrated per-field confidence scores from a generative extractor so that the validation-threshold logic in the diagram still works?

    - How would you measure calibration per field and per document type?
    - What happens to straight-through-processing rate if the model is overconfident by 5 points?

13. [Likely] Generative extraction can hallucinate a plausible invoice number that is not in the document. How do you detect and prevent that?

    - How would you ground each extracted value to a bounding box or token span?
    - What is your metric for "unsupported" values?

14. [Likely] Compare three extraction approaches for a new document type with 20 labelled samples: a fine-tuned layout model, a prompted generative model and a hybrid. How do you choose?

- How does the answer change at 2,000 samples?

15. [Likely] UiPath's SDK exposes three extraction project types side by side: pretrained models, DU Modern projects and IXP projects. If you owned the roadmap, would you converge them into one model family or keep them separate? (context: [5])

- What does each type buy the customer in cost, latency, controllability and labelling effort?
- How would you migrate a customer's trained DU Modern project to IXP without a drop in accuracy?

16. [Likely] How would you handle multi-page bundles (a loan file with 40 mixed documents) for classification and page splitting?

- Where do long-context models help here, and where do they hurt?

17. [Likely] The SDK's default extraction path takes several classification results and keeps the one with the highest confidence. When is "take the argmax class" the wrong policy for a document pipeline? (context: [5])

- How would you handle near-ties, out-of-taxonomy documents and bundles that contain several types?
- Should a low classification margin open a classification-validation task even when extraction confidence is high?

18. [Likely] Tables, checkboxes, signatures, stamps and handwriting cause most DU errors. How would you prioritise model work across them?

- What error analysis would you run first?

19. [Likely] Design the evaluation framework for DU across hundreds of customer document types.

- Field-level F1, document-level accuracy or straight-through rate: which is the north-star metric and why?
- How do you build a held-out set that reflects the customer distribution without seeing customer data?

20. [Likely] A large customer reports that extraction accuracy fell after a model upgrade, while your global benchmark improved. How do you investigate, and how do you prevent this?
21. [Likely] DU and IXP extraction calls name a published project version by tag (or version number). Design the release process for a customer's project: how a retrained model earns a new tag, how it is shadow-tested against the current tag, and when traffic moves. (context: [5])

- What offline and online evidence do you require before a customer's production tag moves?
- How do you roll back if straight-through rate drops after promotion?

22. [Likely] Human review in the SDK is split into classification validation and extraction validation tasks in Action Center. Treat human validation capacity as a fixed budget:

how would you allocate it across documents and fields to maximise end-to-end accuracy? (context: [5][10])

- How does the policy change when the validation queue backs up and SLAs are at risk?

23. [Likely] IXP extraction in the SDK is asynchronous: start the job, then poll for the result. How would you schedule GPU inference for a mix of one-page receipts and 300-page contracts under per-tenant SLAs? (context: [5])

- Where would you batch, where would you split documents, and how would you decide?

24. [Likely] How should OCR and the generative extractor interact? Would you move to an OCR-free, pixel-input model?

- What does that do to latency, cost and multilingual coverage?

25. [Likely] Design an active-learning loop that uses human validation in Action Center to improve a tenant's extractor without catastrophic forgetting or label noise.

- How do you handle validators who disagree or who make systematic mistakes?

26. [Likely] How would you serve DocPath for on-premises and air-gapped customers with limited GPUs?

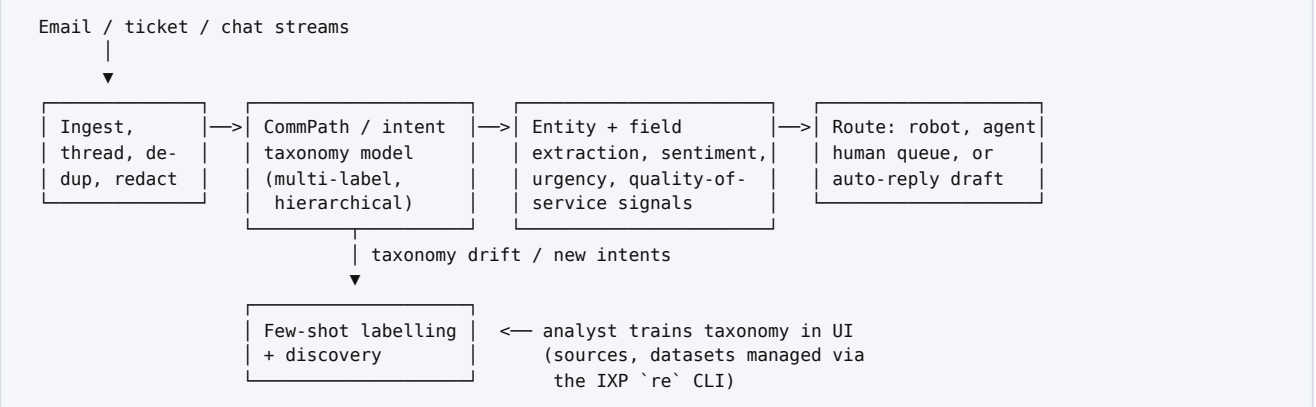
- Which compression methods would you try (distillation, quantisation, smaller specialised heads), and how do you show that quality held?

27. [Likely] How do you extend document models to new languages and scripts (Arabic, Japanese, Cyrillic) with little labelled data?
28. [Likely] What role should DU play inside an agent? Should extraction be a tool the agent calls, or should the agent read documents directly?

- How do confidence and validation signals pass through to the agent's decisions?

3. CommPath, Communications Mining and IXP

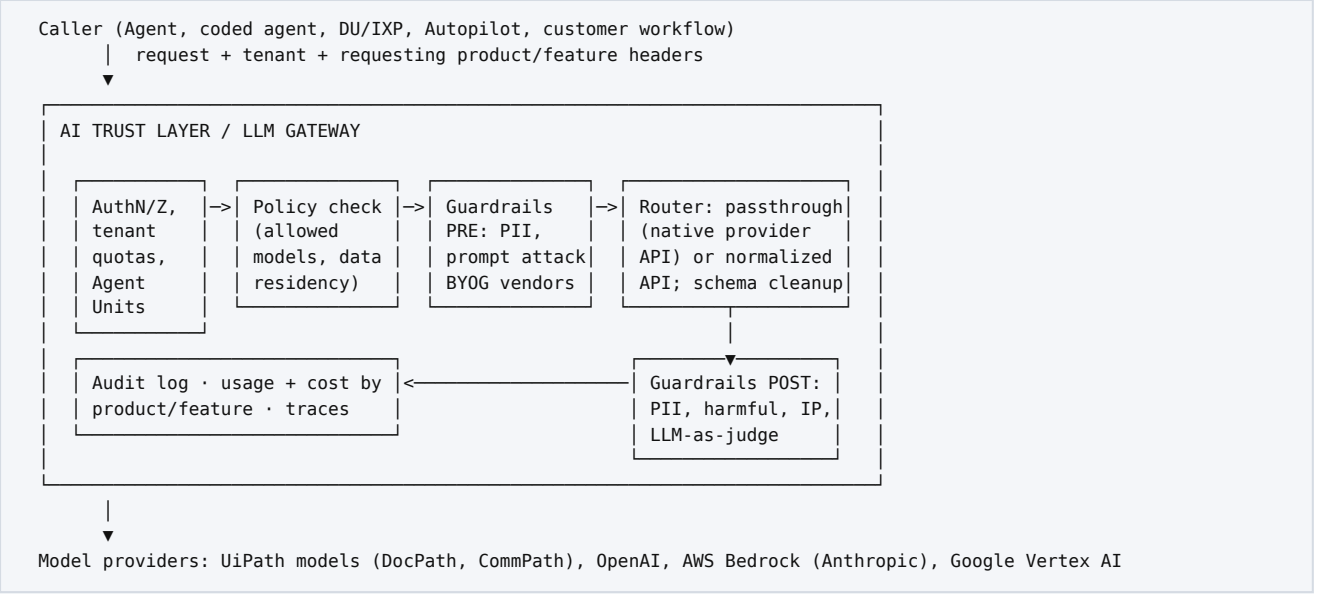
System diagram: communications understanding



29. [Likely] Communications Mining lets business users build their own intent taxonomy with few labels. How would you design the model so a non-expert gets a usable classifier after about 50 labels?
    - How do you choose which messages to show the user next?
30. [Likely] How would you discover new or emerging intents in a stream of customer emails before anyone has labelled them?
    - How do you avoid flooding analysts with junk clusters?
31. [Likely] Why build a communications-specific model (CommPath) rather than prompt a general LLM per message?
    - Answer in terms of cost per million messages, latency and stability of the taxonomy.
32. [Likely] UiPath's Communications Mining tooling now presents itself as part of UiPath IXP, alongside document extraction. What is the science case for one extraction platform over emails and documents, and what breaks when you merge them? (context: [5][12])
    - Can one label schema and one model family cover an email thread with three PDF attachments?
    - How would you migrate existing Communications Mining datasets and taxonomies without retraining every customer's model from scratch?
33. [Likely] How do you evaluate a hierarchical multi-label intent model when the taxonomy changes every month?
34. [Likely] Design the handoff from a classified email to an agent that drafts a reply and a robot that updates the CRM. Where are the human checkpoints?
    - What failure would worry you most in that chain?

4. LLM Gateway and AI Trust Layer

System diagram: governed LLM access



35. [Likely] What should an enterprise LLM gateway do that a direct API call to a model provider does not?

- Which of those functions need applied-science work rather than plain engineering?

36. [Likely] UiPath's gateway offers two interfaces: passthrough classes that expose each provider's native API, and a normalized interface that is uniform across providers but loses provider-specific features. The SDK docs recommend passthrough. Which would you make the default for UiPath's own agents, and why? (context: [2][3])

- What does each choice do to portability, to evaluation and to the cost of switching providers later?

37. [Likely] The models a tenant can call depend on that tenant's AI Trust Layer policy. An agent built and evaluated on one model is deployed to a tenant whose policy blocks that model. Design what should happen. (context: [3])

- How do you guarantee quality on a substitute model you never tested the agent with?
- Should the platform refuse to deploy, fall back automatically, or re-run the agent's evaluation set first?

38. [Likely] Design PII and PHI detection and masking for prompts going to third-party LLMs.

- How do you mask without destroying the information the LLM needs (names in a contract, account numbers in an invoice dispute)?
- What is your recall target, and how do you measure it across languages?
- How do you re-identify the masked values safely in the response?

39. [Likely] UiPath's PII guardrail takes a per-entity confidence threshold (for example email at 0.5). How would you pick thresholds per entity type, language and customer, and what evidence would you show a compliance officer for each choice? (context: [4])

- Is one threshold per entity calibrated across languages, or do you need per-language calibration?

40. [Likely] Guardrails can run at three scopes (tool, LLM call, whole agent) and at PRE, POST or both stages. Where would you place each validator for a claims-processing agent, and what latency and cost budget would you set for the guardrail stack? (context: [4][16])

- Which checks are worth a model call on every step, and which only at agent input and output?

41. [Likely] The user-prompt-attack validator runs only before a call, not after. Most real prompt injection reaches an agent indirectly, through documents, emails, web pages or tool outputs. How would you design detection for that indirect path? (context: [4])

- Where does that check belong: in the gateway, in the agent runtime, or in both?
- How do you measure false positives that break legitimate automations?

42. [Likely] UiPath offers an LLM-as-judge guardrail at runtime, alongside fixed classifiers. When is an LLM judge a sound runtime guardrail, and when is it a liability? (context: [4])

- What happens when the judge and the agent run on the same model family?
- How do you bound the extra latency and cost per run?

43. [Likely] "Bring Your Own Guardrail" lets an admin plug a third-party guardrail vendor into the Trust Layer. How would you qualify a customer's guardrail vendor, and what should happen when it disagrees with UiPath's built-in validator? (context: [4])
44. [Likely] UiPath's guardrails include an "intellectual property" validator for outputs. What could that mean technically (code licence detection, copyrighted text, trade secrets), and how would you build and evaluate it? (context: [4])
45. [Likely] A guardrail can log, block or modify (redact) the data it inspects. For a multi-step agent, when should a violation block the run, and when should it redact and continue? (context: [4])

- How does a silent redaction change the agent's later reasoning, and how would you detect harm from it?

46. [Likely] Design a model router that picks between a small specialised model and a frontier model per request.

- What features does the router use, and what does it optimise (cost, latency, quality, policy)?
- How do you train and evaluate the router without ground-truth labels for every request?

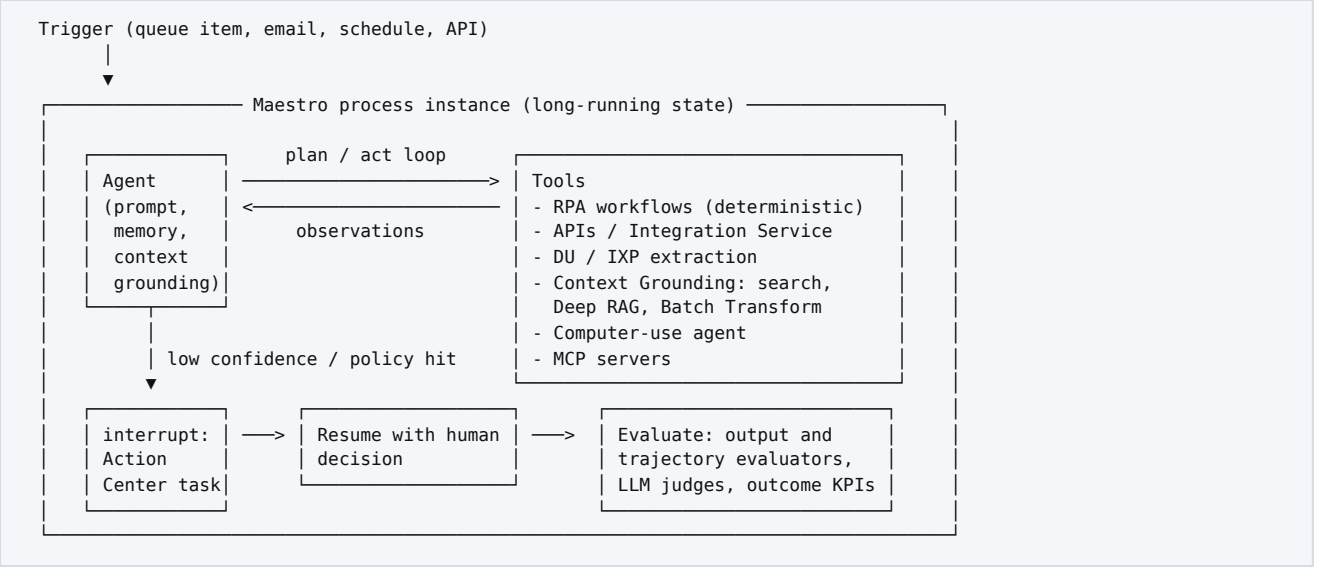
47. [Likely] Customers can bring their own model or API keys. How do you keep agent and DU quality guarantees when the underlying model is not yours?

- What compatibility test suite would you require before a customer model is allowed?

48. [Likely] UiPath's gateway rewrites JSON schemas for structured output (it strips titles and forces `additionalProperties: false`), and it handles provider quirks such as reasoning models that reject a temperature setting. How would you guarantee schema-valid structured output across OpenAI, Anthropic and Gemini models that differ in strict-mode support? (context: [2])
    - How would you maintain and test a capability matrix of models against features (tools, schemas, reasoning effort, vision)?
49. [Likely] Every gateway call carries "requesting product" and "requesting feature" headers. Design cost and quality attribution across UiPath products from that telemetry. (context: [2])
    - How would you use it to find which product feature gives the worst quality per dollar?
50. [Likely] The SDK's model list still contains old model versions next to new ones. Design a deprecation process for a model that thousands of customer agents are pinned to. (context: [2])
    - What regression evidence do you need before forcing a migration, and who signs off?
51. [Likely] Design the audit and observability layer so a compliance officer can reconstruct why an agent took an action six months later.
    - What do you log, and what must you not log?
52. [Likely] How would you design semantic caching in the gateway, and when is it unsafe?
53. [Likely] A new frontier model is released. Walk through how you would qualify it before turning it on for all tenants.
    - What regression suites, canary strategy and rollback triggers would you use?
54. [Likely] How would you enforce data residency (EU-only processing, for example) while still giving customers access to the best models?
55. [Likely] What guardrail models would you build in-house (toxicity, groundedness, policy adherence, structured-output validity), and which would you buy?
56. [Likely] How should cost and token budgets be enforced per tenant, per agent and per run, and how do you stop a looping agent from burning budget?

5. Agentic automation: Agent Builder, Maestro, Context Grounding and computer use

System diagram: an enterprise agent run



57. [Likely] UiPath frames its approach as agents, robots and humans working together. Design how Maestro should decide which of the three handles each step.

- Is that decision made at design time, at runtime, or both?
- How do you learn better routing from historical runs?

58. [Likely] Design the evaluation system for agents built by customers in Agent Builder, who have few test cases and no ML background.

- How would you generate evaluation cases automatically from process documentation or past runs?
- When is LLM-as-judge acceptable, and how do you calibrate it against humans?

59. [Likely] UiPath's agent evaluation framework splits evaluators into output-based ones (exact match, contains, JSON similarity, LLM judge) and trajectory-based ones (tool-call order, count, arguments and outputs, LLM judge of the trajectory). When does a trajectory evaluator punish a correct agent, and how would you fix that? (context: [8])

- How would you score an agent that reaches the right answer by a different but valid tool order?

60. [Likely] The LLM-judge output evaluator returns a score from 0 to 1 with a written justification. How would you turn that score into a pass/fail release gate, and how do you stop the gate drifting when the judge model is upgraded in the gateway? (context: [9])
61. [Likely] How would you make a non-deterministic agent safe enough to run inside a process that auditors expect to be repeatable?

- What would "deterministic replay" mean for an LLM agent?

62. [Likely] UiPath agents suspend on `interrupt(...)` while a person handles an Action Center task, then resume when it is done. Design the checkpoint for a run that waits three days for a human. (context: [10])

- What state must be saved, and what must be re-read on resume?

- What should happen if the prompt, model or policy changed while the run was suspended?

63. [Likely] UiPath's computer-use work has been reported on public desktop-agent benchmarks such as OSWorld (check the current standing before the interview). What do such benchmarks miss about enterprise desktop automation?

- How would you build an internal benchmark for legacy apps (Citrix, SAP GUI, mainframe terminals)?

64. [Likely] OSWorld now runs a "Verified" leaderboard where the maintainers run the agent code themselves instead of accepting self-reported scores. How should UiPath report computer-use results so that enterprise buyers and researchers trust them? (context: [15])

- What would you publish beyond a single success rate (cost per task, steps, failure taxonomy, variance across runs)?

65. [Likely] Design a computer-use agent that uses existing RPA selectors and recorded workflows as priors instead of acting from raw pixels.

- How do you recover when the UI changes and a selector breaks?
- How do you decide between a grounding model, the accessibility tree and OCR for a given screen?

66. [Likely] How would you turn the large base of existing RPA workflows into training data or tools for agents?

- What are the licensing and tenant-isolation constraints?

67. [Likely] Design "self-healing" automation: an agent that fixes broken robot workflows at runtime. How do you stop it from silently doing the wrong thing?
68. [Likely] How should Context Grounding (enterprise retrieval) be built so agents get the right policy document and not a stale or out-of-scope one?

- How do you evaluate retrieval quality per tenant without labelled queries?

69. [Likely] Context Grounding indexes live in Orchestrator folders, and a search can fall back to tenant-wide system indexes when the named index is not found. What permission-leak and wrong-context risks does that create, and how would you design retrieval to enforce document-level permissions? (context: [6][7])
70. [Likely] Context Grounding's Deep RAG runs a research-style synthesis across many documents in an index, with a choice of citation mode. How would you evaluate the faithfulness and citation accuracy of a multi-document synthesis? (context: [6])

- When should an agent be forced to cite, and what should happen when a claim has no supporting passage?

71. [Likely] Batch Transform applies one LLM prompt to every row of a CSV in an index, adds new output columns, and can optionally ground on web search. Design quality control for row-level LLM transformations over a million rows. (context: [6])

- How do you sample rows for review, and how do you detect systematic errors?
- What new risks does web-search grounding bring into an enterprise data pipeline?

72. [Likely] Unified search supports a semantic mode and a tabular mode, and can return partial results while ingestion is still running. How would you route a natural-language query between unstructured and tabular retrieval, and when is it unsafe for an agent to act on partial results? (context: [6])
73. [Likely] Context Grounding can build an ephemeral index for one run's attachments. When is per-run indexing better than putting the whole document set into a long-context model, and how would you decide by cost and quality? (context: [6])
74. [Likely] Autopilot helps developers and business users build automations from natural language. How would you measure whether Autopilot-generated workflows are correct?

- Would you use execution-based evaluation, and how would you sandbox it?

75. [Likely] How would you use Process Mining and Task Mining data to find which processes are worth turning into agents?

- How do you estimate the ROI of agentifying a process before building it?

76. [Likely] What are the main failure modes of multi-step enterprise agents you have seen or expect, and how would you instrument for each?

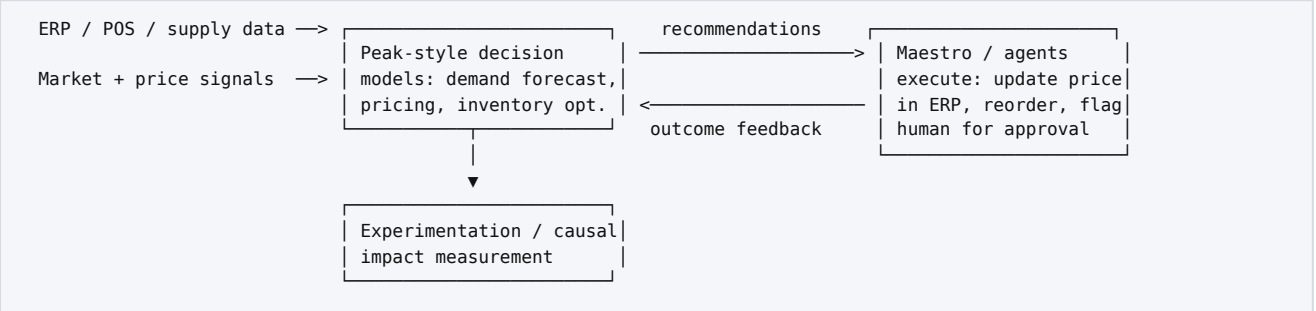
- Which ones can a better model fix, and which need system design?

77. [Likely] How would you design agent memory across a long-running case that lasts weeks (a claims process, for example)?
78. [Likely] Should UiPath train its own agent or action models, or orchestrate third-party ones? Lay out a decision framework.
79. [Likely] How would you support open agent protocols (such as MCP and agent-to-agent interop) while keeping governance through the Trust Layer? (context: [13])

- What new attack surface does a customer-hosted MCP server open?

6. Peak acquisition and decision intelligence

System diagram: decision intelligence joined to automation



80. [Likely] UiPath announced the acquisition of Peak, a decision-intelligence company focused on areas such as pricing and inventory, in 2025. Why would an automation

company buy a decision-intelligence company? (The deal details could not be verified in this pass; check them before the interview.)

- What does "closing the loop from decision to action" mean in system terms?

81. [Likely] Design a system where a pricing model recommends a price change and an agent carries it out in the ERP. Where do you put humans, guardrails and rollback?

- How do you stop a feedback loop where the model's own actions corrupt its training data?

82. [Likely] How would you measure the business impact of automated pricing or replenishment decisions when you cannot run a clean A/B test across stores?

- Which causal-inference designs would you consider?

83. [Likely] How would you combine classical forecasting and optimisation models with LLM agents? Which part should never be done by an LLM?
84. [Likely] What are the main technical integration risks when merging an acquired ML stack (different data platform, model serving and MLOps) into UiPath's platform?

- What would you unify in the first six months, and what would you leave alone?

85. [Likely] How would you expose Peak-style decision models as tools inside Agent Builder so a business user can use them safely?

7. UiPath research and publications

86. [Likely] Have you read any UiPath research papers, technical blog posts or open-source releases? Pick one and critique its method and evaluation. (UiPath's open-source agent SDKs, guardrails and evaluation code on GitHub are a fair target; find UiPath-affiliated papers on arXiv before the interview.) (context: [1][8])

- What follow-up experiment would you run?

87. [Likely] UiPath is a product company, not a lab. How do you balance publishing with shipping, and when is publishing worth it for the business?

- How do you protect customer data and competitive advantage in a paper?

88. [Likely] Which public benchmarks matter for UiPath's products (document understanding, GUI and computer-use agents, tool use, enterprise RAG), and which are misleading?

- Propose a new benchmark UiPath could release, and explain why it would be credible.

89. [Likely] How would you set up a research agenda that shares components across DU, IXP, agents and computer use?

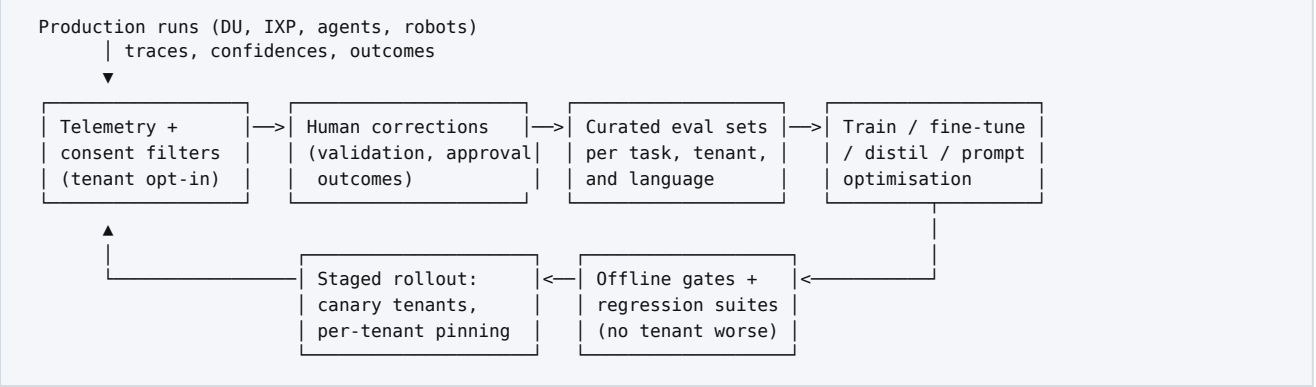
- Which shared foundation (a multimodal backbone, an eval harness or a data flywheel) gives the most leverage?

90. [Likely] How would you turn a strong arXiv result (a new GUI-grounding method, for example) into a production feature within two quarters?

- What would make you stop the effort early?

8. Cross-cutting ML system design probes on UiPath's stack

System diagram: the data and evaluation flywheel



91. [Likely] Design the data flywheel in the diagram for a multi-tenant SaaS product where many tenants do not consent to data use for training.

- What can you learn from non-consenting tenants (aggregate metrics, federated methods), and what is off limits?

92. [Likely] What release gate would you set for a model that serves thousands of tenants: average improvement, or "no tenant gets worse by more than X"?

- How do you get statistical power per tenant?

93. [Likely] How do you run the same model family across Automation Cloud, dedicated cloud, on-premises Automation Suite and air-gapped deployments?

- What breaks first when you cannot call home for telemetry?
- Air-gapped Automation Suite customers must host their own package feeds even for Python agent dependencies. How would you ship model weights, guardrail models and evaluation suites into such an environment, and keep them up to date? (context: [11])

94. [Likely] Pick one UiPath AI product and design its online monitoring: drift, quality, cost and safety. What alerts would page someone at 2 a.m.?
95. [Likely] How would you estimate and lower the cost per automated transaction across OCR, the LLM, retries and human review?

- Which levers have the largest effect?

96. [Likely] Design human-in-the-loop thresholds that adapt over time as the model improves, without letting error rates creep up unnoticed.

97. [Likely] How would you handle the EU AI Act and similar regulation for agents that make decisions in HR, credit or insurance processes run on UiPath?

- What does a "high-risk" classification change in your system design?

98. [Likely] Sketch, end to end, an "invoice exception agent". It reads an invoice with DocPath, matches it to a PO through a robot, resolves mismatches by emailing the supplier with CommPath, uses a Peak-style model to decide whether to accept a price variance, and escalates to a human. Where would you invest science effort first?

- Which component's error dominates end-to-end accuracy, and how would you find out?
- How does the whole chain behave when the LLM provider has an outage?

9. Questions you should be ready to ask the interviewer

99. [Likely] (To ask) How is applied science organised across DU, IXP, Agents and the platform, and where would this role sit?
100. [Likely] (To ask) Which model decisions are centralised (gateway, guardrails, shared backbones), and which are owned by product teams?
101. [Likely] (To ask) How much of the roadmap depends on in-house models (DocPath, CommPath, computer-use models) compared with third-party LLMs over the next 18 months?
102. [Likely] (To ask) How is the Peak team being integrated, and what joint science work is planned?
103. [Likely] (To ask) What does success in the first year look like for a Principal Applied Scientist here?

References

Sources 1-16 were fetched and read in the review pass (October 2026) as raw GitHub files. They back the product facts behind the *(context: [n])* markers. None of them is a candidate interview report. Sources 17-19 are official UiPath pages that could not be reached from this environment (DNS failure or proxy denial); they are listed as the places to check DocPath, CommPath, the AI Trust Layer admin guide and the Peak acquisition, and no question relies on them.

1. UiPath, "uipath-python" README: Python SDK and CLI with services for processes, assets, buckets, context grounding, jobs, queues and Action Center tasks (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-python/main/README.md
2. UiPath, LLM Gateway service source, `uipath-platform` package: OpenAI-compatible and normalized APIs, model constants, schema cleanup for structured output, requesting-product and requesting-feature headers, reasoning-model handling (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath-platform/src/uipath/platform/chat/_llm_gateway_service.py
3. UiPath, "Chat Models" documentation, `uipath-langchain-python`: OpenAI, Anthropic on Bedrock and Gemini on Vertex through the LLM Gateway; models subject to AI Trust Layer tenant policy; usage consumes Agent Units; passthrough versus normalized

classes (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-langchain-python/main/docs/chat_models.md

4. UiPath, "Guardrails" documentation, `uipath-python`: tool, LLM and agent scopes; PRE and POST stages; PII, harmful content, user prompt attacks, intellectual property, LLM-as-judge, custom and Bring Your Own Guardrail validators; log, block and custom actions (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath/docs/core/guardrails.md
5. UiPath, Documents service source, `uipath-platform` package: pretrained, DU Modern and IXP project types; published project tags and versions; classification with confidences; asynchronous IXP extraction; classification and extraction validation (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath-platform/src/uipath/platform/documents/_documents_service.py
6. UiPath, Context Grounding service source, `uipath-platform` package: folder-scoped and system indexes, ephemeral indexes, Deep RAG with citation modes, Batch Transform with optional web-search grounding, unified semantic and tabular search, search during ingestion (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath-platform/src/uipath/platform/context_grounding/_context_grounding_service.py
7. UiPath, "Context Grounding" documentation, `uipath-langchain-python`: indexes created in Orchestrator folders from storage buckets; ContextGroundingRetriever (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-langchain-python/main/docs/context_grounding.md
8. UiPath, "Agent Evaluations" documentation, `uipath-python`: output-based, trajectory-based and custom evaluators (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath/docs/eval/index.md
9. UiPath, "LLM Judge Output Evaluators" documentation, `uipath-python`: semantic-similarity and strict-JSON judges with a 0-1 score and justification (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath/docs/eval/llm_judge_output.md
10. UiPath, "Human In The Loop" documentation, `uipath-langchain-python`: agents suspend on `interrupt(...)` and resume after an Action Center task (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-langchain-python/main/docs/human_in_the_loop.md
11. UiPath, "Automation Suite" documentation, `uipath-python`: package feeds for air-gapped deployments (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath/docs/AutomationSuite.md
12. UiPath (Re:infer), `re` CLI README: "the official command line interface for UiPath IXP", linked to the Communications Mining developer guide; manages sources and datasets (official; fetched). https://raw.githubusercontent.com/reinfer/cli/master/README.md
13. UiPath, "uipath-mcp-python" README: hosting MCP servers on the UiPath platform (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-mcp-python/main/README.md
14. UiPath, "uipath-langchain-python" README: LangGraph agents on UiPath, human-in-the-loop through Action Center, built on the UiPath Runtime Protocol (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-langchain-python/main/README.md

15. XLANG Lab, "OSWorld" README: OSWorld-Verified release (2025-07-28) and a verified leaderboard run by the maintainers (fetched). https://raw.githubusercontent.com/xlang-ai/OSWorld/main/README.md
16. UiPath, "Guardrails" documentation, `uipath-langchain-python`: middleware and decorator patterns with before/after agent, model and tool-call hooks (official; fetched). https://raw.githubusercontent.com/UiPath/uipath-langchain-python/main/docs/guardrails.md
17. UiPath Documentation portal, docs.uipath.com (Document Understanding, IXP, Communications Mining, AI Trust Layer, Agents and Maestro; not reachable in this pass). https://docs.uipath.com
18. UiPath Blog, uipath.com (product, AI and acquisition announcements, including Peak; not reachable in this pass). https://www.uipath.com/blog
19. UiPath Careers, uipath.com (official careers page; not reachable in this pass). https://www.uipath.com/company/careers

08. End-to-End ML System Design: A PDF Tagging System (Structure, Accessibility and Document Understanding)

How to read this chapter. The rest of the pack touches PDF tagging in scattered pieces: chapter 04 section C (Document Understanding), chapter 05 sections A and B (ML coding on layout and tables), and the leadership diagram in chapter 06. This chapter puts the whole system in one place. It opens with one end-to-end diagram. Each section then has a smaller diagram for its stage, numbered questions and follow-up probes. It has questions only, with no answers. The diagrams are plain text so they render the same in Markdown and in a PDF export.

Tagging system. Every question carries one of two tags:

- **[Reported] [n]**: a candidate report links this question to UiPath. The number points to the References list.
- **[Likely]**: an inference from the role (Principal Applied Scientist), the level and UiPath's document products: Document Understanding (DU Modern), IXP, validation in Action Center, and published project versions and tags. No candidate has reported it. A **[Likely] [n]** tag means source [n] confirms the *premise* of the question, such as a product behaviour, dataset or metric definition. It does not mean a candidate was asked it.
- Indented sub-bullets are follow-up probes an interviewer may use to push a strong first answer toward the principal bar.

"Tag" has two meanings here. (1) A *PDF structure tag*: an element of the tagged-PDF structure tree, such as H1, P, L/LI, Table/TR/TH/TD, Figure with alt text, or Span. This is what PDF/UA accessibility needs. (2) A *semantic tag or label*: a document class, a key-value field or an entity, as in Document Understanding and IXP. UiPath also uses "tag" for a published project version [1]. A principal-level answer names which meaning it is using, and the chapter tests both.

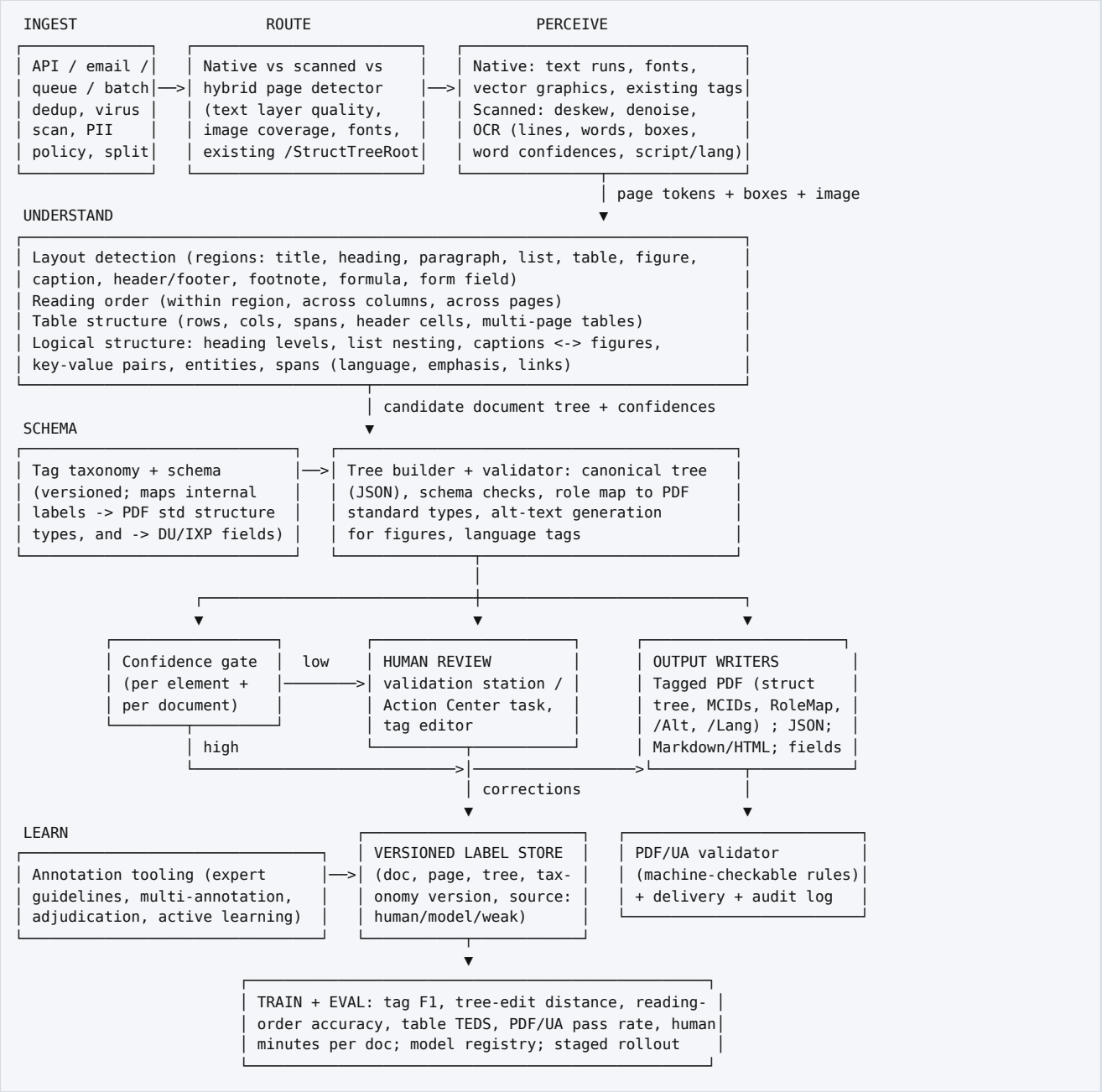
Research limits (please read). Two passes (October 2026) checked this chapter. In both, the workflow's shared web-search budget was already used up, so no search could run. The fetch proxy could not resolve glassdoor.com, teamblind.com, uipath.com, docs.uipath.com, arxiv.org, w3.org or pdfa.org. Only GitHub-hosted sources were reachable. The second pass re-fetched all 11 original sources and confirmed each claim the questions rely on. It also fetched 15 more GitHub sources [12]-[26] on benchmarks, OCR, table metrics, label quality and the PDF/UA rule set. These sources confirm product behaviour, datasets and metric definitions, not interview content. So **no question below is tagged [Reported]**, and the chapter does not invent reports. The UiPath documentation portal could not be reached, so

> it was removed from the references. Check product names and current PDF/UA details (ISO 14289-1 and -2, the Matterhorn Protocol, the PDF 2.0 structure namespace) against primary documents before the interview.

Format notes for this round

Item	Expected shape (inferred, not confirmed)
Round	ML / applied science system design. For a document-AI team it may be a deep dive on one document pipeline, or a design prompt followed by a probe on your own past document work
Duration	About 60 min: 5 min scoping, 30-35 min design, 15 min deep dives (evaluation, labelling, scale), 5 min your questions
Interviewer	Principal or staff scientist, or an architect from the document / IXP side; possibly a product manager for the human-review and customer angle
Likely prompt	"Design a system that turns any incoming PDF (native or scanned) into a correctly tagged, accessible PDF and structured data, for enterprise customers at scale"
Principal-level bar	You define the tag taxonomy and the success metrics before you pick models. You treat evaluation, labelling and human review as first-class systems, not afterthoughts. You separate the deterministic work (PDF parsing, schema validation, PDF/UA checks) from the probabilistic work (layout, reading order, semantics). You reason about cost per page, latency tiers, on-prem and regulated deployment, and versioning. You name the open research problems and a multi-quarter plan.
Common failure modes	Treating it as "just OCR plus an LLM"; no reading-order story; ignoring nested tables and multi-page objects; no plan for the confidence threshold and the review queue; metrics that do not match what the customer (screen-reader user, auditor or downstream robot) experiences

Reference diagram: the end-to-end PDF tagging system



A. Problem framing, users and success metrics

1. [Likely] Design a system that takes any enterprise PDF and produces (a) a tagged, accessible PDF and (b) structured data for downstream automation. Where do you start, and what do you ask before you draw anything?
    - Who are the users: screen-reader users, compliance officers, downstream robots or agents, or data teams? Do they need the same tree?
    - What does "done" mean for one document: PDF/UA machine-check pass, human-verified, or fields extracted?
    - What volume, page mix (native vs scanned), languages and latency tiers do you assume, and how would you check those assumptions?
2. [Likely] [1] UiPath already exposes classify, extract and validate as separate steps with published project versions and tags. How would a structure-tagging capability fit next to those steps rather than duplicate them?

    - Does structure tagging run before extraction, as shared infrastructure, or as a separate product?
    - What contract (schema) would extraction consume from the tagging stage?
3. [Likely] What is your north-star metric for this system, and what are the guardrail metrics?
    - How do you turn "accessible" into something you can measure offline?
    - How do you weigh human-correction minutes per document against model accuracy?
4. [Likely] Which parts of this problem are deterministic and which are probabilistic? How does that split shape the architecture?
    - What should never be left to a model (for example, writing valid PDF syntax or checking the schema)?
5. [Likely] An enterprise customer has 40 million legacy PDFs to remediate for accessibility in 12 months. How does that change your design compared with a real-time stream of new invoices?
    - What would you do in month one so that you learn the most?
6. [Likely] What would you build versus buy versus take from open source (for example, layout models, table models, OCR, PDF/UA validators)?
    - How do licences shape that choice, for example a model released under a non-commercial licence [7], a dataset limited to research use [11], or Apache-licensed code whose model weights carry a revenue-capped licence [16]?
7. [Likely] The same PDF can need PDF/A (archiving) and PDF/UA (accessibility) at once, and tools such as OCRmyPDF already output PDF/A by default [17]. Where do the two goals conflict in your design, and which do you give up when they cannot both hold?
    - Who in the customer organisation owns that decision, and how does the system record it?
8. [Likely] A signed contract or a certified form arrives for tagging. Adding a structure tree changes the file. How do you handle digital signatures, incremental updates and legal evidentiary value?
    - When do you refuse to tag, and when do you return a tagged copy next to the untouched original?

B. Ingest, routing and native vs scanned detection

```
PDF ──> sanitise (encryption, corrupt xref, JS, embedded files) ──> page split
     ──> per-page features: text-layer char count, glyph->Unicode mapping quality,
         image area ratio, invisible OCR text layer present?, existing StructTreeRoot?
     ──> router: NATIVE | SCANNED | HYBRID | ALREADY-TAGGED (verify / repair)
```

9. [Likely] How do you decide, per page, whether a PDF is native, scanned or hybrid? What features and labels would you use?
    - How do you detect a text layer that exists but is garbage (broken ToUnicode maps, ligatures, Type 3 fonts)?
    - What do you do with a scanned page that already has an invisible OCR layer from a previous tool?
10. [Likely] Why decide per page instead of per document? What breaks downstream if you route a whole document one way?

11. [Likely] Some incoming PDFs are already tagged, but badly. Do you trust, repair or discard the existing tags? How do you decide?
    - How would you measure the quality of existing tags without a human label?
12. [Likely] What security and privacy controls belong at ingest (encrypted PDFs, embedded JavaScript, attachments, PII), and how do they interact with the ML stages?
    - What changes for an on-prem or air-gapped customer?
13. [Likely] How do you handle very long documents (1,000+ pages) and very large pages (engineering drawings) in the pipeline?
    - Where do you parallelise, and which stages need the whole document?
14. [Likely] [1] UiPath's document APIs already run a separate digitization step (start, then poll for a result) before classification and extraction [1]. Should the structure tree become part of the digitization output? What compatibility duties does that create for every existing extraction project?
    - How do you ship a richer digitization result without changing extraction results that customers have already validated?

C. OCR and the perception layer

15. [Likely] For scanned pages, what does your OCR stage output, and what extra outputs does the tagging stage need beyond plain text?
    - Why do word-level boxes and confidences matter for reading order and review?
16. [Likely] How would you choose between a classical OCR engine, a document-specific OCR model and an end-to-end vision-language model that outputs structure directly?
    - What are the cost, latency, hallucination and auditability trade-offs?
    - How do you guard against a generative model "reading" text that is not on the page?
17. [Likely] How do you handle mixed scripts, right-to-left text, vertical text, handwriting and low-quality faxes?
    - How do these affect the /Lang attribute and reading order in the output?
18. [Likely] For native PDFs, the text is "free", but the order of drawing operators often does not match the reading order. How do you turn content-stream order into usable tokens?
    - How do you handle text drawn as vector paths, or text hidden under images?
19. [Likely] Where does pre-processing (deskew, dewarp, binarisation, rotation detection) belong, and how do you stop it from hurting native pages?
20. [Likely] [18] OCR-free models such as Donut read the page image straight into structured output, with no OCR engine [18]. For a *tagging* product, every structure element must point to real marked content on the page. What do you lose when the model gives you text without word boxes tied to glyphs, and how would you recover the link?
21. [Likely] [14] A VLM-based OCR project lists, among its fixes, hallucinations on blank documents and auto-rotation detection [14]. Design an adversarial test set for a generative OCR stage: blank pages, rotated pages, watermark-only pages, near-duplicate pages and pages in unseen scripts. Which failures are silent, and how would you catch them in production without labels?

22. [Likely] [23] Some open models target scientific documents and output LaTeX for formulas [23]. How would you decide which regions to send to a specialised formula model, and how would the result appear in the tagged PDF (Formula element, alternate text, or MathML in PDF 2.0)?
    - How do you evaluate formula output when two different LaTeX strings render the same?

D. Layout analysis and reading order

```
page image + tokens ──> layout detector (regions + class + score)
                    ──> region-level reading order (columns, sidebars, floats)
                    ──> token-level order inside regions
                    ──> cross-page links (continued paragraphs, tables, lists)
```

23. [Likely] [3] Define the layout classes you would detect. Public datasets such as DocLayNet use 11 classes over about 80,000 pages from six document categories [3]. Is that the right taxonomy for an enterprise tagging product?
    - What would you add or merge (form fields, signatures, stamps, key-value regions)?
    - How do you keep the layout classes separate from, but mappable to, the PDF structure tags?
24. [Likely] [3] DocLayNet annotated a fraction of pages two or three times to estimate annotation uncertainty [3]. How would you use multiply-annotated pages when you set targets for your own layout model?
    - If humans agree on only 80% of boxes for a class, what accuracy target is meaningful for the model?
25. [Likely] [7] Compare a vision-only detector, a multimodal model that combines text, layout and image (for example, LayoutLMv3 [7]) and a vision-language model for layout. Which would you use for which stage, and why?
    - How does the choice change if the customer runs on CPU-only on-prem hardware?
26. [Likely] [6] How would you model reading order? Compare heuristic sorting (XY-cut, column detection), a learned seq2seq model such as LayoutReader [6] and graph or pairwise-ranking approaches.
    - LayoutReader reports large gains over a heuristic baseline on ReadingBank [6]. Why might those gains shrink on your customers' documents?
    - ReadingBank's labels come from weak supervision on Word documents [6]. What biases does that bring?
27. [Likely] How do you define the "correct" reading order when two reasonable people would disagree (sidebars, pull quotes, footnotes, multi-column with figures)?
    - How does that ambiguity affect your metric?
28. [Likely] How do you handle objects that continue across pages: paragraphs split by a page break, tables with repeated headers, numbered lists?
    - Where in the pipeline do you merge them, and how do you represent the merge in the tagged PDF?
29. [Likely] Headers, footers, page numbers and watermarks should usually become artifacts, not content. How do you detect them robustly across a whole document?
    - What happens to a screen-reader user if you get this wrong in either direction?

30. [Likely] [19] [20] PubLayNet built its labels by matching PDFs to their XML sources [20]. DocBank built token-level labels for 12 kinds of semantic unit from LaTeX sources by weak supervision [19]. Does a tagger need token-level labels, region-level labels or both? What does each label type let you train, and what does it make hard?
31. [Likely] [13] OmniDocBench labels 28 block-level and 4 span-level element types over 1,651 pages from 10 document types [13]. How would you map an outside benchmark's taxonomy onto your own, and how do you report results for classes that do not map one to one?

E. Logical structure tagging: headings, lists, tables, figures, key-value pairs, spans

32. [Likely] How do you infer heading levels (H1-H6) from a page that only shows font sizes and positions? What signals do you use beyond font size?
    - How do you keep heading levels consistent across a 300-page document?
    - Is document-level inference (a global pass) worth the cost over page-level inference?
33. [Likely] How do you detect lists and their nesting, including lists whose bullets are images or whose numbers are drawn separately from the text?
    - How do you represent label (Lbl) and body (LBody) separately?
34. [Likely] [4] [8] Design the table pipeline: detection, structure recognition (rows, columns, spanning cells), header detection and cell content mapping. Which parts would you learn and which would you compute?
    - Public models such as TableFormer predict structure and cell boxes [8], trained on sets such as PubTabNet, FinTabNet and TableBank [4] [8] [11]. Why may these not transfer to scanned invoices or bank statements?
    - How do you decide which cells are headers, and how do you emit header-to-cell associations (scope or IDs) for accessibility?
    - How do you handle borderless tables, nested tables and tables split across pages?
35. [Likely] [12] The Table Transformer project releases pre-trained structure weights only for PubTables-1M and FinTabNet, and asks users to train their own multi-domain model [12]. Plan a multi-domain table model for invoices, bank statements, insurance forms and scientific reports. When is a domain-specific fine-tune worth the cost of serving one more model?
    - How do you decide that a table failure is a data gap and not a model-capacity limit?
36. [Likely] When is something a table for accessibility purposes but not for data extraction, or the other way round (for example, a layout table or a form laid out as a grid)?
37. [Likely] How do you link figures to their captions, and how do you decide whether a figure is meaningful or decorative?
38. [Likely] Design alt-text generation for figures, charts and diagrams. How do you evaluate it, and when does a human have to write or approve it?
    - What are the risks of a vision-language model writing confident but wrong alt text for a chart in a financial filing?
    - How long should alt text be, and how do you use surrounding context?

39. [Likely] How do you detect key-value pairs on forms and semi-structured documents, and how do they map both to structure tags and to DU or IXP fields?
    - Should a key-value pair be one structure element or two? Who decides?
40. [Likely] Which inline spans matter (language changes, emphasis, links, abbreviations with expansions, formulas), and how do you detect and tag them?
    - How would you produce accessible output for mathematical formulas?
41. [Likely] How do you tag footnotes, endnotes, citations and cross-references (Note, Reference, Link) so that a screen-reader user can follow them and come back? How do you generate a table of contents (TOC/TOCI) and bookmarks that match the heading tree?
42. [Likely] Some incoming PDFs already have interactive form fields. How do you tag the existing widgets (Form elements, tooltips, tab order) so they agree with the visual labels your model detects? What do you do when the field names and the printed labels disagree?
43. [Likely] [16] Some pipelines add an optional LLM pass that merges tables across pages and pulls values out of forms [16]. Where in your cascade would an LLM "repair" pass sit? How do you stop it from changing or inventing text that is on the page?
    - What hard constraint would you put on its output (for example, every output token must map to an input token)?
44. [Likely] Would you use one large model that outputs the whole tree, or a cascade of specialised models? Defend the choice on accuracy, debuggability, cost and how fast you can iterate.
    - How do errors compound through a cascade, and how would you measure that?
    - How do you ensure a generated tree is always well formed (constrained decoding, post-validation, repair)?

F. Tag taxonomy and schema

```
internal taxonomy v(N)            role map                   PDF standard structure types
 title, section_heading(level) ──────────────────────────────> H1..H6 (or H in PDF 2.0 usage)
 paragraph, list(item, label) ───────────────────────────────> P, L / LI / Lbl / LBody
 table(row, header_cell, cell) ──────────────────────────────> Table / TR / TH / TD
 figure(alt), caption ───────────────────────────────────────> Figure (/Alt), Caption
 kv_pair(key, value), entity ──> custom types ──RoleMap──> standard type (+ DU/IXP field)
 header/footer/page_no ──────────────────────────────────────> Artifact (not in tree)
```

45. [Likely] Design the tag taxonomy and its schema. What is the canonical internal representation, and why not use PDF structure types directly?
    - How do you represent confidence, provenance (model, version, human) and bounding boxes on each node?
    - Docling, for example, uses one internal document model and exports it to Markdown, HTML, DocTags and lossless JSON [2]. What do you gain and lose with a single canonical tree?
46. [Likely] How do you version the taxonomy, and what happens to old labels, old models and customer outputs when you split or merge a class?
    - How do you migrate a label store from taxonomy v3 to v4 without relabelling everything?

47. [Likely] Customers want their own custom tags, such as "ClauseIndemnity" or "InvoiceLineItem". How do you support them without breaking accessibility output?
    - How does role mapping to standard types keep a custom tag valid for assistive technology?
48. [Likely] How strict should the schema be? Which rules are hard constraints (a TD must sit inside a TR) and which are soft (a document should have one H1)?
    - Where do you enforce each: in decoding, in post-processing, or in validation?
49. [Likely] How would you make the same tree serve accessibility, extraction and RAG chunking for agents? Where do these uses conflict?
50. [Likely] [24] Docling defines its canonical document as a Pydantic model [24]. Your training code is in Python, but the PDF writer and the review UI may run in other stacks. How do you keep the schema in one place and evolve it without breaking any service?
    - What is your compatibility rule: can an old reader read a new tree, and can a new reader read an old tree?
51. [Likely] [26] The PDF 2.0 and PDF/UA-2 specifications still receive errata and amendments through a public tracker [26]. How do you version the "standards profile" (role map, writer rules, validator profile) separately from the tag taxonomy and the models?
    - When a standard changes, which customer outputs must you re-validate or regenerate?

G. Tagged-PDF and PDF/UA output

```
canonical tree ──> structure tree writer: StructTreeRoot, StructElems, parent tree,
                   marked-content IDs (MCIDs) linking tree <-> page content,
                   RoleMap for custom types, /Alt, /ActualText, /Lang, Artifacts
               ──> PDF/UA validator (machine-checkable rules) ──> report
               ──> residual checks needing human judgement (alt-text quality, order)
```

52. [Likely] Walk through how you write the tagged PDF: how the structure tree links to the page content, and how you mark headers and footers as artifacts.
    - What goes wrong if marked content is split, or if one content stream draws text for several elements?
    - Do you rewrite content streams or only add structure? What are the risks of each?
53. [Likely] [9] [10] Open-source validators such as veraPDF check PDF/UA-1, PDF/UA-2 and WTPDF rules, with WCAG 2.2 profiles in an experimental stage [9] [10]. What share of accessibility can a machine validator check, and what remains for human judgement?
    - How do you report "passes machine checks" versus "human-verified accessible" honestly to customers?
54. [Likely] How do you set /Lang, /Alt, /ActualText and expansion text, and which of them can you generate automatically?
55. [Likely] How do you handle PDF 2.0 and PDF/UA-2 versus PDF 1.7 and PDF/UA-1 outputs? Does the internal tree change?
56. [Likely] How do you test the output with real assistive technology (screen readers) and with accessibility experts, not only validators? How would you scale that testing?

    - How do you turn their findings into a repeatable metric that you can track release to release?
57. [Likely] Writing a tagged PDF must not change how the page looks. How do you guarantee visual fidelity and detect regressions in the writer?
    - What is your regression test corpus for the writer, and how is it different from the ML eval set?
58. [Likely] [22] veraPDF's PDF/UA-1 profile encodes about 106 machine rules, such as "the MarkInfo dictionary has Marked set to true" and "content marked as an artifact should not sit inside tagged content" [22]. How would you use the validator as a release gate for the writer and as a runtime check on every output?
    - Most legacy customer files fail some rules before you touch them. How do you report "our output fixed X of Y failures" honestly?
    - Which rule failures should block delivery, and which should only warn?
59. [Likely] A customer later edits a tagged PDF in a desktop editor (adds a page, redacts a paragraph). The structure tree goes stale. Do you support incremental re-tagging of only the changed pages? What must the canonical tree store to make that possible?

H. Confidence, calibration and human review

```
per-element scores ──> calibrated confidence ──> document-level risk score
   ├─ all elements >= threshold and validator passes ──> auto-publish
   ├─ some elements low ──> targeted review (only the risky regions)
   └─ document very low / unsupported type ──> full manual remediation queue
corrections ──> label store (with reviewer id, time spent, taxonomy version)
```

60. [Likely] [1] UiPath's document APIs include validation actions that block until a person completes them in Action Center [1]. Design the human-review step for structure tags: what does the reviewer see, and what do they fix?
    - How do you show a tree, reading order and table structure so that review is fast?
    - What is the unit of review: a page, an element, or a document?
61. [Likely] How do you produce calibrated confidence for a structured output like a tree or a table, not just for a single class?
    - How do you combine element confidences into a document-level decision?
    - How do you check calibration after a model update?
62. [Likely] How do you set the auto-publish threshold? Who owns the trade-off between review cost and risk, and how does it vary by customer or document type?
    - What happens when the threshold sends 60% of a large customer's documents to review on day one?
63. [Likely] How do you measure and reduce reviewer effort (minutes per page, edits per page), not just model error?
    - How would you run an A/B test on the review UI itself?
64. [Likely] How do you prevent reviewers from rubber-stamping model output (automation bias), and how would you detect it in the data?
    - Would you inject known errors to audit reviewers? What are the ethical and practical concerns?
65. [Likely] Review corrections are a training signal but they are biased: reviewers only see low-confidence items. How do you use them without biasing the next model?

66. [Likely] A customer asks for a statistical guarantee, such as "at most 1% of auto-published documents have a wrong table header". How would you set a threshold that gives that guarantee (for example, with conformal or risk-control methods)? Which assumptions break when the document mix shifts?
67. [Likely] Reviewers often fix only part of a reading order or part of a table and leave the rest. How do you record a partial correction in the label store so that training does not treat the unfixed parts as confirmed?

I. Annotation and labelling tooling

68. [Likely] Design the annotation tool for structure tags. How is it different from a bounding-box tool?
    - How do annotators label reading order, nesting and table spans efficiently?
    - How do you pre-fill with model output without anchoring annotators?
69. [Likely] Write the outline of the annotation guideline for headings, lists and tables. Which edge cases would you settle up front?
    - How do you measure inter-annotator agreement for trees and orders, not only boxes?
    - What process resolves disagreements (adjudication), and how do guideline changes flow back into old labels?
70. [Likely] [3] [6] [11] Public datasets use different labelling strategies: expert manual labelling [3] and weak supervision from Word or LaTeX source files [6] [11]. When would you use each for your product, and how would you combine them?
    - Could you build synthetic tagged PDFs from Word or HTML sources? What gap would remain to real scanned documents?
71. [Likely] Design an active-learning loop. Which documents should humans label next, and how do you avoid only labelling the hardest outliers?
    - How do you balance labelling budget across document types, languages and tag classes?
72. [Likely] Customers' documents are confidential. How do you get labelled data from them (consent, on-tenant labelling, federated or per-tenant fine-tuning), and how do you keep tenants separate?
73. [Likely] Would you use LLMs or VLMs as annotators or pre-labellers? How do you check the quality of machine labels before they enter training?
74. [Likely] [21] Tools such as cleanlab estimate annotator quality and consensus labels for multi-annotator data with confident learning [21]. Those methods assume class labels. How would you extend consensus and annotator-quality estimation to trees, reading orders and table structures?
75. [Likely] [18] Donut pre-trains partly on SynthDoG, a synthetic document generator, across several languages [18]. Design a synthetic generator for tagged PDFs with a known ground-truth tree. How would you measure and close the gap between synthetic and real scanned documents?
    - Which tags are easy to synthesise, and which (alt text, reading order of complex layouts) are not?
76. [Likely] Labelling a full structure tree costs far more per page than drawing boxes. Build a labelling budget for the first year: which label types, how many pages, which mix of experts and vendors, and what you buy with each extra dollar.

J. Versioned label store and data management

```
label record = (doc_id, page, element_id, geometry, tag, parent, order_index,
                attributes, taxonomy_version, source{human|model|weak|synthetic},
                annotator_id, adjudicated?, created_at, supersedes)
snapshots: train/val/test frozen by hash  ──>  model registry links model <-> snapshot
```

77. [Likely] Design the versioned label store. What is the record schema, and how do you represent a tree, an order and a table so they can be diffed and queried?
    - How do you store several competing labels for the same element (multi-annotation, model vs human)?
78. [Likely] How do you make training and evaluation reproducible: frozen snapshots, dataset hashes, and links from each model version to the exact data it saw?
79. [Likely] How do you prevent leakage between train and test when the same template or vendor appears many times?
    - Would you split by document, by template, by customer or by time? Why?
80. [Likely] A customer asks you to delete all their data, including labels and anything derived from it. What does your label store and model lineage need to support?
81. [Likely] How do you track label quality over time and find bad labels in the store (for example, with model disagreement or confident-learning methods)?
82. [Likely] The same document comes back re-scanned, re-exported or with one page changed, so its hash is new. How do you keep document and element identity across versions so that labels, review history and customer outputs stay linked?

K. Evaluation

```
offline:  layout mAP | tag F1 (per class, per level) | tree-edit distance (whole tree)
          reading-order accuracy (pairwise / BLEU / edit distance) | table TEDS (+structure-only)
          alt-text quality (human rubric) | PDF/UA machine-check pass rate
online:   auto-publish rate | review minutes per doc | post-publish defect rate
          customer-reported accessibility issues | cost per page | p95 latency
```

83. [Likely] Define tag F1 for a tree. How do you match predicted elements to ground-truth elements before you count true positives?
    - Do you require geometric overlap, text overlap or both? What IoU threshold, and why?
    - Do you report micro or macro F1? How do you weight rare but critical tags (table headers) against frequent ones (paragraphs)?
84. [Likely] Why is element-level F1 not enough for a tree? Define a tree-edit-distance metric for the whole structure tree.
    - What are the insert, delete and rename costs, and should a wrong heading level cost as much as a missing table?
    - How do you normalise it so that scores compare across documents of different sizes?
85. [Likely] [4] [5] TEDS scores tables as 1 minus the tree edit distance divided by the size of the larger tree. Renaming a cell costs the normalised Levenshtein distance of its content, and a structure-only mode ignores the content [4] [5]. What are the weaknesses of TEDS for your use case?

    - The reference code does not clamp the score, so in theory it can fall below 0 [5]. Does that matter in reporting?
    - When would you report structure-only TEDS, full TEDS and cell-level adjacency F1 side by side?
    - How would you score header-cell identification, which accessibility depends on?
86. [Likely] [6] How do you measure reading-order accuracy? Compare page-level BLEU, average relative distance [6], Kendall's tau and pairwise accuracy within regions.
    - Should you measure reading order on tokens, on lines or on regions? What does each hide?
    - How do you score order when the ground truth allows several valid orders?
87. [Likely] How do you build an evaluation set that reflects your customers and not public benchmarks? How large must it be per class to detect a 1-point regression?
    - How do you stratify (native vs scanned, language, document type, page complexity)?
88. [Likely] Offline metrics improved but customers report more accessibility complaints. How do you debug the gap?
89. [Likely] How do you evaluate alt text and other generated text without relying only on an LLM judge? If you do use an LLM judge, how do you validate it?
90. [Likely] [15] olmOCR-Bench checks simple, machine-checkable "facts" about each page, like unit tests, and avoids soft metrics such as edit distance because a correct parse can differ from the reference [15]. Would you build a fact-based test suite for structure tagging? What facts would you write, and what would it miss that tree-edit distance or TEDS catches?
91. [Likely] [4] [12] Compare TEDS [4] with GriTS, the grid table similarity metric from the Table Transformer work [12]. For accessibility, where header-to-cell relations matter most, which would you trust, and why?
92. [Likely] [13] OmniDocBench reports one "Overall" score as the average of (1 - text edit distance) x 100, table TEDS and formula CDM [13]. What can go wrong when you rank systems on a composite like that? Would you publish a single number to leadership?
    - The same benchmark changed its matching algorithm between versions [13]. How can the matching step alone change a leaderboard, and how do you stop teams from tuning to it?
93. [Likely] How do you compute confidence intervals and significance for document metrics when pages from one document, or documents from one template, are not independent?
    - Would you bootstrap by page, by document or by template? What does each choice do to the width of the interval?
94. [Likely] Define the release gate for a new model version. Which metrics block a release, and which are only monitored?
    - How do you handle a model that improves table TEDS but makes heading F1 worse?

L. Serving at scale

```
         ┌─ realtime API (single doc, seconds; p95 SLA) ─┐
request ─┤                                               ├─> stage DAG on GPU/CPU pools
         └─ batch / backlog (millions of pages, hours) ──┘     (page-level fan-out,
                                                               doc-level fan-in)
caches: page hash -> OCR/layout result ; model registry ; per-tenant config + taxonomy
observability: per-stage latency, queue depth, confidence drift, validator failures
```

95. [Likely] Design serving for two workloads: an interactive API (one document, a few seconds) and a backlog of millions of pages. What is shared and what is separate?
    - Which stages run per page in parallel and which need the whole document?
    - How do you schedule GPU work across tenants fairly?
96. [Likely] Estimate the cost per 1,000 pages for your design. Which stage dominates, and what would you change to cut cost by 5x?
    - When is a smaller distilled model plus targeted review cheaper than a large VLM on every page?
    - One open VLM pipeline claims under $200 per million pages converted [14]. What does that figure leave out (review minutes, PDF writing, validation, retries, on-prem hardware)?
97. [Likely] How do you cache and deduplicate work (identical pages, repeated templates, re-runs after a model update)?
98. [Likely] How do you roll out a new model version safely: shadow mode, canary by tenant, rollback? How do customers pin a version [1]?
    - What does a customer expect when the same document gives a different tree after an upgrade?
99. [Likely] What do you monitor in production to detect drift (new templates, new scanners, new languages) before customers notice?
    - Which signals work without labels: confidence distribution, validator failure rate, review edit rate?
100. [Likely] Page cost for a VLM varies widely: a blank page and a dense 8-point financial table differ by an order of magnitude in tokens. How do you batch and schedule mixed pages on GPUs to meet a p95 latency target without wasting capacity?
101. [Likely] Large customers want models fine-tuned on their own documents. How would you serve hundreds of per-tenant variants (for example, adapters on a shared base model)? What breaks first: memory, cold starts, evaluation or release management?
102. [Likely] How does the design change for an on-prem, air-gapped deployment with no GPU?
103. [Likely] How do you make the pipeline fail safe? If a stage crashes or times out on a 500-page document, what does the customer get? - How do retries and idempotency work when outputs are written to the customer's storage?

M. Integration with UiPath products and agents

104. [Likely] [1] How would robots and agents consume the tagging output, for example, to drive classification, extraction and validation steps [1]? - What do agents need from the tree that a screen reader does not (stable element IDs, provenance, confidence)?
105. [Likely] How would you use the structure tree to improve retrieval (chunking by section, keeping tables whole) for agent grounding over enterprise documents? - How would you measure whether structure-aware chunking actually improves answer quality?

106. [Likely] An agent asks a question about a table on page 47, and the answer is wrong because the table was split across pages. Whose bug is this, and how would your metrics have caught it?
107. [Likely] [25] UiPath's Python SDK exposes a Context Grounding service that agents use to search indexed content [25]. Define the contract between the tagging system and that index: which fields from the tree must the index keep (element IDs, section path, table cells, page boxes) so that an agent can cite the exact source?
108. [Likely] How would you govern this system for regulated customers: audit logs, explainability of each tag, data residency, and model risk management?

N. Research depth and open problems

109. [Likely] What are the hardest open research problems in document structure understanding today, and which would you fund first? - Where do you expect vision-language models to replace the cascade within two years, and where not?
110. [Likely] How would you build a single model that predicts layout, reading order and hierarchy jointly? What would the training targets and the loss look like? - How do you represent a tree as a target sequence, and how do you keep the decoded output valid?
111. [Likely] How would you adapt the system to a new document domain with 50 labelled documents? - Which parts transfer, and where do you spend the labels?
112. [Likely] How would you use self-supervised or weakly supervised signals from native PDFs that already have good tags, or from Word and HTML sources, to train models for scanned documents?
113. [Likely] How would you estimate and report uncertainty over a whole tree rather than over single nodes?
114. [Likely] [14] One open document-OCR model reports gains from synthetic data and reinforcement learning [14]. Design a reinforcement-learning setup for structure tagging with verifiable rewards, for example PDF/UA rule passes, fact-based tests and table-structure checks. How would you stop the model from gaming the reward?
115. [Likely] A per-page model cannot see that "Section 4.2" on page 90 must sit under "Chapter 4" on page 61. How would you give a model document-level context for hierarchy and cross-page objects at an acceptable cost?
116. [Likely] What would make you stop using public benchmarks such as DocLayNet, PubTabNet or ReadingBank [3] [4] [6] for decisions? What would you publish instead?

O. Principal-level scoping, roadmap and trade-offs

117. [Likely] Present a 4-quarter roadmap for this system. What ships in quarter 1 to deliver customer value while you build the learning loop? - What do you deliberately not build?
118. [Likely] Which teams own which parts (OCR, layout, tables, review UI, PDF writer, label operations, platform)? Where do you expect friction, and how do you resolve it?
119. [Likely] Product wants "100% PDF/UA compliant" in marketing. How do you respond, and what claim would you sign off on?
120. [Likely] A large customer escalates: 2% of tagged PDFs read tables in the wrong order to screen-reader users. Walk through your first week. - How do you separate a

model problem from a writer bug or a taxonomy gap? - What process change stops this class of issue from recurring?

121. [Likely] How would you decide between investing in a better model and investing in a better review experience, given a fixed budget?
122. [Likely] Walk through a document-AI or structured-prediction system you shipped. Which design decision in this chapter would you make differently because of it? - What metric did you get wrong at first, and how did you find out? - What would you tell a new team on day one so they avoid the same mistake?

References

1. UiPath, Documents service source, `uipath-platform` package of `uipath-python`: classify, extract (DU Modern and IXP), published project version and tag parameters, a separate digitization step (start, then poll for a result), and validation actions that block until a person completes them in Action Center; marked experimental (official; fetched in this pass). GitHub, 2025-2026. https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath-platform/src/uipath/platform/documents/_documents_service.py
2. Docling project, "Docling" README: PDF understanding including page layout, reading order, table structure, OCR for scanned PDFs, a unified DoclingDocument model, and export to Markdown, HTML, DocTags and lossless JSON (fetched). GitHub. https://raw.githubusercontent.com/docling-project/docling/main/README.md
3. DS4SD (IBM Research), "DocLayNet" README: 80,863 expert-annotated pages from six document categories, 11 layout classes, with a fraction of pages annotated two or three times (fetched). GitHub, 2022. https://raw.githubusercontent.com/DS4SD/DocLayNet/main/README.md
4. IBM, "PubTabNet" README: over 568k table images with HTML structure annotations and cell boxes; introduces the TEDS metric (fetched). GitHub, 2019-2020. https://raw.githubusercontent.com/ibm-aur-nlp/PubTabNet/master/README.md
5. IBM, PubTabNet TEDS reference implementation, `metric.py`: TEDS = 1 - edit distance / max node count, normalised Levenshtein rename cost for cells, structure-only option (fetched). GitHub. https://raw.githubusercontent.com/ibm-aur-nlp/PubTabNet/master/src/metric.py
6. Microsoft, "LayoutReader" README (unilm): reading-order prediction with text and layout, ReadingBank benchmark of about 500K pages with weakly supervised labels, average page-level BLEU and ARD metrics; EMNLP 2021 (fetched). GitHub. https://raw.githubusercontent.com/microsoft/unilm/master/layoutreader/README.md
7. Microsoft, "LayoutLMv3" README (unilm): unified text and image masking, word-patch alignment, results on FUNSD and PubLayNet; content licensed CC BY-NC-SA 4.0 (fetched). GitHub, 2022. https://raw.githubusercontent.com/microsoft/unilm/master/layoutlmv3/README.md
8. Docling project, "docling-ibm-models" README: TableFormer table structure and cell box prediction; lists PubTabNet, FinTabNet and TableBank (fetched). GitHub. https://raw.githubusercontent.com/docling-project/docling-ibm-models/main/README.md
9. veraPDF, "veraPDF-library" README: industry-supported PDF/A and PDF/UA validation (fetched). GitHub. https://raw.githubusercontent.com/veraPDF/veraPDF-library/integration/README.md

10. veraPDF, "veraPDF-validation-profiles" README: rules and profiles for PDF/UA-1, PDF/UA-2 and WTPDF 1.0; WCAG 2.2 profiles experimental (fetched). GitHub. https://raw.githubusercontent.com/veraPDF/veraPDF-validation-profiles/integration/README.md
11. Microsoft / doc-analysis, "TableBank" README: 417,234 tables labelled by weak supervision from Word and LaTeX sources; detection and structure recognition tasks; research use only (fetched). GitHub. https://raw.githubusercontent.com/doc-analysis/TableBank/master/README.md
12. Microsoft, "Table Transformer (TATR)" README: PubTables-1M (CVPR 2022), the GriTS grid table similarity metric, pre-trained weights for PubTables-1M and FinTabNet only, with guidance to train your own multi-domain model (fetched). GitHub, 2022-2023. https://raw.githubusercontent.com/microsoft/table-transformer/main/README.md
13. OpenDataLab, "OmniDocBench" README: 1,651 PDF pages, 10 document types, 28 block-level and 4 span-level element types; normalised edit distance, TEDS and CDM; Overall = ((1 - text edit distance) x 100 + table TEDS + formula CDM) / 3; matching algorithm updates across versions (fetched). GitHub, 2024-2025. https://raw.githubusercontent.com/opendatalab/OmniDocBench/main/README.md
14. Allen Institute for AI, "olmOCR" README: VLM-based PDF conversion, under $200 per million pages claimed; release notes on fixing hallucinations on blank documents and auto-rotation, and on synthetic data plus RL training (fetched). GitHub, 2025. https://raw.githubusercontent.com/allenai/olmocr/main/README.md
15. Allen Institute for AI, "olmOCR-Bench" README: machine-checkable per-page "facts" like unit tests, in place of soft metrics such as edit distance (fetched). GitHub, 2025. https://raw.githubusercontent.com/allenai/olmocr/main/olmocr/bench/README.md
16. Datalab, "marker" README: Apache 2.0 code with model weights under a modified OpenRAIL-M licence with revenue limits; optional LLM mode that merges tables across pages and extracts form values (fetched). GitHub, 2025-2026. https://raw.githubusercontent.com/datalab-to/marker/master/README.md
17. OCRmyPDF README: adds an OCR text layer to scanned PDFs, "lossless" where possible, and outputs PDF/A by default (fetched). GitHub. https://raw.githubusercontent.com/ocrmypdf/OCRmyPDF/main/README.md
18. NAVER Clova, "Donut" README: OCR-free document understanding transformer and SynthDoG synthetic document generator (fetched). GitHub, 2022. https://raw.githubusercontent.com/clovaai/donut/master/README.md
19. Microsoft / doc-analysis, "DocBank" README: 500K pages with token-level labels for 12 kinds of semantic unit, built by weak supervision (fetched). GitHub, 2020. https://raw.githubusercontent.com/doc-analysis/DocBank/master/README.md
20. IBM, "PubLayNet" README: layout annotations generated automatically by matching PDFs to their XML from the PubMed Central commercial-use subset (fetched). GitHub, 2019. https://raw.githubusercontent.com/ibm-aur-nlp/PubLayNet/master/README.md
21. Cleanlab, "cleanlab" README: confident-learning label-error detection, and consensus and annotator-quality estimation for multi-annotator data (fetched). GitHub. https://raw.githubusercontent.com/cleanlab/cleanlab/master/README.md
22. veraPDF, PDF/UA-1 validation profile `PDFUA-1.xml`: 106 machine-checkable rules, including MarkInfo Marked = true and no artifact content inside tagged content

(fetched). GitHub. https://raw.githubusercontent.com/veraPDF/veraPDF-validation-profiles/integration/PDF_UA/PDFUA-1.xml
23. Meta AI, "Nougat" README: academic-document PDF parser that outputs LaTeX math and tables (fetched). GitHub, 2023. https://raw.githubusercontent.com/facebookresearch/nougat/main/README.md
24. Docling project, "docling-core" README: the DoclingDocument data model defined as a Pydantic model (fetched). GitHub. https://raw.githubusercontent.com/docling-project/docling-core/main/README.md
25. UiPath, "UiPath Python SDK" README: SDK for UiPath Cloud Platform services, including the Context Grounding service used to search indexed content (official; fetched). GitHub, 2025-2026. https://raw.githubusercontent.com/UiPath/uipath-python/main/README.md
26. PDF Association, "PDF Specification Issues" tracker README: public errata and amendments for ISO 32000-2 (PDF 2.0) and PDF/UA-2 (fetched). GitHub. https://raw.githubusercontent.com/pdf-association/pdf-issues/main/README.md

09. Cross-Portal ML System Design Bank: Peer-Company Prompts with Block Diagrams and UiPath Recasts

> **How to read this chapter.** You asked for ML system design diagrams and an explanation of the tag system, so this chapter is a bank of design prompts. Each prompt family has one plain-text block diagram, a source tag and a "UiPath recast" note that turns the generic prompt into a UiPath problem. The diagrams draw only the prompt's problem space: the inputs, the parts the interviewer expects you to fill in, and the outputs. They are scaffolds, not solutions. As with the rest of the pack, this chapter gives questions only, not answers.

The tag system (read this first)

Each numbered question carries exactly one tag.

Tag	Meaning	Evidence needed
[Reported] [n]	A candidate report that we opened, or an official company page, links this question to a named company's loop (UiPath or a peer).	A primary source [n] that we fetched and read in this run.
[Reported-2nd] [n]	A candidate-report aggregator that we fetched and read [n] links the question to a named company's interview, with a report date or date range. We could not open the original candidate post.	A secondary source [n] that we fetched and read in this run. Treat the company link as probable, not certain.
[Bank] [n]	The prompt appears in a public ML system design question bank or study guide [n]. That source either ties it to no company, or carries only a maintainer-assigned company label with no per-question report behind it.	A source [n] that we fetched and read in this run.
[Likely]	An inference from the Principal Applied Scientist role, the level and UiPath-peer products. No source reports it.	None; it is judgement.

Other markers:

- **Peer anchor:** a published engineering system at a peer company (title, company, year, as listed in the curated index [9]) that shows a real production version of the prompt. A peer anchor is context. It is **not** evidence that the company asks the prompt in interviews.
- **UiPath recast:** how to turn the prompt into a UiPath problem (agents, Document Understanding, Communications Mining, Autopilot, the AI Trust Layer, Process Mining, UI-automation vision). Recasts are always [Likely].
- Company and date notes in brackets after a question, such as "(Microsoft MLE loop; reports 2026 Q1-Q3)", come from the cited source.
- Indented sub-bullets are follow-up probes, phrased as questions. Probes are [Likely] unless they carry their own citation.

Research limits (please read)

This chapter set out to collect prompts that candidates report on Glassdoor, Blind, LeetCode Discuss, Exponent, interviewing.io, IGotAnOffer, Reddit, Interview Query and 1point3acres for Applied, Principal and Staff Scientist loops at Microsoft, Amazon, Google, ServiceNow, Automation Anywhere and Celonis. Two passes ran in October 2026. In both, the workflow's shared web-search budget (200 searches per turn) was already used up, so no web search could run. Direct fetches failed with DNS errors for igotanoffer.com, tryexponent.com, interviewing.io, glassdoor.com, reddit.com, medium.com, interviewquery.com, 1point3acres.com, huyenchip.com and web.archive.org. Only github.com could be reached.

What the second pass did instead:

- It re-fetched and checked the original GitHub sources [1]-[10]. All resolved. Several questions had cited the MLSD directory listing [5] for prompts that appear only in the sample-question list of [1]. Those citations now point to [1].
- It used GitHub code search to find GitHub-hosted mirrors of candidate-report aggregators. These are TrueInterview's company guides and question pages, mirrored with a CC BY 4.0 licence [11]-[20], a curated list of Reddit, Blind and X interview threads [21], and an archived LinkedIn interview-experience post [24]. Questions taken from these sources are tagged **[Reported-2nd]**, because the original candidate posts could not be opened.
- Two company-tagged GitHub banks, [22] and [23], label questions with companies but give no per-question report. Their questions are tagged **[Bank]**.
- **No source found names UiPath, ServiceNow, Automation Anywhere or Celonis for an ML system design question.** The ServiceNow and Celonis code searches returned no interview content, and the UiPath search returned only unrelated pages. The [Reported-2nd] items therefore come from Microsoft, Amazon, Google, Meta, Databricks and other enterprise-AI or fraud-heavy peers. Treat them as calibration for the shape and level of the round, not as UiPath's own questions.
- To upgrade [Reported-2nd] to [Reported], open the original posts on the portals above in a run that has search budget.

Format notes for this round

Item	What the sources say	Source
Structure	A 9-step frame: problem formulation, metrics (offline and online), high-level architecture, data, features, modelling and offline evaluation, prediction service, online testing and deployment, scaling and monitoring	[1], [2]
Flexibility	Interviewers may want you to skip some components and go deep on one or two	[1]
GenAI round	"Increasingly common as a separate round"; interviewers "care more about cost, latency, guardrails, and monitoring" than about the diagram; "evaluation methodology is the new system design"	[1]
What an Amazon AS2 candidate saw (2026)	ML system design was "the most important round". It was "less about fancy models": scope, data, baseline then advanced model, offline versus online training, business-aligned evaluation and A/B tests, batch versus real-time serving, scale, and drift and feedback-loop monitoring	[24]

Item	What the sources say	Source
Prompt shape at peers (2025-2026)	Product-shaped prompts with a deliberate twist: position-bias correction, calibration, offline metrics that disagree with online ones, adversarial inputs, or "is the new model ready to replace the old one?"	[12], [13], [14], [21]
Company spread	Prep authors find interview components "very similar across FAANG companies"; startups tailor prompts to their own products	[10]
Duration and interviewer	Not given by any fetched source. Peer loops usually run 45-60 minutes with a staff or principal scientist or engineer; confirm with your recruiter	[Likely]

Principal-level bar (inferred, [Likely]). At principal level, the bar is less about naming the right model and more about these:

- framing the business problem and its SLOs;
- choosing where ML belongs and where deterministic logic (rules, robots, workflows) should take over;
- designing the evaluation programme before the model;
- planning for cost, latency and multi-tenant governance;
- describing a rollout and a feedback loop that other teams can run;
- naming the two or three decisions that are hard to reverse.

Canvas used by every diagram below

```
[ Inputs / data ] --> [ ? stage A ] --> [ ? stage B ] --> [ Output / action ]
                          ^                                      |
                          +------- feedback / labels <-----------+
Boxes marked "?" are the parts the interviewer expects you to design.
```

A. Search and ranking

A1. Product, document and enterprise search with retrieval and ranking

Tag basis: [Bank] [1], [3], [7], [23]; [Reported-2nd] [12], [21]. Peer anchors in [9]: Amazon "Semantic Product Search" (2019), Amazon "QUEEN: Neural query rewriting in e-commerce" (2021), Microsoft "AI at Scale in Bing" (2020), Google "A Guide to Google Search Ranking Systems" (2022).

```
query --> [ ? query understanding ] --> [ ? candidate retrieval ] --> [ ? ranker ]
          (rewrite, intent, lang)       (lexical | dense | hybrid)    (LTR / x-enc)
                                                                        |
catalogue / corpus --> [ ? indexing + embeddings ]                      v
clicks, conversions, dwell --> [ ? labels + debiasing + training ] --> ranked results
```

UiPath recast: search over the automation catalogue (Marketplace components, processes, agents, API workflows) or over Orchestrator assets, where a query such as "invoice approval for SAP" must return reusable automations. Labels come from reuse and deploy events, not clicks.

1. Design a search system for an e-commerce platform that returns a ranked list of products for a text query at about 50M daily active users. [Bank] [3]

    - Which of the clarifying questions in [3] (product-only versus general search, personalisation, languages, latency budget) would change your architecture most?
    - How do you split the latency budget between retrieval and ranking?
2. Design a search retrieval and ranking pipeline, including online experiments and position-bias correction. (Amazon MLE loop; reports 2026 Q1-Q3) [Reported-2nd] [12]
    - How do you estimate position bias without degrading the live experience too much?
    - Recast: the automation catalogue shows only a few results above the fold. How would you debias deploy and reuse signals there? [Likely]
3. Design a unified query engine over a user's data spread across several products, such as mail and file storage. (Google, AI role) [Reported-2nd] [21]
    - How do you rank results that come from sources with incomparable relevance scores?
    - Recast: one query engine over Orchestrator queues, process-mining logs, Communications Mining streams and DU outputs for one tenant. [Likely]
4. Design a document search system that supports full-text and semantic queries. [Bank] [1]
    - When does hybrid retrieval beat pure dense retrieval, and how would you show it offline?
5. Design search for an internal catalogue of automations and agents, where "relevance" means "the user deployed and reused it". [Likely]
    - What is your implicit-feedback signal when most users search once and never click?
    - How do you stop popular legacy automations from crowding out better new ones?
6. Design a learning-to-rank system for a marketplace where most items have little interaction data (cold start). [Likely]
    - How would you use content features or LLM-generated descriptions to rank new items?
7. Design a system that answers from content that appeared only 30 minutes ago. (Perplexity) [Bank] [23]
    - Recast: how do you keep an agent's knowledge layer current when a customer's SOPs change mid-day? [Likely]

A2. Query autocompletion and related searches

Tag basis: [Bank] [1], [5], [8].



UiPath recast: prompt completion in Autopilot or in a natural-language-to-workflow box, where the completion must map to activities that actually exist in the tenant.

8. Design an autocompletion or typeahead suggestion system. [Bank] [1], [5]
    - How would you meet a per-keystroke latency target with an LLM in the loop?
9. How would you generate a list of related searches for each query? [Bank] [8]

    - How do you keep related searches from leaking another tenant's query terms?
10. Design next-step suggestions for a workflow designer: given the steps a user has built so far, suggest the next activity. [Likely]
    - Is this ranking, sequence modelling or retrieval, and why?
    - How do you evaluate it when "accepted" suggestions may still be wrong?

A3. Ranking a feed, a page or a queue

Tag basis: [Bank] [1], [5], [7], [8]; [Reported-2nd] [14], [18]. Peer anchors in [9]: LinkedIn "Understanding Dwell Time to Improve LinkedIn Feed Ranking" (2020); Airbnb "Managing Diversity in Airbnb Search" (2020).

```
items (posts | modules | notifications | tasks) --> [ ? features ] --> [ ? multi-objective ranker ]
user / agent state ------------------------------------^                    |
                                                                            v
                                    [ ? re-rank: diversity, SLA, fairness, caps ] --> ordered list
```

UiPath recast: rank the human-in-the-loop queue in Action Center, or the exception queue for a process, by business urgency, SLA risk and which reviewer is best placed to handle each item.

11. Design a newsfeed ranking system. [Bank] [1], [5], [7]
    - How do you combine several objectives (click, dwell, long-term retention) into one score?
12. Design a unified ranking model for notifications from several products, balancing engagement, latency and fairness across notification types. (Meta MLE loop; reports 2025 Q4-2026 Q3) [Reported-2nd] [14]
    - Recast: one ranker for alerts from robots, agents, process-mining monitors and DU validation tasks, sent to the same operations lead. How do you stop one source from flooding the others? [Likely]
13. Predict the probability that a user dislikes a feed post, and explain why offline and online metrics disagree. (Meta MLE loop) [Reported-2nd] [14]
14. Build a top-level ranking of the modules shown on an e-commerce landing page. (Wayfair; last reported Sep 2026) [Reported-2nd] [18]
    - Recast: rank the widgets on an Automation Cloud home page (failed jobs, pending approvals, suggested automations) per persona. [Likely]
15. How do you build a model that ranks all the answers to a question, and what does it cost to run? [Bank] [8]
16. Design a ranking system for a queue of human-review tasks that a fleet of agents and robots generates, so that reviewers clear the items with the highest business risk first. [Likely]
    - What label tells you a ranking was "good" when every item eventually gets done?
    - How would you stop the ranker from starving low-value but SLA-bound items?
17. Rank the top 10 results for a rental-listing search. [Bank] [8], [7]
    - How would you add diversity without hurting conversion?

B. Recommendation

B1. Item recommendation (two-stage) and model replacement

Tag basis: [Bank] [1], [5], [7], [8]; [Reported-2nd] [13], [21]. Peer anchors in [9]: Amazon "Item-to-Item Collaborative Filtering" (2003), YouTube "Deep Neural Networks for YouTube Recommendations" (2016), Google "Self-supervised Learning for Large-scale Item Recommendations" (2021).

```
user history + context --> [ ? candidate generation ] --> [ ? ranking model ]
                           (CF | two-tower | graph)        |
item catalogue -------------------^                         v
                              [ ? business rules / re-rank ] --> top-N recs
impressions, accepts --> [ ? logging + training loop ] --------------^
                                    |
                     [ ? champion / challenger gate ] --> replace old model?
```

UiPath recast: recommend which processes to automate next, or which agent or Marketplace component to adopt, for a customer tenant. Use process-mining signals and the tenant's existing automations as the history.

18. Design a video recommendation system. [Bank] [1], [5], [7]
    - Why use two stages, and what does each stage optimise?
19. Design a real-time news-feed recommender, covering cold start, latency budgets, drift monitoring and fallbacks. (Google MLE loop; reports 2025 Q4-2026 Q3) [Reported-2nd] [13]
    - What is your fallback when the feature store is down, and how do you test it?
20. You have trained a new recommendation model. How do you make sure it is ready to replace the old one? (Netflix MLE) [Reported-2nd] [21]
    - Which offline checks, shadow traffic and online guardrail metrics would you require?
    - Recast: a new DU extraction model or agent-planning model is ready for thousands of tenants. What is the gate? [Likely]
21. Design a recommendation system to suggest replacement items when the one ordered is unavailable. [Bank] [8], [1]
22. Design a system that recommends the next automation opportunity to a customer, using task-mining and process-mining data from that customer. [Likely]
    - What is the label: automations built, automations still running after 90 days, or hours saved?
    - How do you recommend for a new tenant with no automation history?
23. Design "people you may know" or who-to-follow recommendations. [Bank] [1], [5], [8]
    - Recast: how would the same graph approach recommend which colleague or Centre of Excellence member should review an automation? [Likely]

B2. LLM-assisted recommendation and personalisation

Tag basis: [Bank] [1]; [Reported-2nd] [11].

```
user profile + history + memory --> [ ? retrieval (classic) ] --> candidates
                                                                    |
                              [ ? LLM as ranker / feature generator / explainer ]
                                                                    |
                                                                    v
                                                  ranked + explained recommendations
```

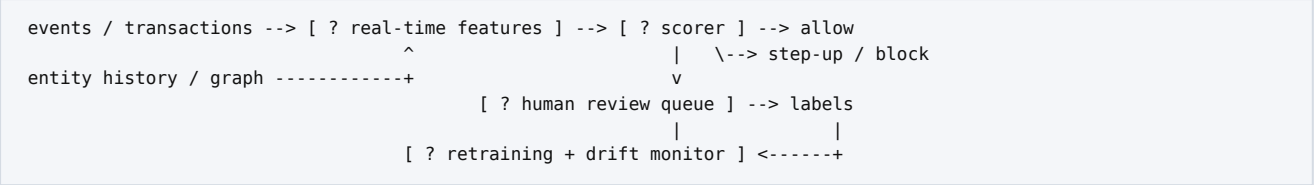
UiPath recast: Autopilot suggests which agent or automation to build, with a natural-language reason the Centre of Excellence lead can audit.

24. Design an LLM-based recommendation or personalisation system, using the LLM as a ranker or as a feature generator. [Bank] [1]
    - When is an LLM ranker worth its cost and latency compared with a gradient-boosted ranker?
    - How do you test that the LLM's explanations are faithful to the real ranking signal?
25. Design the personalisation and memory system for a chatbot. (Microsoft MLE loop; reports 2026 Q1-Q3) [Reported-2nd] [11]
    - What should be remembered, for how long, and who can delete it?
    - Recast: per-user and per-tenant memory for UiPath agents under the AI Trust Layer's retention rules. [Likely]
26. Design a system that recommends reusable workflow templates while a user describes a task in natural language. [Likely]

C. Anomaly and fraud detection

C1. Transaction fraud and account takeover with humans in the loop

Tag basis: [Bank] [1], [8]; [Reported-2nd] [15]. Peer anchor in [9]: Uber "Project RADAR: Intelligent Early Fraud Detection System with Humans in the Loop" (2022).



UiPath recast: detect fraud or policy breaches in invoices and payments that Document Understanding and finance automations process, such as duplicate invoices, vendor-bank-detail changes and split purchase orders. Flagged items go to Action Center.

27. How would you build a fraud detection system from labelled purchase records? [Bank] [8], [1]
    - How do you train when confirmed fraud labels arrive weeks late and are biased by what was reviewed?
    - How do you set thresholds when the review team's capacity is fixed?
28. Design an account-takeover detection system that catches the takeover at the moment the attacker tries a hazardous action. (Stripe, Affirm, Robinhood, Roblox, Uber; last reported Oct 2025) [Reported-2nd] [15]
    - What latency budget does a "hazardous action" check have, and which features must be precomputed?
    - Recast: detect a hijacked robot or agent credential at the moment it tries to change vendor bank details. [Likely]
29. Design an anomaly detection system for invoices flowing through an accounts-payable automation, so that suspicious invoices are held before payment. [Likely]

    - Which signals come from the document itself, and which from the vendor's history?
    - How would you explain a hold to an auditor?
30. Design a system that detects when the automation itself is being misused, for example a robot account used to move money outside policy. [Likely]

C2. Abuse and anomaly detection in behaviour logs

Tag basis: [Likely]. Peer anchors in [9]: LinkedIn "Detecting and Preventing Abuse on LinkedIn using Isolation Forests" (2019), LinkedIn "Preventing Abuse Using Unsupervised Learning" (2020), LinkedIn "Using deep learning to detect abusive sequences of member activity" (2021).

```
activity / telemetry streams --> [ ? sessionise + featurise ] --> [ ? unsup. / seq model ]
                                                                          |
                                    [ ? alert triage + dedup ] <--------+
                                              |
                                              v
                                 investigations --> labels --> [ ? semi-sup. retrain ]
```

UiPath recast: anomaly detection on robot and agent run logs in Orchestrator. Flag runs whose step sequence, duration or error pattern drifts. These often signal that a target application's UI changed and the robots are about to break.

31. Design an unsupervised anomaly detection system for event sequences, with almost no labels at the start. [Likely]
    - How would you move from isolation-forest style scores to a supervised model as investigation labels arrive?
32. Design a system that predicts robot or agent run failures before they happen, from execution logs and screenshots. [Likely]
    - How do you tell "the target app changed" apart from "the robot logic is wrong"?
    - What alert precision do on-call operators need before they trust the system?
33. Design a monitoring system that finds process-level anomalies (bottlenecks, rework loops, unusual paths) in process-mining event logs across thousands of process variants. [Likely]

D. RAG enterprise assistants and agents

D1. Chat with your docs / enterprise Q&A with permissions

Tag basis: [Bank] [1], [8], [22]; [Reported-2nd] [11], [21].

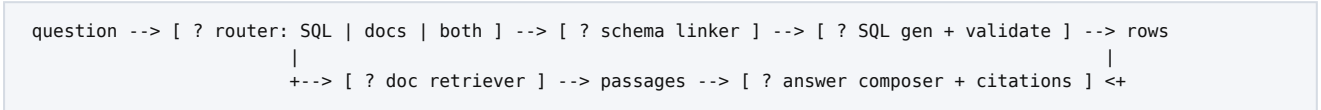
```
enterprise sources (wiki, tickets, PDFs, email) --> [ ? ingest: parse, chunk, ACL tag ]
                                                                          |
                                                          [ ? embed + index ]
                                                                          |
user question --> [ ? query rewrite ] --> [ ? hybrid retrieve + re-rank ]
                                                                          |
                        [ ? grounded generation + citations + "no answer" ]
                                                                          |
                                    [ ? guardrails ] --> answer --> feedback
```

UiPath recast: an Autopilot or agent knowledge layer, grounded in a customer's SOPs, process documentation and past exception resolutions, that must respect per-document permissions and the AI Trust Layer's data controls.

34. Design a RAG document Q&A ("chat with your docs") system that can extract answers from a large collection of documents. [Bank] [1], [8]
    - When would you pick RAG, fine-tuning, long context or tools and memory for this problem? [1]
    - How do you handle questions the corpus cannot answer?
35. Design an enterprise RAG assistant over 10M documents with per-user permissions. (Labelled OpenAI, Microsoft, Amazon, Databricks, Scale AI by the bank's maintainer) [Bank] [22]
    - How do you enforce document-level access control at retrieval time, not after generation?
    - How do you keep the index fresh when source permissions change hourly?
36. Design a Copilot feature that answers questions over a user's work email, documents and meetings without leaking content the user cannot see. (Labelled Microsoft) [Bank] [22]
37. Design a RAG system with agentic tools. (Microsoft MLE loop; reports 2026 Q1-Q3) [Reported-2nd] [11]
    - When should the agent call a tool instead of retrieving, and how do you evaluate that choice?
38. A RAG system is hallucinating in production. How do you tell whether the retriever or the generator is at fault? (Perplexity MLE) [Reported-2nd] [21]
    - Which traces would you need to have logged in advance to answer this within a day?
39. Design a RAG assistant that answers employees' "how do I do X in system Y" questions and can hand off to an automation that does X. [Likely]
    - Where does the boundary sit between answering a question and executing an action?
40. Design retrieval over scanned and semi-structured documents (invoices, contracts, forms), where layout carries meaning. [Likely]
    - How do you chunk a table or a multi-page form without breaking the facts in it?

D2. Question answering over structured and unstructured data

Tag basis: [Reported-2nd] [13]; [Bank] [22].



UiPath recast: an agent that answers "which vendors missed SLA last quarter, and what did their contracts say?" by joining process-mining tables with contract PDFs extracted by Document Understanding.

41. Design a chatbot that answers from both structured tables and unstructured documents. (Google MLE loop; reports 2025 Q4-2026 Q3) [Reported-2nd] [13]

- How do you evaluate an answer that is half a SQL result and half a cited passage?

42. Design a Text-to-SQL system over a warehouse with thousands of tables. (Labelled Databricks, Palantir) [Bank] [22]
    - How do you find the right 5 tables out of 5,000 before generation?
    - Which generated queries must a human approve before they run?

D3. Customer-support assistant with human fallback

Tag basis: [Bank] [1], [8], [22]; [Reported-2nd] [12].

```
customer message --> [ ? intent + risk triage ] --> [ ? RAG answerer | tool calls ]
                                |                                |
                                v                                v
                    [ ? human agent queue ] <-- low confidence / policy hit
                                |
                                v
                     resolution --> [ ? feedback + eval set ]
```

UiPath recast: a Communications Mining plus agent pipeline. Incoming customer emails are classified, entities are extracted, the agent drafts or completes the resolution through robots, and humans handle the rest.

43. Architect an AI chatbot, covering intent detection, dialogue state, generation, fallback and continuous learning. (Amazon MLE loop; reports 2026 Q1-Q3) [Reported-2nd] [12]
    - What confidence signal decides the handoff, and how do you calibrate it?
44. Design a customer-support agent that can take real actions, with escalation to humans. (Labelled Sierra) [Bank] [22]
    - Which actions need a human approval step, and how do you decide?
45. Build a chatbot to help people book hotels. [Bank] [8]
    - Recast: how does the design change when "booking" is an enterprise transaction a robot executes in SAP? [Likely]
46. Design a system that reads high-volume inbound email for a bank's operations team, routes each message, extracts fields and triggers the right automation. [Likely]
    - How do you add a new intent without retraining everything?
    - How do you measure automation rate without rewarding wrong automations?

D4. Agentic assistant (planning, tools, memory)

Tag basis: [Bank] [1], [22]; [Reported-2nd] [20].

```
goal --> [ ? planner ] --> [ ? tool / robot selection ] --> [ ? executor ]
           ^                                                    |
           |                 [ ? memory: short / long ] <-------+
           |                                                    v
           +------------ [ ? verifier + HITL checkpoint ] <-- results
```

UiPath recast: this is the core UiPath agentic-automation problem (agents that orchestrate robots, APIs and people). Chapter 04 covers it in depth, so this chapter keeps only the peer versions.

47. Design an agentic workflow or AI assistant covering planning, tool use and memory. [Bank] [1]
    - How do you detect and stop loops and hallucinated tools? [1]
    - Where do you put human-in-the-loop checkpoints?
48. Design an agent that operates a web browser to complete multi-step tasks. How do you make it reliable enough to ship? (Labelled Amazon) [Bank] [22]
    - Recast: the same agent drives a legacy desktop app over Citrix with no DOM. What changes in perception, recovery and evaluation? [Likely]
49. Design a stock-trading agent system. (Databricks; last reported Apr 2026) [Reported-2nd] [20]
    - Which actions are irreversible, and what limits, approvals and kill switches sit around them?
50. Design a code assistant or coding agent that uses RAG over a repository, tool use and verification. [Bank] [1]
    - Recast: how would the same verifier pattern apply to an agent that generates automation workflows? [Likely]

E. Content moderation and safety

E1. Harmful and policy-violating content detection

Tag basis: [Bank] [1], [4], [22]; [Reported-2nd] [14], [16], [21].

```
post / ad (text | image | video) --> [ ? per-modality encoders ] --> [ ? fusion ]
                                                                          |
                                        [ ? multi-label / multi-task heads ]
                                                                          |
                                        remove | demote | allow | send to review
user reports + reviewer labels --> [ ? label pipeline + retrain ] -------^
```

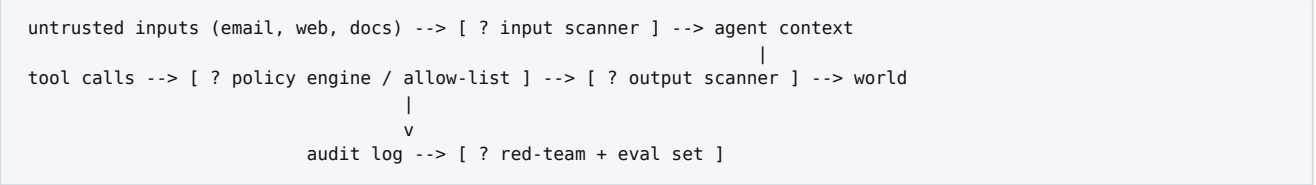
UiPath recast: screen what agents send out (emails, chat replies, generated documents) and what they take in (inbound documents and emails that may carry prompt injection, PII or toxic content) inside the AI Trust Layer.

51. Design a system that monitors posts, detects harmful content and demotes or removes it (about 500M posts a day and about 10K labelled examples). [Bank] [4]
    - Which clarifying questions in [4] (harm types, languages, user reports, explainability, legal constraints) change the design most?
    - How do you get to a usable model from 10K labels?
52. Design a content moderation system, given traffic data. (Google L6 MLE) [Reported-2nd] [21]
53. Design detection of weapon sales in ads, from data collection and labelling to the enforcement pipeline. (Meta MLE loop) [Reported-2nd] [14]
    - How do you handle sellers who change wording and images to evade the classifier?
54. Design copyright detection for uploaded images, including multi-image posts, adversarial overlaid text and photos of copyrighted photos. (Meta MLE loop) [Reported-2nd] [14]

    - Recast: detect a re-photographed or re-scanned copy of an invoice that was already paid. [Likely]
55. Design a content-moderation system that combines classifiers and LLMs. (Labelled Meta) [Bank] [22]
    - Which decisions go to the cheap classifier, which to the LLM, and which to a human?
56. A spam classifier shows strong offline metrics but misses a large share of spam in live traffic. Investigate and redesign it for production. (Indeed; last reported Aug 2026) [Reported-2nd] [16]
    - Which of label shift, adversarial drift, sampling bias and feature skew would you check first, and how?
57. Design an output guardrail that checks every outbound message an enterprise agent writes for PII, policy breaches and hallucinated commitments before it is sent. [Likely]
    - What is the latency budget, and which checks can run asynchronously?
    - How do you measure the false-block rate customers will tolerate?

E2. Prompt-injection, data-leak and memorisation defence

Tag basis: [Bank] [1], [22]; [Reported-2nd] [14].



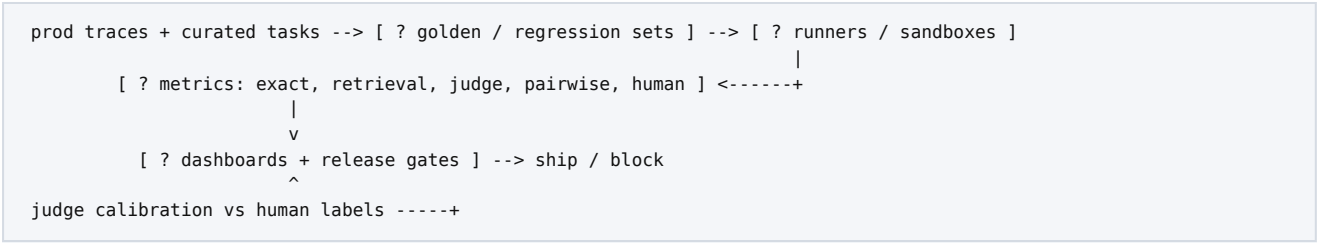
UiPath recast: agents that read customer emails and documents and then act in ERP systems give an injected instruction a direct path to a real transaction.

58. Design guardrails for a consumer-facing assistant. Do you filter inputs, outputs or both? (Labelled Sierra, Character.AI) [Bank] [22]
59. Design a defence against prompt injection for an agent that reads untrusted documents and can call tools. [Likely]
    - Which defences are ML classifiers, and which must be deterministic policy?
    - How do you red-team it continuously as attacks change?
60. Design a way to stop a code-generation model from reproducing private-repository code it was trained on. (Meta MLE phone screen) [Reported-2nd] [14]
    - Recast: how do you stop a model fine-tuned on several tenants' documents from reproducing one tenant's data for another? [Likely]
61. Design a PII detection and redaction service that sits between enterprise data and third-party LLM APIs. [Likely]
    - How do you keep redaction from destroying the information the LLM needs to do the task?

F. LLM and agent evaluation platforms

F1. Evaluation platform for LLM, RAG and agent features

Tag basis: [Bank] [1], [22]; [Reported-2nd] [12], [19].



UiPath recast: one evaluation platform for every agent, Document Understanding model and Autopilot feature, which product teams use to gate releases and customers use to check their own agents before deployment.

62. Design an evaluation system for a RAG product covering retrieval quality and answer faithfulness. [Bank] [1]
    - How do you build the golden set, and how do you keep it from going stale?
63. Design an LLM-as-judge evaluation. What are its known biases, and how do you correct for them? (Labelled Perplexity) [Bank] [22]
    - How do you show that your judge agrees with human raters, and how do you track drift?
64. How would you evaluate an agent, as opposed to a single model response? (Labelled Moonshot AI, Zhipu AI, Scale AI, Cognition) [Bank] [22]
65. Design an evaluation framework that checks whether a text-to-image system renders a requested brand name correctly. (Amazon MLE loop; reports 2026 Q1-Q3) [Reported-2nd] [12]
    - Recast: how would you check that a DU model or an agent reproduces exact strings (vendor names, IBANs, amounts) without silent edits? [Likely]
66. Evaluate agents that generate or optimise GPU-kernel code: check candidates against a trusted reference on test cases and pick the acceptable one with the fewest operators. (AMD; last reported Sep 2026) [Reported-2nd] [19]
    - Recast: how would you verify that an agent-generated automation workflow is equivalent to a reference workflow before it runs on production systems? [Likely]
67. Design an offline-to-online evaluation pipeline for an LLM feature: offline evaluation, online A/B tests and regression testing before shipping. [Bank] [1]
68. How would you evaluate a meeting-summarisation feature before shipping it to a hundred million users? (Labelled Microsoft) [Bank] [22]
69. Design an evaluation platform for multi-step agents, where success depends on the final state of external systems, not on the text. [Likely]
    - How do you build sandboxed replicas of enterprise apps to score agent runs?
    - How do you give credit or blame to individual steps in a 30-step trajectory?
70. Design a platform that lets enterprise customers evaluate their own agents on their own data without their data leaving their tenant. [Likely]
    - What evaluation data can you share across tenants, and what must never leave its tenant?

F2. Monitoring LLM systems in production

Tag basis: [Bank] [1]; [Reported-2nd] [21].

```
requests --> [ ? tracing (per step) ] --> [ ? online metrics + sampled judging ]
                                                          |
                        [ ? drift / cost / latency alerts ] --> on-call
                                                          |
                        [ ? triage --> new eval cases ] --> eval set (F1)
```

UiPath recast: observability for agent runs in Orchestrator, with traces that join LLM calls, robot steps and human approvals.

71. Design production monitoring for an LLM application: hallucination rate, retrieval hit rate, latency, cost and user feedback. [Bank] [1]
    - Which metrics can you compute on every request, and which only on a sample?
72. A chatbot's accuracy dropped from 95% to 80% in six weeks. What do you do? (Databricks MLE) [Reported-2nd] [21]
    - Recast: the drop coincides with a silent foundation-model update by the provider. How would your system have caught it on day one? [Likely]
73. Design tracing and root-cause analysis for agent failures that span LLM calls, robot actions and human approvals. [Likely]
    - How do you turn a production failure into a regression test automatically?

G. Serving, cost, data and platform (cross-cutting)

G1. Model routing, gateways and cost control

Tag basis: [Bank] [1], [22].

```
request --> [ ? difficulty / risk classifier ] --> small model --+
                        |                                          +--> [ ? cache ] --> response
                        +----------------------> large model --+
                                                               |
                              [ ? cost + quality feedback --> router retrain ]
```

UiPath recast: a gateway that serves many tenants' agents across several LLM providers, with per-tenant cost caps and data-residency rules.

74. Design a routing layer that sends simple requests to small models and hard ones to large models. [Bank] [1]
    - How do you train the router without labels for "this request needed the large model"?
75. Design the routing layer that decides whether a request is handled on-device or in the cloud. (Labelled Apple) [Bank] [22]
    - Recast: decide whether a step runs in a local robot model, a customer-hosted model or a cloud LLM, given data-residency rules. [Likely]
76. Should you self-host or call an API for a given LLM workload? Design the decision and the serving stack for each option. [Bank] [1]
77. Design an LLM gateway: routing across providers, failover, caching, budgets and rate limits. (Labelled Perplexity, Palantir) [Bank] [22]
    - How do you degrade gracefully when one provider has an outage? [1]
    - How do you attribute cost to tenants and enforce data-residency per tenant? [Likely]

78. Design a multi-tenant inference platform that serves many foundation models to thousands of customers. (Labelled Amazon) [Bank] [22]

G2. Document-to-structure pipelines and batch generation

Tag basis: [Bank] [1], [22]; [Reported-2nd] [17].

```
uploaded / archived docs --> [ ? batch + priority scheduler ] --> [ ? layout + OCR / VLM workers ]
                                                                          |
                                                  [ ? structure / field output + confidence ]
                                                                          |
                                       [ ? safety + sampled QA ] --> outputs + audit --> [ ? HITL fixes ]
```

UiPath recast: Document Understanding and IXP at scale, such as overnight extraction over millions of archived documents during a customer's migration. Chapter 08 covers the PDF-tagging version end to end.

79. Design a production service that converts uploaded PDF files into Markdown. (Mistral AI; last reported Apr 2026) [Reported-2nd] [17]
    - How do you handle reading order, tables and multi-column layouts, and how do you measure them?
    - How do you price and schedule a 2,000-page upload against many 2-page ones?
80. Design a document-intelligence pipeline that takes scanned PDFs in and returns structured fields, at 10M documents. (Labelled Palantir) [Bank] [22]
81. Design content generation or summarisation at scale, with batch processing and safety filtering. [Bank] [1]
    - How do you estimate cost and throughput before you run it?

G3. Training-data quality and model-building pipelines

Tag basis: [Reported-2nd] [11], [21]; [Bank] [23].

```
raw data (OCR text, logs, user tasks) --> [ ? filters: dedup, quality, PII, contamination ]
                                                          |
                                  [ ? labelling / synthetic data ] --> [ ? train + eval ]
                                                          |
                              [ ? data-quality dashboards + audits ] <------+
```

UiPath recast: build training sets for DU, Communications Mining and agent models from customer corrections in Action Center, under tenant consent and data-use rules.

82. Design a pipeline to clean and filter OCR-extracted text for language-model training. (Microsoft MLE loop; reports 2026 Q1-Q3) [Reported-2nd] [11]
    - Which quality signals can you compute without a human label?
83. Design a quality check for an LLM training dataset of spreadsheet-copilot tasks. (Microsoft MLE loop; reports 2026 Q1-Q3) [Reported-2nd] [11]
    - Recast: quality-check a dataset of (natural-language request, generated workflow) pairs for an Autopilot workflow generator. [Likely]
84. Design a video vision-language model from data collection through evaluation. (Microsoft MLE loop; reports 2026 Q1-Q3) [Reported-2nd] [11]
    - Recast: a screen-recording model that turns a user's recorded task into an automation. [Likely]

85. A platform logs billions of assistant requests a month. How would you use this data to improve its speech recognition? (Apple, AI role) [Reported-2nd] [21]
    - Recast: how would you use millions of human validations in Action Center to improve DU models without breaching tenant boundaries? [Likely]
86. For a new domain, would you fine-tune or use a complex prompting strategy? Design the decision process. (Anthropic MLE) [Reported-2nd] [21]
87. Roll out a new embedding model over an index holding billions of vectors. (Labelled Glean) [Bank] [23]
    - How do you re-embed without downtime, and how do you compare old and new indices online?

H. Classic NLP and vision prompts that recast directly

H1. Text classification, language identification and entity linking

Tag basis: [Bank] [1], [8].

```
text --> [ ? normalise + tokenise ] --> [ ? encoder ] --> [ ? task head ] --> label / entity
labels (sparse, noisy) --> [ ? active learning + weak supervision ] ---^
```

UiPath recast: Communications Mining taxonomies (intents and fields defined per customer) and entity linking to master data such as vendor IDs and customer accounts.

88. Design a system to predict the language a text is written in. [Bank] [8], [1]
89. Design a named-entity linking system. [Bank] [1]
    - Recast: how would you link an extracted vendor name on an invoice to the right record in a customer's ERP? [Likely]
90. How would you map nicknames (Pete, Andy, Nick, Rob) to real names? [Bank] [8]
    - Recast: how would you resolve the same customer across email, CRM and invoice records? [Likely]
91. Design a text classifier where each enterprise customer defines its own label taxonomy with only a few dozen examples per label. [Likely]
    - Fine-tuned small model, LLM prompting or embeddings with a light head: how do you choose for each tenant?

H2. OCR, object detection and image search

Tag basis: [Bank] [1], [5].

```
image / screenshot --> [ ? detector (regions / UI elements) ] --> [ ? recogniser (OCR) ]
                                                                          |
                                    [ ? layout / structure model ] --> fields | elements
```

UiPath recast: Document Understanding (scanned documents to fields) and UI-automation computer vision (screenshots to clickable elements for robots and computer-use agents).

92. Design an OCR / text recognition system. [Bank] [1]
93. Design an object detection system. [Bank] [5]

    - Recast: how would you detect and classify UI elements in screenshots of remote-desktop sessions (such as Citrix) where no DOM is available? [Likely]
94. Design an image search system, or a multimodal (text plus image) search system. [Bank] [1], [5]
    - Recast: how would you find "the same screen" across app versions to make robot selectors robust? [Likely]
95. Design an end-to-end document extraction system for invoices from thousands of vendors with different layouts, where an extraction error has a financial cost. [Likely]
    - Where do you place confidence thresholds and human validation?
    - How do validated corrections flow back into training without one tenant's data leaking to another?

H3. Prediction and estimation prompts

Tag basis: [Bank] [1], [7], [8]; [Reported-2nd] [14].

```
historical events + context --> [ ? features ] --> [ ? regressor / survival model ] --> estimate
                                                                                          |
                                                [ ? calibration + uncertainty ] ---+
```

UiPath recast: predict how long a process instance will take, or when a case will breach its SLA, from process-mining event logs.

96. Design a delivery-time estimation system. [Bank] [1], [7]
    - Recast: how would you predict case completion time and SLA breach risk from process-mining data? [Likely]
97. For a streaming service, predict whether a user stopped watching a show from fatigue or only to take a break. [Bank] [8]
    - Recast: how would you predict when a customer will stop using an automation or churn from the platform? [Likely]
98. Design an ads ranking system and explain how you calibrate predicted click and conversion probabilities. (Meta MLE loop) [Reported-2nd] [14]
    - What transfers from CTR calibration to calibrating the confidence of a DU extraction or an agent's "I am done" signal? [Likely]

I. Principal-level stress probes (apply to any prompt above)

These are not standalone prompts. Use them after any family above to check your answer against the principal bar. All are [Likely].

99. Which two decisions in your design are hardest to reverse, and what evidence would you gather before committing to them? [Likely]
100. Where in this design should ML stop and deterministic logic (rules, robots, workflows) take over? [Likely]
101. Estimate cost per 1,000 requests and the main cost driver. What would you cut first if the budget were halved? [Likely]

102. How does the design change when every customer is a separate tenant with its own data, labels, taxonomy and compliance regime? [Likely]
103. What would you build first in 90 days, what would you measure, and what would make you stop? [Likely]
104. Which parts of this system would you turn into a shared platform for other teams, and which should stay product-specific? [Likely]
105. How would you explain this system's failure modes to a regulated customer's auditor? [Likely]
106. If a competitor shipped a frontier-model-only version of this system tomorrow, which parts of your design would still be an advantage? [Likely]

Coverage summary

Topic family	Questions	[Reported-2nd]	[Bank]	[Likely]	[Reported]
A. Search and ranking	1-17	5	8	4	0
B. Recommendation	18-26	3	4	2	0
C. Anomaly and fraud detection	27-33	1	1	5	0
D. RAG enterprise assistants and agents	34-50	5	9	3	0
E. Content moderation and safety	51-61	5	3	3	0
F. LLM and agent evaluation platforms	62-73	3	6	3	0
G. Serving, cost, data and platform	74-87	6	8	0	0
H. Classic NLP and vision prompts that recast directly	88-98	1	8	2	0
I. Principal-level stress probes	99-106	0	0	8	0
Total	**106**	**29**	**47**	**30**	**0**

References

1. "ML System Design" (9-step formula; Section 2 sample questions including GenAI/LLM systems; Section 3 GenAI/LLM system design topics), alirezadir/Machine-Learning-Interviews, GitHub, accessed October 2026. https://github.com/alirezadir/Machine-Learning-Interviews/blob/main/src/MLSD/ml-system-design.md
2. "ML System Design Template" (9 steps), alirezadir/Machine-Learning-Interviews, GitHub, accessed October 2026. https://github.com/alirezadir/Machine-Learning-Interviews/blob/main/src/MLSD/mlsd-template.md
3. "Search System" (e-commerce search design, 50M DAU, clarifying questions), alirezadir/Machine-Learning-Interviews, GitHub, accessed October 2026. https://github.com/alirezadir/Machine-Learning-Interviews/blob/main/src/MLSD/mlsd-search.md
4. "Harmful Content Detection" design (500M posts a day, about 10K labelled examples, clarifying questions), alirezadir/Machine-Learning-Interviews, GitHub, accessed October 2026. https://github.com/alirezadir/Machine-Learning-Interviews/blob/main/src/MLSD/mlsd-harmful-content.md

5. MLSD case-study directory (files for video and event recommendation, typeahead, newsfeed, PYMK, image search, object detection, ads ranking, harmful content), alirezadir/Machine-Learning-Interviews, GitHub, accessed October 2026. https://github.com/alirezadir/Machine-Learning-Interviews/tree/main/src/MLSD
6. "ML Companies" reading list, alirezadir/Machine-Learning-Interviews, GitHub, accessed 2026. https://github.com/alirezadir/Machine-Learning-Interviews/blob/main/src/MLSD/ml-companies.md
7. "machine-learning-interview" (ML design section: YouTube recommendation, LinkedIn feed ranking, ad click prediction, delivery time, Airbnb search ranking), khangich, GitHub, accessed October 2026. https://github.com/khangich/machine-learning-interview
8. Chip Huyen, "Machine Learning Systems Design: Exercises" (27 open-ended design questions), chiphuyen/machine-learning-systems-design, GitHub, 2019, accessed October 2026. https://github.com/chiphuyen/machine-learning-systems-design/blob/master/content/exercises.md
9. Eugene Yan, "applied-ml: Papers and tech blogs by companies sharing their work on data science and machine learning in production" (source of the peer-anchor titles; all 13 cited titles confirmed), GitHub, accessed October 2026. https://github.com/eugeneyan/applied-ml
10. alirezadir, "Machine-Learning-Interviews" (repository overview; notes on FAANG versus startup interviews), GitHub, accessed October 2026. https://github.com/alirezadir/Machine-Learning-Interviews
11. TrueInterview (mirrored), "Microsoft Machine Learning Engineer interview process" (sample ML system design questions; "candidate-reported interview experiences, primarily from 2026 Q1 through 2026 Q3"), kevin-2023-code/Tech-Interview-Questions, GitHub, accessed October 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/microsoft/interview-process-machine-learning-engineer.md
12. TrueInterview (mirrored), "Amazon Machine Learning Engineer interview process" (based on 20 candidate-reported experiences, 2026 Q1-Q3), kevin-2023-code/Tech-Interview-Questions, GitHub, accessed October 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/amazon/interview-process-machine-learning-engineer.md
13. TrueInterview (mirrored), "Google Machine Learning Engineer interview process" (based on 13 candidate-reported experiences, 2025 Q4-2026 Q3), kevin-2023-code/Tech-Interview-Questions, GitHub, accessed October 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/google/interview-process-machine-learning-engineer.md
14. TrueInterview (mirrored), "Meta Machine Learning Engineer interview process" (based on 25 candidate-reported experiences, 2025 Q4-2026 Q3), kevin-2023-code/Tech-Interview-Questions, GitHub, accessed October 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/meta/interview-process-machine-learning-engineer.md
15. TrueInterview (mirrored), "Design an Account Takeover Detection System" (asked at Stripe, Affirm, Robinhood, Roblox, Uber; last reported Oct 2025), kevin-2023-code/Tech-Interview-Questions, GitHub, accessed October 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/questions/system-design/account-takeover-prediction-system/README.md

16. TrueInterview (mirrored), "Redesign a spam classifier for production" (asked at Indeed; last reported Aug 2026), kevin-2023-code/Tech-Interview-Questions, GitHub, accessed October 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/questions/system-design/redesign-a-spam-classifier-for-production/README.md
17. TrueInterview (mirrored), "Design a PDF-to-Markdown inference API" (asked at Mistral AI; last reported Apr 2026), kevin-2023-code/Tech-Interview-Questions, GitHub, accessed October 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/questions/system-design/design-a-pdf-to-markdown-inference-api/README.md
18. TrueInterview (mirrored), "Rank homepage modules for an e-commerce product" (asked at Wayfair; last reported Sep 2026), kevin-2023-code/Tech-Interview-Questions, GitHub, accessed October 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/questions/system-design/rank-homepage-modules-for-an-e-commerce-product/README.md
19. TrueInterview (mirrored), "Evaluate agents that generate or optimize kernels and compiler code" (asked at AMD; last reported Sep 2026), kevin-2023-code/Tech-Interview-Questions, GitHub, accessed October 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/questions/system-design/evaluate-agents-that-generate-or-optimize-kernels-and-compiler-code/README.md
20. TrueInterview (mirrored), Databricks company page ("Stock Trading Agent System Design", reported Apr 2026), kevin-2023-code/Tech-Interview-Questions, GitHub, accessed October 2026. https://github.com/kevin-2023-code/Tech-Interview-Questions/blob/main/companies/databricks/README.md
21. Alexey Grigorev, "AI Engineering Field Guide: discussion threads" (curated Reddit, TeamBlind and X interview threads with links to the original posts; the original posts were not opened in this run), alexeygrigorev/ai-engineering-field-guide, GitHub, accessed October 2026. https://github.com/alexeygrigorev/ai-engineering-field-guide/blob/main/interview/data/sources/discussion-threads.md
22. pallavi-shekhar, "AI engineering interview questions, company-wise" (says it is "compiled from publicly reported interview experiences"; the "Asked at" labels are the maintainer's, with no per-question source), GitHub, accessed October 2026. https://github.com/pallavi-shekhar/ai-engineering-interview-questions-company-wise
23. ombharatiya, "AI Companies Interview Questions" (says it is compiled from candidate reports, blogs and guides; only some items are labelled as reported), FAANG-Coding-Interview-Questions, GitHub, accessed October 2026. https://github.com/ombharatiya/FAANG-Coding-Interview-Questions/blob/main/AI-Companies-Interview-Questions.md
24. Subrata Roy, "Interview experience: Amazon Applied Scientist 2 (ML + DSA)", LinkedIn post archived in snknitin/LLMwiki (bookmarked 2026-03-20), GitHub, accessed October 2026. https://github.com/snknitin/LLMwiki/blob/main/Bookmarks/LinkedIn/2026-03-20%20-%20subrata-roy%20-%20interview-experience-amazon-applied-scientist-2-ml-dsa%20-%20743936.md

Reference [6] was read for context, but no question cites it.

10. ML Platform and MLOps System Design: Data Flywheel, Model Lifecycle, Capacity and Rollout (with Diagrams)

How to read this chapter. The rest of the pack raises MLOps only as follow-up probes. This chapter is the dedicated design chapter for it. It covers training-data pipelines and labelling operations, feature and embedding stores, model registry and versioning (AI Center-style), CI/CD for models and prompts, drift and data-quality monitoring for document and agent models, per-tenant fine-tuning and adapter management, GPU capacity planning, on-prem, air-gapped and multi-region rollout, and rollback and incident response. Each section opens with a plain-text diagram, then gives numbered questions with follow-up probes. It has questions only, with no answers. The diagrams are whiteboard scaffolds that show the parts of the system the questions refer to. They are not model answers. They are plain text so they render the same in Markdown and in a PDF export.

Tagging system. Every question carries one of these tags:

- **[Reported] [n]**: a candidate report or interview-experience post links this question to UiPath. The number points to the References list.
- **[Likely]**: an inference from the role (Principal Applied Scientist), the level and UiPath's products: AI Center-style model hosting, Document Understanding (DU Modern), IXP and Communications Mining, Context Grounding, the LLM Gateway and AI Trust Layer, agents and Action Center, and Automation Suite for on-prem customers. No candidate has reported it.
- **[Likely] [n]**: as [Likely], but source [n] confirms the *premise* of the question, such as a product behaviour (for example, published project tags and versions, or AI Trust Layer model policy). It does **not** mean a candidate was asked it.
- Indented sub-bullets are follow-up probes an interviewer may use to push a strong first answer toward the principal bar.

"Tag" in this chapter. UiPath uses "tag" for a published version alias of a Document Understanding or IXP project. For non-pretrained projects a caller must pass exactly one of `version` or `tag` (passing both is rejected), and an unknown tag raises an error that lists the project's available tags [2]. Several questions below test how that tag-as-alias model works for release, rollback and per-tenant pinning. PDF *structure* tags (accessibility tagging) are covered in chapter 08.

Research limits (please read). Neither the first pass nor the adversarial review (both 14 October 2026) could run a web search: the workflow's shared budget of 200 searches was already used up. Direct fetches of glassdoor.com, interviewquery.com and docs.uipath.com failed DNS lookup, and earlier passes

found teamblind.com, reddit.com, leetcode.com, prepfully.com and uipath.com unreachable. Only GitHub-hosted files could be read. The chapter therefore rests on UiPath's public SDK source and documentation, UiPath's IXP CLI and a general ML system design guide [1]-[15]. Those sources confirm product behaviour and general round format, not interview content. So **no question below is tagged [Reported]**, and the chapter does not invent reports. A later run with search budget should check Glassdoor, Blind, LeetCode Discuss, 1point3acres, Prepfully and Interview Query for UiPath-specific MLOps prompts, and upgrade tags where sources support them.

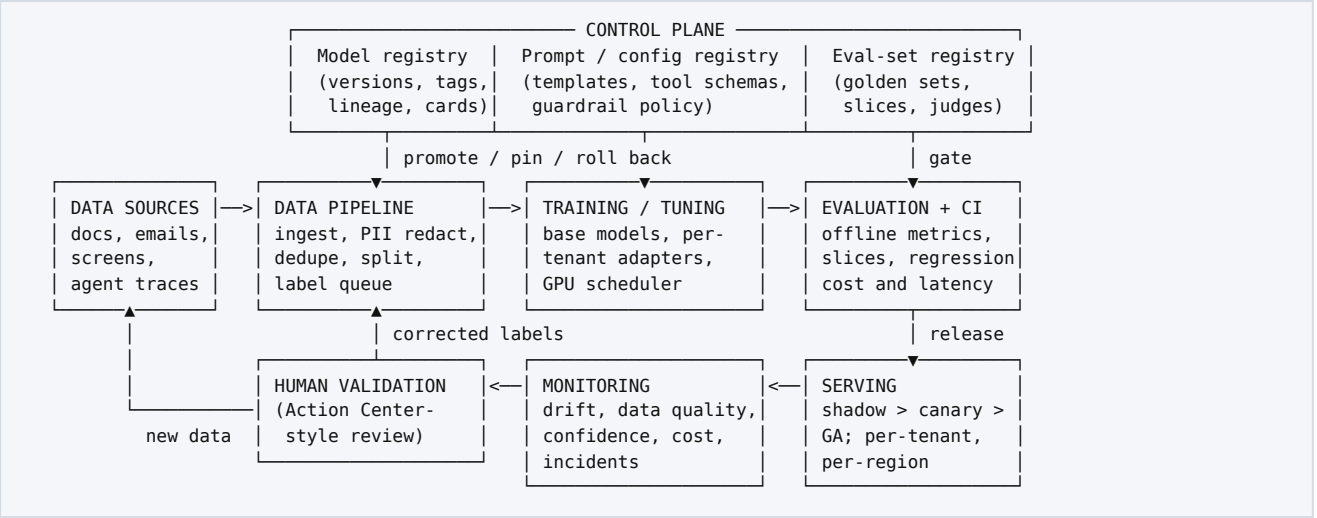
Review pass (14 October 2026). An adversarial review re-fetched all 7 original GitHub sources and confirmed each claim. It corrected the `version`/`tag` wording (passing both is rejected, rather than the tag being ignored), and noted that searching an index during ingestion is opt-in. It removed two references that could not be reached or verified (the docs.uipath.com portal and the UiPath careers page; no question cited them) and two duplicate questions: right-to-deletion for adapters (covered by the consent-revocation probe in section A) and funding evaluation capacity (covered by framing question 5). It reworded one probe that duplicated the vendor-drift monitoring question. It then read 8 more official UiPath GitHub documents [8]-[15] and added 28 principal-level [Likely] questions on gaps: escalation outcomes as labels, redacted traces, heterogeneous tenant schemas, ephemeral indexes and index tiering, release artefacts and dependency locking, identity and lineage, semantic versioning for models, trajectory gates, CI credentials, guardrail and ring-based rollout, cost gates, eval contamination, trace sampling, guardrail-hit monitoring, customer-owned taxonomies, UI drift, adapter poisoning, prefix caching, admission control, per-environment resolution, telemetry opt-out bias, CVE response, rollback without data rollback, build-or-buy and agents that do MLOps.

Format notes for this round

Item	Notes	Status
Where it appears	Probably not as its own round. More likely it is the second half of the ML system design round ("now take it to production"), or a deep dive during research-presentation Q&A ("how did you deploy, monitor and retrain it?"). See chapter 01	Inferred
Duration	45-60 minutes when it is a full design round; 15-25 minutes when it is the production section of another round	Inferred
Interviewer	Principal or staff scientist, ML platform architect, or an engineering director who owns the AI platform	Inferred
General round shape	Open-ended, no single right answer. A general guide puts serving, online testing (A/B, shadow, canary) and scaling, monitoring and retraining as the last three steps of a nine-step flow. For 2026 GenAI rounds, it says interviewers weigh cost, latency, guardrails and monitoring more than the architecture diagram, and that "evaluation methodology is the new system design" [1]	Sourced (general, not UiPath-specific)
Principal bar	Owns the lifecycle, not just the model. Sets SLOs and release gates. Designs for thousands of tenants with different data, regions and deployment models (cloud, dedicated, on-prem, air-gapped). Puts numbers on GPU, cost and blast radius. Makes the data flywheel (human	Inferred

Item	Notes	Status
	validation leading to retraining) a deliberate, governed system. Names who owns each failure	
Common failure modes	One global model with no tenant story. "Retrain weekly" with no trigger or gate. Monitoring only latency and errors. No rollback plan for prompts or adapters. Ignoring air-gapped customers. No cost model	Inferred

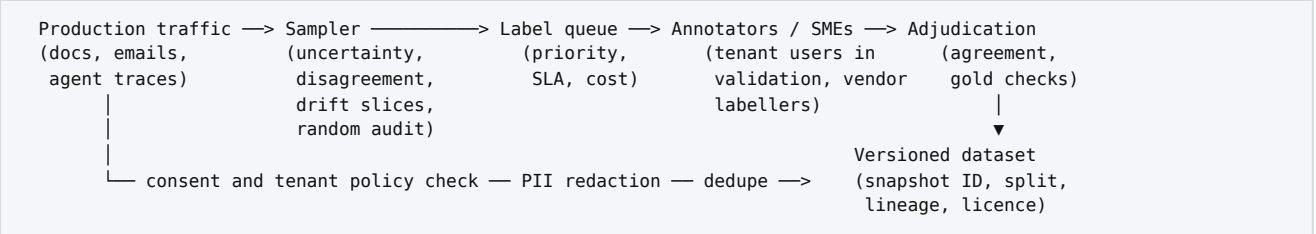
Canvas: the end-to-end model lifecycle (whiteboard scaffold)



Framing questions

1. [Likely] Design the ML platform that lets UiPath ship, monitor and retrain document, communications and agent models for thousands of enterprise tenants. Start from the SLOs, not the components.
    - Which three SLOs would you put on the board first, and who in the company owns each?
    - Which parts are shared across tenants, and which must be isolated per tenant?
2. [Likely] Walk me through one model's life, from the first labelled document to its retirement. Where do most production failures come from?
    - Which step is most often skipped in practice, and how would your platform make skipping it impossible?
3. [Likely] What does "MLOps" mean differently for a fine-tuned extractor, a prompt-based LLM agent and a retrieval index? Draw the lifecycle of each next to the others.
    - Which artefacts need versioning in each case: weights, prompts, tool schemas, indexes, eval sets, guardrail policy?
4. [Likely] You join and find each team has its own training scripts, buckets and deployment path. What platform would you build first, and what would you deliberately not build?
    - How do you get adoption without a mandate?
5. [Likely] How would you estimate the business value of a better MLOps platform, so that leadership funds it over a new model feature?
    - Which metric would you track for a year to show it worked?

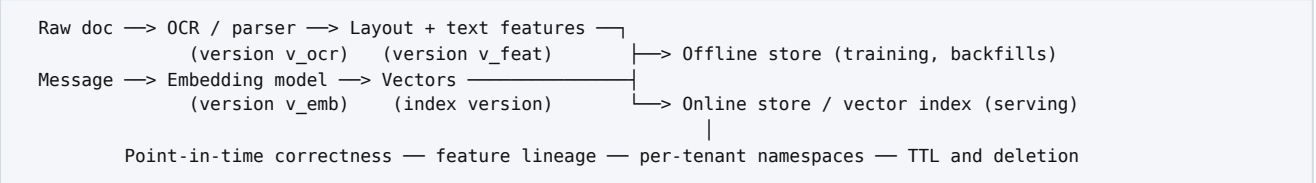
A. Training-data pipelines and labelling operations



6. [Likely] [2] Design the data flywheel in which human corrections from document validation tasks become training data. Validation tasks block until a person completes them [2]. How do you turn those completed tasks into clean labels?
    - A validator accepts a wrong extraction because they are rushed. How do you detect low-quality corrections?
    - Which corrections can be used for a shared model, and which only for that tenant's model?
7. [Likely] How do you decide which production samples to send for labelling when the labelling budget is 1% of traffic?
    - How do you mix uncertainty sampling, drift-slice sampling and a random audit sample, and why do you need the random part?
    - How do you avoid a feedback loop in which the model only ever gets labels on cases it already finds hard?
8. [Likely] Design a labelling operation for a new document type with no existing labels, where the only experts are the customer's accounts-payable staff.
    - How would you use an LLM to pre-label, and how do you stop annotators from rubber-stamping it?
    - How do you measure and report inter-annotator agreement for key-value fields and tables?
9. [Likely] [6] UiPath's `re` CLI, described as the command-line interface for UiPath IXP (formerly Communications Mining), can download raw messages from sources and datasets together with human-applied annotations [6]. How would you build a reproducible training snapshot from a live, continuously annotated dataset?
    - Annotations change after a snapshot is taken. How do you version them so an old model can still be reproduced?
10. [Likely] How do you split train, validation and test data for documents so that the test score is honest?
    - Same vendor's invoices in train and test: is that leakage? How do you split by vendor, template or time?
    - How do you keep a frozen test set that never leaks into labelling priorities?
11. [Likely] A taxonomy change adds two new fields and merges two classes. How does the data pipeline migrate old labels?
    - Which models must be retrained, and which only re-evaluated?
12. [Likely] Design PII handling in the training pipeline for invoices, IDs and emails, where some customers forbid any use of their data for training.
    - Where does consent live, and how is it enforced at read time rather than by convention?
    - A customer revokes consent. How do you find and remove their data from datasets, adapters and shared models?

13. [Likely] How would you generate synthetic documents to cover rare layouts, and how do you stop the model from learning synthetic artefacts?
    - What share of synthetic data would you allow in an eval set, if any?
14. [Likely] How do you turn agent traces (tool calls, human escalations, final outcomes) into training or evaluation data?
    - The outcome arrives weeks later (an invoice is paid or disputed). How do you join delayed labels to the original run?
15. [Likely] [14] Coded agents suspend on an interrupt such as an Action Center task or escalation, and resume when a person completes it [14]. Design the logging so that every escalation outcome becomes a usable label linked to the exact model, prompt and trace span that caused it.
    - Escalations are routed by priority and to specific users or groups. How does that routing bias the labels you collect, and how do you correct for it?
16. [Likely] [9] Agent traces can redact inputs and outputs (`hide_input`, `hide_output`) or transform them with processors before recording [9]. How do you build evaluation and training sets from traces when the most informative fields are the ones customers redact?
    - Who decides the redaction policy, and how do you stop a later policy change from silently changing what your eval set contains?
17. [Likely] What is the unit cost of a label in your design, and how would you lower it by 5x without losing quality?
    - When is a cheaper, noisier label better than a costly, clean one?
18. [Likely] How would you detect and handle duplicate and near-duplicate documents across tenants without moving tenant data across boundaries?
19. [Likely] Each tenant extends the shared document schema with its own custom fields and names. How do you train and evaluate a shared extractor over heterogeneous, tenant-defined schemas?
    - How do you map "Invoice No.", "Bill number" and "Ref" across tenants without a human curating every mapping?
20. [Likely] Who owns label quality: the scientist, the labelling vendor, the product team or the customer? How would you set that up organisationally?

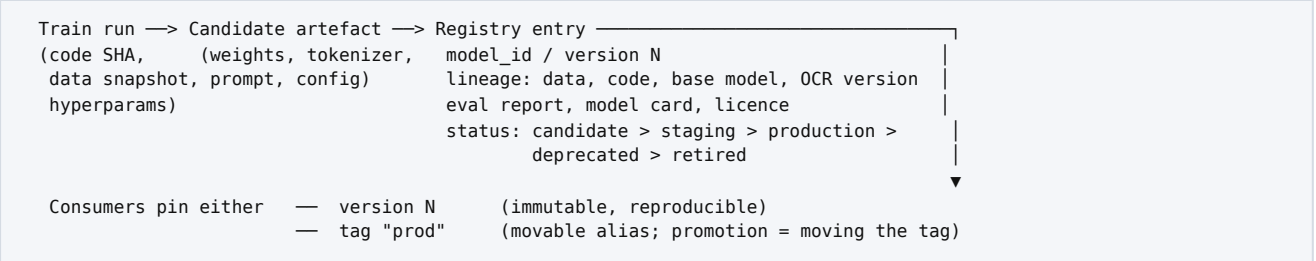
B. Feature stores and embedding stores



21. [Likely] Do document and agent models need a feature store? Argue for and against, then design the smallest one that solves the real problems.
    - What does training-serving skew look like when OCR or the parser is the "feature pipeline"?
22. [Likely] The OCR engine is upgraded. Every downstream extractor sees different tokens and boxes. How does the platform handle this?
    - Do you retrain, re-evaluate or pin the old OCR per model version? What does the registry record?

23. [Likely] [4] Design an embedding store for retrieval over tenant knowledge sources. Context Grounding indexes can be scoped to folders, built from buckets and other connectors, and re-ingested on a schedule; an ingestion request that overlaps a running one is rejected, and searching during ingestion is an explicit opt-in [4]. How do you version and refresh such indexes?
    - You change the embedding model. How do you re-embed millions of chunks per tenant without downtime?
    - How do you serve queries while ingestion is still running, and what consistency do you promise users?
24. [Likely] How do you test that a new embedding model is better for retrieval inside an agent, not just on a public benchmark?
    - Which offline retrieval metrics, and which end-to-end agent metrics, would gate the switch?
25. [Likely] How do you make sure a deleted document disappears from the vector index, caches and any derived summaries within a stated time?
    - How would you prove that to an auditor?
26. [Likely] Design point-in-time-correct features for an agent that predicts whether an invoice exception will be approved.
    - Which features would leak the label if computed at training time from current data?
27. [Likely] One shared vector index with tenant filters, or one index per tenant? Walk through cost, isolation, noisy neighbours and deletion.
    - At what tenant size does your answer change?
28. [Likely] How do you monitor the health of an embedding index: staleness, coverage and duplicate rate?
29. [Likely] [14] Agents can create short-lived (ephemeral) Context Grounding indexes for Deep RAG and batch-transform jobs, and suspend until the job completes [14]. Design the lifecycle for ephemeral indexes: creation, retention, deletion, cost attribution and monitoring.
    - A long-running agent is suspended for three days waiting for a human. Does its ephemeral index survive, and who pays for it?
30. [Likely] Retrieval indexes for 10,000 tenants cost more to store and keep warm than to query. Design hot, warm and cold tiers for vectors and the rules that move an index between them.
    - What query latency do you promise for a cold tenant, and how do you hide the reload from an agent mid-run?
31. [Likely] [4] Context Grounding supports encrypted indexes and a choice of extraction strategy at index creation [4]. How do these options change the re-index, migration and debugging story?

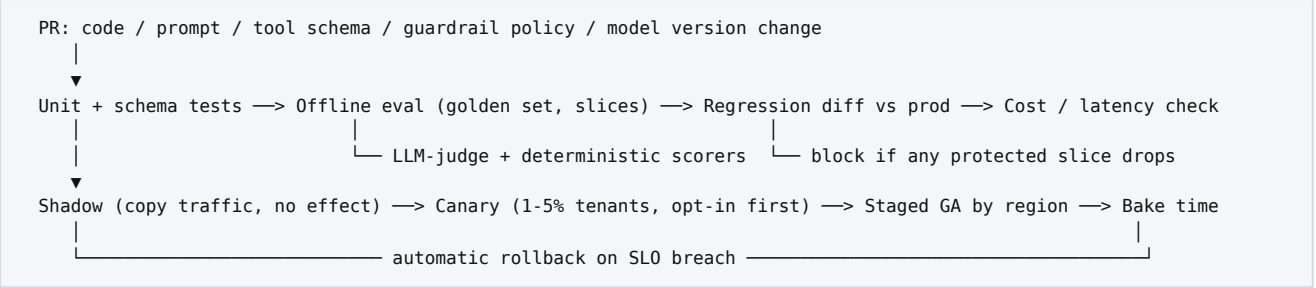
C. Model registry and versioning (AI Center-style)



32. [Likely] Design a model registry for UiPath-style ML packages and skills. What exactly is versioned, what is immutable, and what is a movable pointer?
    - What metadata must exist before a version may be promoted?
33. [Likely] [2] In the Documents service, callers of non-pretrained projects must pass exactly one of `version` or `tag` (passing both is rejected), and an unknown tag raises an error listing the project's available tags [2]. What are the trade-offs of tag-based versus version-based pinning for customer automations?
    - A customer's production robot pins tag "prod". You move the tag to a new version. What can break, and how do you warn them?
    - Which customers should pin an exact version, and how does the platform push them to upgrade?
34. [Likely] How do you make any production model exactly reproducible two years later for a regulated customer?
    - What do you store besides weights: data snapshot, code, container image, OCR version, random seeds?
35. [Likely] Design versioning for an LLM-based extractor where the "model" is a vendor model plus a prompt plus a schema plus a post-processor.
    - The vendor offers only an undated alias for one model. How do you protect customers from silent updates when you cannot pin a version?
36. [Likely] [3] LLM Gateway model IDs carry dated version suffixes, such as `gpt-4.1-mini-2025-04-14`, and which models a tenant can use depends on AI Trust Layer policy [3]. How would you manage the deprecation of a third-party model version that hundreds of customer agents depend on?
    - What migration tooling and evaluation do you give customers before the cut-over date?
37. [Likely] How should the registry represent a base model plus hundreds of per-tenant adapters?
    - What happens to the adapters when the base model is upgraded?
38. [Likely] Design the model card and lineage record for a document classifier, for both internal reviewers and customers' risk teams.
39. [Likely] How do you retire a model safely when you cannot see all of its callers (on-prem robots, long-running agents)?
    - How long do you keep serving a deprecated version, and who decides?
40. [Likely] How would a pipeline that retrains, evaluates and conditionally promotes a model (the "full pipeline" pattern) decide on its own whether to promote?
    - Which decisions would you never automate?
41. [Likely] How do you version evaluation sets and judges so that a score from last quarter is comparable with today's?
42. [Likely] [10] `uipath pack` builds a `.nupkg` and by default locks dependencies with `uv.lock`; with `--nolock` each run resolves the latest compatible versions [10]. For a

deployed agent, define the immutable release artefact. Which of code, lock file, prompt, model ID, tool schemas, guardrail policy and eval set are inside it, and which are referenced?
    - A locked dependency gets a security fix. How do you patch thousands of locked customer agents without breaking reproducibility?
43. [Likely] [10] Each coded project carries a stable GUID in `uipath.json`; changing it makes the project look like a new, unrelated one and breaks the link to what was published under the old ID [10]. Design the identity model that separates a model or agent's identity, its versions and its deployments across personal-workspace and tenant feeds.
    - How do you keep lineage when a customer forks a UiPath template agent and edits it?
44. [Likely] [11] UiPath's Python SDK follows semantic versioning with release candidates, a deprecation period of at least one minor release and migration guides for breaking changes [11]. Would you apply semantic versioning to models and prompts? Define what counts as a "breaking change" for an extractor or an agent.
    - Is a 2-point accuracy gain that changes 5% of outputs a patch, a minor or a major release?

D. CI/CD for models and prompts

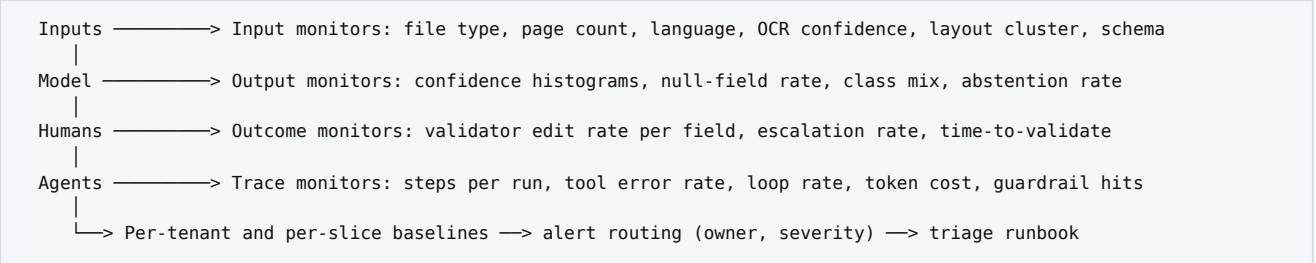


45. [Likely] [5] Design CI/CD for agent prompts and tools. UiPath's SDK runs eval sets with output-based, trajectory-based and custom evaluators through `uipath eval`, with JSON results and a custom run ID [5]. How would you turn that into a release gate?
    - What is the pass rule: mean score, per-slice floors, or no regression on any "must-pass" case?
    - How do you handle non-determinism so that the gate is not flaky?
46. [Likely] A prompt change improves the average score by 3 points and drops one customer's slice by 10. Do you ship? Who decides?
47. [Likely] How do you test a change to a tool schema or guardrail policy that is not itself a model change but changes agent behaviour?
48. [Likely] Design shadow deployment for a document extractor. What do you compare, and how do you label disagreements cheaply?
    - Shadowing doubles inference cost. How do you limit it?
49. [Likely] How do you run a canary when many tenants have little traffic and some have strict change windows?
    - Who gets the canary first: internal tenants, opt-in customers, or the largest tenants?

50. [Likely] [12] UiPath's LLM-judge evaluators return a 0-1 score with a written justification, from a configurable judge model run at temperature 0 by default [12]. How do you use such a judge in CI without the judge's own drift, or its deprecation, breaking your gates?
    - How often do you recalibrate the judge against humans, and what triggers recalibration?
51. [Likely] How long should the full pre-release evaluation take, and what would you cut to keep it under one hour?
52. [Likely] Design an A/B test for a new extractor where the outcome is "fields changed by the human validator".
    - The validator's behaviour changes with the model (automation bias). How does that bias the experiment?
53. [Likely] How do you release a change that must be coordinated across a model, a prompt and a robot workflow that customers own?
54. [Likely] What is your branching and environment model (dev, staging, prod) for prompts that business users can edit themselves?
    - How do you stop a customer's prompt edit from being overwritten by a platform release?
55. [Likely] What release metrics would you report to the VP each week to show that releases are getting safer and faster?
56. [Likely] [13] UiPath's tool-call-order evaluator can require an exact sequence (score 1 or 0) or give partial credit with a longest-common-subsequence score [13]. When is partial credit the wrong rule for a release gate? Design trajectory gates that encode hard business constraints, such as "approve before pay".
    - How do you combine hard trajectory constraints with soft quality scores into one go/no-go decision?
57. [Likely] [10] The CLI supports a non-interactive OAuth client-credentials flow for CI/CD, so a pipeline can run `uipath publish` and `uipath invoke` [10]. Design the promotion pipeline from a developer's personal workspace to a production tenant feed. Who holds which credentials, and how do you enforce separation of duties?
    - How do you prove to a regulated customer that the package in production is the one that passed the gate?
58. [Likely] [8] Guardrails can log, block or modify data, at tool, LLM or agent scope, before or after a call [8]. Design the rollout of a new guardrail policy across tenants without breaking working automations.
    - Would you start in log-only mode as a shadow? Which false-positive rate lets you switch to block, and per which slice?
59. [Likely] [8] Some UiPath AI features are enabled ring by ring behind feature flags (for example, Bring Your Own Guardrail is described as still rolling out) [8]. Design a ring-based rollout for model and prompt changes. How are rings defined, and which ML metrics must hold before you promote to the next ring?
    - Early rings are not representative of large regulated tenants. How do you correct for that?
60. [Likely] A model change keeps quality flat but raises tokens per run by 40% on long documents. Design a cost and latency gate that would have caught it, and say who may override it.
    - How do you set cost budgets per tenant tier and per product line?

61. [Likely] Your golden set leaks into prompts over time: engineers copy failing eval cases into few-shot examples. How do you detect and prevent eval contamination in a prompt CI system?

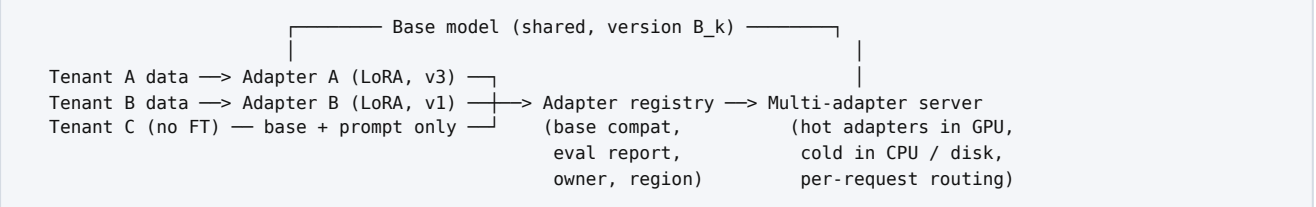
E. Drift and data-quality monitoring for document and agent models



62. [Likely] Design drift monitoring for a document extraction model across thousands of tenants when true labels arrive only for the small share of documents that humans validate.
    - Which label-free signals best predict an accuracy drop, and how did you check that they do?
    - How do you set per-tenant baselines for tenants with 50 documents a month?
63. [Likely] A large customer changes its invoice template and field accuracy drops for that tenant only. How does your system detect it, and how fast?
    - Is the right response a retrain, a tenant adapter, a rule, or routing to human validation?
64. [Likely] How do you tell data drift from a data-quality bug upstream, such as a scanner change, a broken OCR language pack or a truncated file?
65. [Likely] Design monitoring for an LLM agent where there is no single accuracy metric. What do you watch, and what pages a human at 3 a.m.?
    - Which agent metrics are leading indicators, and which are lagging?
66. [Likely] Validator edit rate goes down after a release. Is the model better, or are validators trusting it more? How do you find out?
67. [Likely] How do you monitor calibration of confidence scores over time, given that customers set straight-through-processing thresholds on them?
    - Calibration drifts. Do you recalibrate silently, or tell customers their thresholds now mean something else?
68. [Likely] How would you detect that a third-party LLM behind the gateway has changed behaviour, when the provider does not tell you?
69. [Likely] Design the alerting system so that thousands of tenant-level monitors do not cause alert fatigue.
    - How do you aggregate, deduplicate and route alerts to the right owner?
70. [Likely] How do you monitor retrieval quality in production for a RAG-based agent without labelled queries?
71. [Likely] What data-quality checks would you put at the very start of the training pipeline, and which ones block training?
72. [Likely] Over a year, which signals would tell you to retrain on a schedule, and which would justify event-triggered retraining instead?

73. [Likely] [9] Traces of agent runs are visible per Orchestrator job and in Maestro [9]. At millions of runs a day, design trace sampling and retention: what you keep in full, what you sample, and how tail-based sampling keeps the failures.
    - How do you bound trace storage cost per tenant while keeping enough data to debug a one-in-10,000 failure?
74. [Likely] [8] Guardrail block rate for PII doubles the day after a model release [8]. Is the new model leaking more PII, is the guardrail misfiring, or has customer data changed? Design the monitors that tell these apart.
    - Silent blocks show up as failed or escalated runs, not as errors. How do you count them as a quality regression?
75. [Likely] [6] In IXP and Communications Mining, business users keep annotating and the taxonomy changes weekly [6]. How do you monitor model quality when the label set and the model are both moving targets owned by the customer?
    - When validation scores fall, how do you tell a worse model from a harder taxonomy?
76. [Likely] A customer's ERP vendor ships a UI update and a computer-use agent starts failing on one screen. Design drift detection for UI and screen-understanding models, where the "data" is a third party's application.
    - Can you detect the change before the first failed run, for example from screenshots or selectors?

F. Per-tenant fine-tuning and adapter management



77. [Likely] When should a tenant get its own fine-tuned adapter, rather than prompt changes, retrieval or a shared model improvement? Design the decision rule.
    - What is the minimum data and the minimum expected gain for a tenant adapter?
78. [Likely] Design serving for 5,000 tenant adapters on a shared base model with a strict p95 latency SLO.
    - How do you decide which adapters stay in GPU memory? What happens on a cold load?
    - How do you batch requests that use different adapters?
79. [Likely] The base model is upgraded. 5,000 adapters were trained on the old base. What is your migration plan?
    - Which adapters do you retrain automatically, and how do you check that each retrained adapter is no worse for its tenant?
80. [Likely] How do you guarantee that tenant A's adapter, data and outputs can never be served to tenant B?
    - Which test would you add to CI to prove it?
81. [Likely] How do you evaluate a tenant adapter when the tenant's labelled test set has only 40 documents?

82. [Likely] A tenant fine-tunes on its own data and quality gets worse. Who is responsible, and what does the product do?
83. [Likely] How do you attribute GPU and storage cost of fine-tuning and adapter serving to tenants and turn that into pricing?
84. [Likely] Can improvements learned from one tenant's corrections flow into the shared base model? Design the consent, aggregation and privacy controls.
    - How would you test for memorisation of one tenant's data in the shared model?
85. [Likely] Design self-service fine-tuning for customers (upload, train, evaluate, deploy). Which guard rails stop a customer from deploying a bad model?
86. [Likely] A tenant's training data for an adapter could be poisoned, for example by an insider planting documents so that an agent approves certain payments. Design the controls on self-service fine-tuning data and the checks on the resulting adapter before it serves agents that take actions.

G. GPU training and inference capacity planning

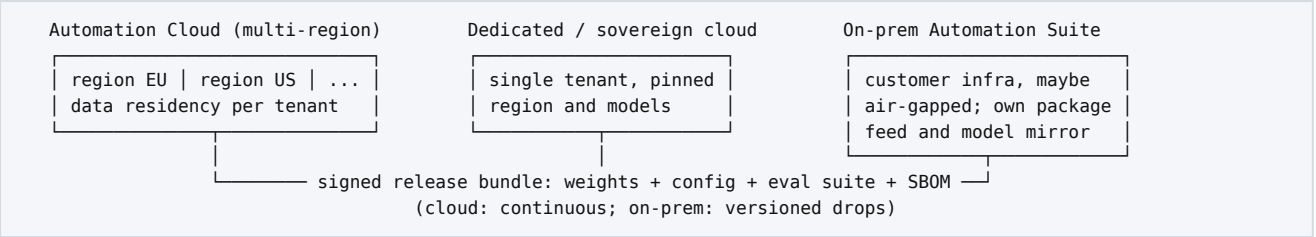
```
Demand ──> interactive (agents, validation UI)   p95 < 2 s     ── reserved GPUs, autoscale
       ──> near-real-time (queue-triggered docs)  minutes       ── shared pool, batching
       ──> batch (backfills, re-embedding, evals) hours         ── preemptible / off-peak
       ──> training and tuning                    days          ── scheduled, quota per team

Capacity = f(requests/s, tokens or pages per request, model size, batch size, utilisation target, headroom)
```

87. [Likely] Do a back-of-envelope estimate on the board: 20 million pages a month across tenants, a 10x peak in the last week of the quarter, a 2-second p95 target. How many GPUs, of which type, in how many regions?
    - What utilisation target would you plan for, and why not 90%?
88. [Likely] Interactive agent traffic and batch document backfills compete for the same GPUs. Design the scheduler and priority model.
    - How do you stop one tenant's backfill from hurting everyone's latency?
89. [Likely] When would you self-host an open-weights model instead of calling a third-party model through the gateway? Put numbers on the break-even.
    - How do quantization, distillation and speculative decoding change the numbers?
90. [Likely] Plan GPU capacity for training for the next year when the research roadmap is uncertain. How do you split reserved and on-demand capacity?
91. [Likely] How do you allocate scarce GPUs among teams (documents, agents, computer vision for UI automation)? Who decides, and on what basis?
92. [Likely] Design autoscaling for LLM inference, where cold start means loading tens of gigabytes of weights.
    - What do you keep warm, and what does it cost?
93. [Likely] How do you capacity-plan for an on-prem customer who has two GPUs, or none?
    - Which models must have a CPU-only path, and what accuracy do you give up?
94. [Likely] What GPU efficiency metrics would you report, and how would you find the biggest waste in a large training cluster?
95. [Likely] A new model is 4% more accurate and 3x more expensive to serve. How do you decide whether to ship it, and to whom?
96. [Likely] How do you reserve capacity for evaluation and re-embedding jobs so that releases are not blocked by production load?

97. [Likely] Agent prompts share long system prompts and tool schemas across runs. How does prefix (KV) caching change GPU capacity planning, and how do you design prompts and routing to maximise cache hits without mixing tenants?
    - How do you measure the cache hit rate, and what happens to capacity if a release changes the shared prefix?
98. [Likely] [3] LLM Gateway usage consumes Agent Units [3]. At quarter end, demand exceeds both GPU capacity and some tenants' unit budgets. Design admission control and graceful degradation: queueing, smaller models, lower reasoning effort or human routing.
    - Which tenants get priority, and how is that written into the contract and the scheduler?

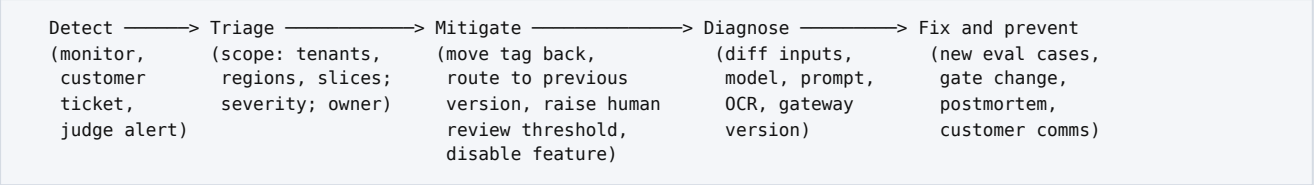
H. On-prem, air-gapped and multi-region rollout



99. [Likely] [7] Air-gapped Automation Suite customers must host their own package index even for Python dependencies [7]. Design how you ship model weights, guardrail models, embedding models and evaluation suites into such an environment and keep them updated.
    - How do you sign, verify and roll back an offline model bundle?
    - How do you support a customer who is three releases behind?
100. [Likely] How do you monitor model quality in an air-gapped deployment where no telemetry leaves the site?
    - What can the customer run locally to tell you whether a regression is the model or their data?
101. [Likely] [3] Some gateway models are not available in some regions, which can cause data-residency errors, and AI Trust Layer policy controls which models a tenant can use [3]. Design model routing across regions under data-residency rules.
    - A model exists only in the US. What do EU tenants get, and how do you keep behaviour consistent?
102. [Likely] How do you roll out a new model region by region, and what decides the order?
    - How do you keep evaluation results comparable when regions use different underlying model versions?
103. [Likely] Design a model release process that serves both the continuous cloud release train and quarterly on-prem releases from one codebase.
    - Which features do you allow to differ between cloud and on-prem, and how do you stop the two from drifting apart?
104. [Likely] How would you run an LLM-based agent on-prem when the customer cannot use any third-party model API?
    - What is the smallest model that keeps the product useful, and how do you prove it to the customer?

105. [Likely] How do you handle a security patch to an inference container across cloud regions, dedicated clouds and on-prem installs?
106. [Likely] A regulator asks which model version processed a given document in a given region on a given date. Design the audit trail that answers this in minutes.
107. [Likely] What is your disaster-recovery plan for the model registry and vector indexes in one region?
108. [Likely] [10] [8] The same agent package can run on Automation Cloud or on an Automation Suite instance with its own URL [10]. Built-in guardrails are backed by the UiPath Guardrails API, powered by Azure Content Safety [8]. Design how model endpoints, guardrails and judge models resolve per environment, and what replaces cloud-hosted guardrails in an air-gapped install.
    - How do you show an air-gapped customer that their local guardrails behave the same as the cloud ones?
109. [Likely] [15] The UiPath CLI collects usage telemetry by default, and users can opt out [15]. If your fleet-level quality and drift metrics come only from tenants that share telemetry, how biased are they, and how do you correct for that?
    - What minimal, privacy-preserving signal would you ask opted-out customers to share, and how would you justify it?
110. [Likely] [7] [10] Air-gapped customers mirror Python packages into their own index, and agent packages lock their dependencies [7] [10]. A critical CVE lands in a library inside hundreds of locked customer agents running on-prem. Design the response, from detection to verified patch.

I. Rollback and incident response for model regressions



111. [Likely] [2] A new extractor version is promoted by moving a published tag. Two hours later, three large tenants report wrong totals. Walk me through the first 60 minutes.
    - Rolling back is moving the tag back [2]. What can still go wrong: cached results, documents in flight, adapters trained on the new base?
    - What do you tell customers, and when?
112. [Likely] How do you roll back a prompt or agent configuration when agent runs are long-running and suspended waiting for a human?
    - Should a resumed run continue on the old version or the new one? Why?
113. [Likely] A third-party model provider changes behaviour and your agents start failing a guardrail at 5x the normal rate. You cannot roll back the provider. What are your options?
114. [Likely] Wrong extractions were sent straight through to a customer's ERP system for six hours before detection. Design the remediation: finding affected records, re-processing and customer communication.
    - What would have reduced the blast radius?
115. [Likely] Design the severity levels and on-call model for ML regressions, as distinct from infrastructure outages.

    - Who is on call for "the model is quietly wrong": scientists, ML engineers or SRE?
116. [Likely] Write the outline of a blameless postmortem for a model regression that passed every offline gate. Which process change would you push for?
117. [Likely] How do you design "safe mode" fallbacks (higher human-review thresholds, rule-based extraction, a previous model) that can be switched on per tenant in seconds?
118. [Likely] How do you test your rollback path regularly, so that it works on the day you need it?
119. [Likely] After an incident, how do you decide whether to add the failing cases to the golden set, a separate regression set or the training data?
    - How do you avoid overfitting the release gate to past incidents?
120. [Likely] A new extractor version writes outputs in a new schema to downstream entities and queues. You roll back the model, and consumers now see two schemas. Design releases so that model rollback never needs a data rollback.
    - When do you need a dual-write or versioned-output period, and how long should it last?
121. [Likely] [8] A newly enabled guardrail blocks 30% of legitimate invoice runs for one region [8]. Walk through mitigation: per-tenant kill switch, switching to log-only, threshold changes, and how you stop the same failure in the next policy release.

J. Principal-level integrative prompts

122. [Likely] You have one quarter and four engineers. Which three MLOps investments would most reduce model-related customer incidents next year? Defend the ranking with data you would collect.
123. [Likely] Design the platform so that a new document type can go from zero labels to a monitored production model in two weeks, mostly self-service by the customer.
    - Where must a UiPath scientist stay in the loop?
124. [Likely] Product wants weekly model releases; large regulated customers want quarterly changes with sign-off. Design a release policy that satisfies both.
125. [Likely] How would you unify the lifecycle for classic ML packages, document models and LLM agents on one platform without slowing any of them down?
    - What would you refuse to unify?
126. [Likely] Pick one MLOps decision from your past work that you would now make differently. What did it cost, and what would you design today?
127. [Likely] Build or buy: would you base UiPath's model registry, experiment tracking and evaluation store on open-source tools (for example MLflow or Kubeflow), a cloud vendor service or an in-house system, given that it must also run on-prem and air-gapped?
    - What would make you change the decision in two years?
128. [Likely] By 2026, agents can triage drift alerts, propose retraining and open release PRs. Which MLOps steps would you let an agent do on its own, which need human approval, and how do you audit the agent doing MLOps?

References

1. alirezadir, "Machine-Learning-Interviews" (AIMLInterviews), ML System Design chapter: a nine-step flow ending with prediction service, online testing (A/B, bandits, shadow, canary) and scaling, monitoring and updates (distribution shift, retraining cadence, active learning). Its 2026 GenAI notes say interviewers weigh cost, latency, guardrails and monitoring, and that "evaluation methodology is the new system design". General guide, not specific to UiPath (fetched). GitHub. https://raw.githubusercontent.com/alirezadir/Machine-Learning-Interviews/main/src/MLSD/ml-system-design.md
2. UiPath, Documents service source, `uipath-platform` package of `uipath-python`: choose exactly one of `version` or `tag` for non-pretrained projects; when a version is given the tag is ignored; unknown tags raise an error listing the published tags; validation actions block until a person completes them in Action Center; non-blocking IXP validation path (official; fetched). GitHub, 2025-2026. https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath-platform/src/uipath/platform/documents/_documents_service.py
3. UiPath, "Chat Models" documentation, `uipath-langchain-python`: models served through the LLM Gateway and subject to AI Trust Layer policy; some models unavailable in some regions (data-residency errors); dated model version IDs; usage consumes Agent Units (official; fetched). GitHub. https://raw.githubusercontent.com/UiPath/uipath-langchain-python/main/docs/chat_models.md
4. UiPath, Context Grounding service source, `uipath-platform` package: index creation from buckets and connectors with extraction strategy, embeddings and encryption options; scheduled indexers; ingestion conflicts rejected; search during ingestion; ephemeral indexes for Deep RAG and batch transform (official; fetched, first 100,000 characters read). GitHub. https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath-platform/src/uipath/platform/context_grounding/_context_grounding_service.py
5. UiPath, "Agent Evaluations" documentation, `uipath-python`: eval sets, output-based, trajectory-based and custom evaluators, `uipath eval` CLI with parallel workers, JSON output file and custom run IDs (official; fetched). GitHub. https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath/docs/eval/index.md
6. UiPath (Re:infer), `re` CLI README: the command-line interface for UiPath IXP; manages sources, datasets and comments; downloads raw messages with human-applied annotations (official; fetched). GitHub. https://raw.githubusercontent.com/reinfer/cli/master/README.md
7. UiPath, "Automation Suite" documentation, `uipath-python`: air-gapped environments need their own package index (devpi, Artifactory, Nexus and others); restricted environments can use Azure DevOps Artifacts (official; fetched). GitHub. https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath/docs/AutomationSuite.md
8. UiPath, "Guardrails" documentation, `uipath-python`: guardrails at tool, LLM and agent scope; PRE, POST and PRE_AND_POST stages; PII, harmful-content, prompt-attack, intellectual-property, LLM-as-judge and custom validators backed by the UiPath Guardrails API (Azure Content Safety); log, block and custom actions; Bring Your Own Guardrail configured under AI Trust Layer and still rolling out across rings (official; fetched). GitHub. https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath/docs/core/guardrails.md

9. UiPath, "Tracing" documentation, `uipath-python`: the `@traced()` decorator; traces shown per Orchestrator job and in Maestro; `hide_input`, `hide_output` and input/output processors to redact data (official; fetched). GitHub. https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath/docs/core/traced.md
10. UiPath, "CLI Reference" documentation, `uipath-python`: OAuth client-credentials flow for CI/CD; `UIPATH_URL` for Automation Suite; stable project GUID in `uipath.json`; `uipath pack` to `.nupkg` with `uv.lock` dependency locking by default and `--nolock`; `publish` to tenant or personal-workspace feeds; `push`, `pull`, `invoke`, `deploy` (official; fetched). GitHub. https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath/docs/cli/index.md
11. UiPath, "Release Policy" documentation, `uipath-python`: semantic versioning, release candidates (`X.Y.ZrcN`), deprecation for at least one minor release, migration guides for breaking changes (official; fetched). GitHub. https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath/docs/release_policy.md
12. UiPath, "LLM Judge Output Evaluators" documentation, `uipath-python`: judge scores from 0.0 to 1.0 with a justification, configurable judge model, temperature 0.0 by default (official; fetched). GitHub. https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath/docs/eval/llm_judge_output.md
13. UiPath, "Tool Call Order Evaluator" documentation, `uipath-python`: strict exact-sequence scoring (1 or 0) versus non-strict longest-common-subsequence partial credit (official; fetched). GitHub. https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath/docs/eval/tool_call_order.md
14. UiPath, "Human In The Loop" documentation, `uipath-langchain-python`: `interrupt()` models that suspend an agent until an Action Center task, escalation, process job, Deep RAG query, ephemeral index, batch transform or IXP extraction/validation completes (official; fetched). GitHub. https://raw.githubusercontent.com/UiPath/uipath-langchain-python/main/docs/human_in_the_loop.md
15. UiPath, "Environment Variables" documentation, `uipath-python`: folder context, and CLI usage telemetry enabled by default with an opt-out (`UIPATH_TELEMETRY_ENABLED=false`) (official; fetched). GitHub. https://raw.githubusercontent.com/UiPath/uipath-python/main/packages/uipath/docs/core/environment_variables.md